



**Bharat Ratna Indira Gandhi College of
Engineering, Solapur**

Master of Computer Application

SUBJECT:

FULL STACK DEVELOPMENT (MCAC301)

SEMESTER-III

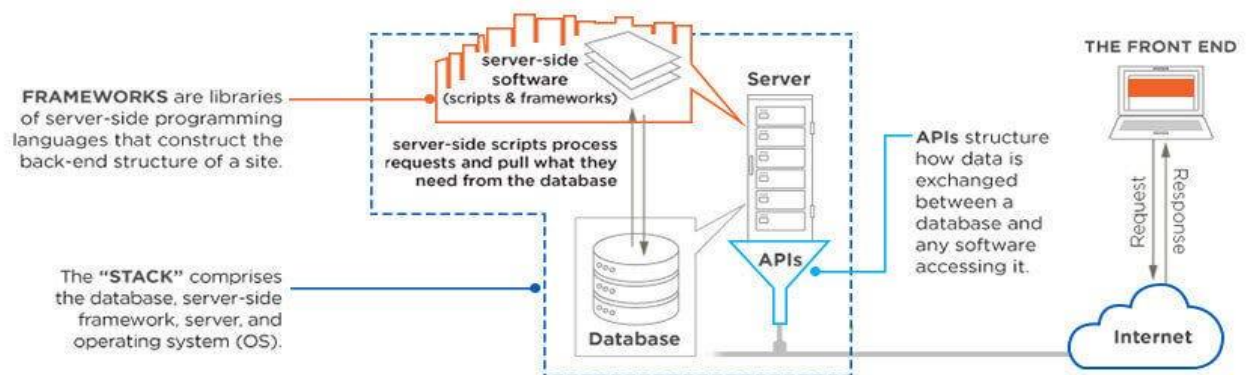
UNIT 2: BACK-END DEVELOPMENT

What is Backend Development? Backend Development refers to the server-side development of the web application. It is the part of the application where the server and database reside and the logics is built to perform operations. Programming languages for the backend are mainly Node. JS (for JavaScript), Django (for Python), Spring (Java), etc.

Backend Development Roadmap:

- Knowledge of Web Server
- Programming Languages and Their Frameworks
- Version Control System (Git)
- Knowledge of Web Security
- APIs (Application Programming Interface)
- Containerization & Testing
- Deployment
- Cloud Providers

BACK-END DEVELOPMENT & FRAMEWORKS IN SERVER SIDE SOFTWARE

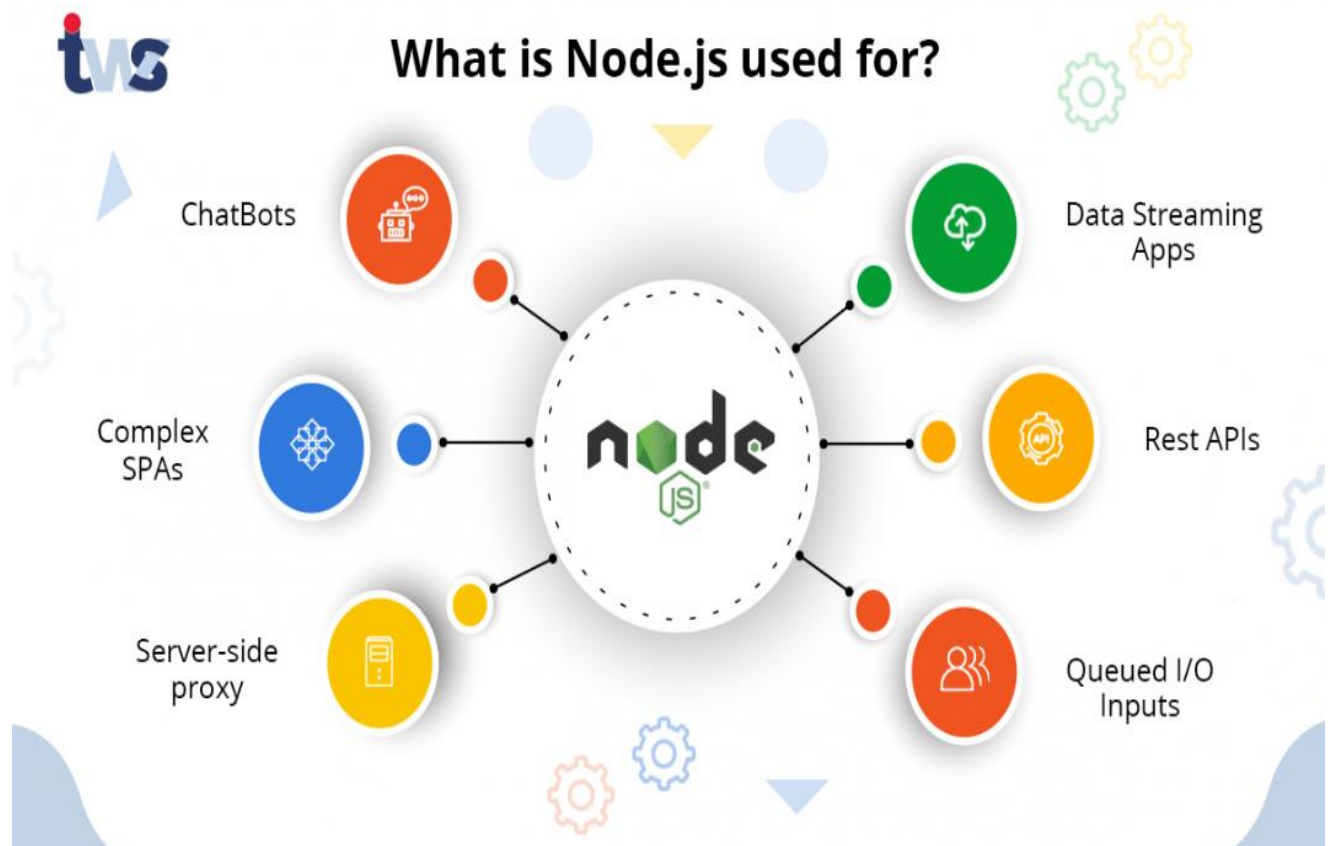


❖ Node.js:

Session 1: Introduction to Node.js

1.Introduction to Node.js

Node.js is not a programming language; it is a runtime environment allowing you to execute JavaScript code on the server side, outside a web browser. Built on the V8 JavaScript engine from Chrome, Node.js compiles JavaScript into machine code for efficient execution



- **Overview**

- I. Node.js is an open-source JavaScript runtime built on Chrome's V8 JavaScript engine.
- II. It enables developers to run JavaScript code on the server-side.
- III. Originated from the need for scalable network applications and real-time applications (e.g., chat applications, live updates).

2.What is Node.js?

Q. What Can Node.js Do?

- Node.js can generate dynamic page content
- Node.js can create, open, read, write, delete, and close files on the server
- Node.js can collect form data
- Node.js can add, delete, modify data in your database

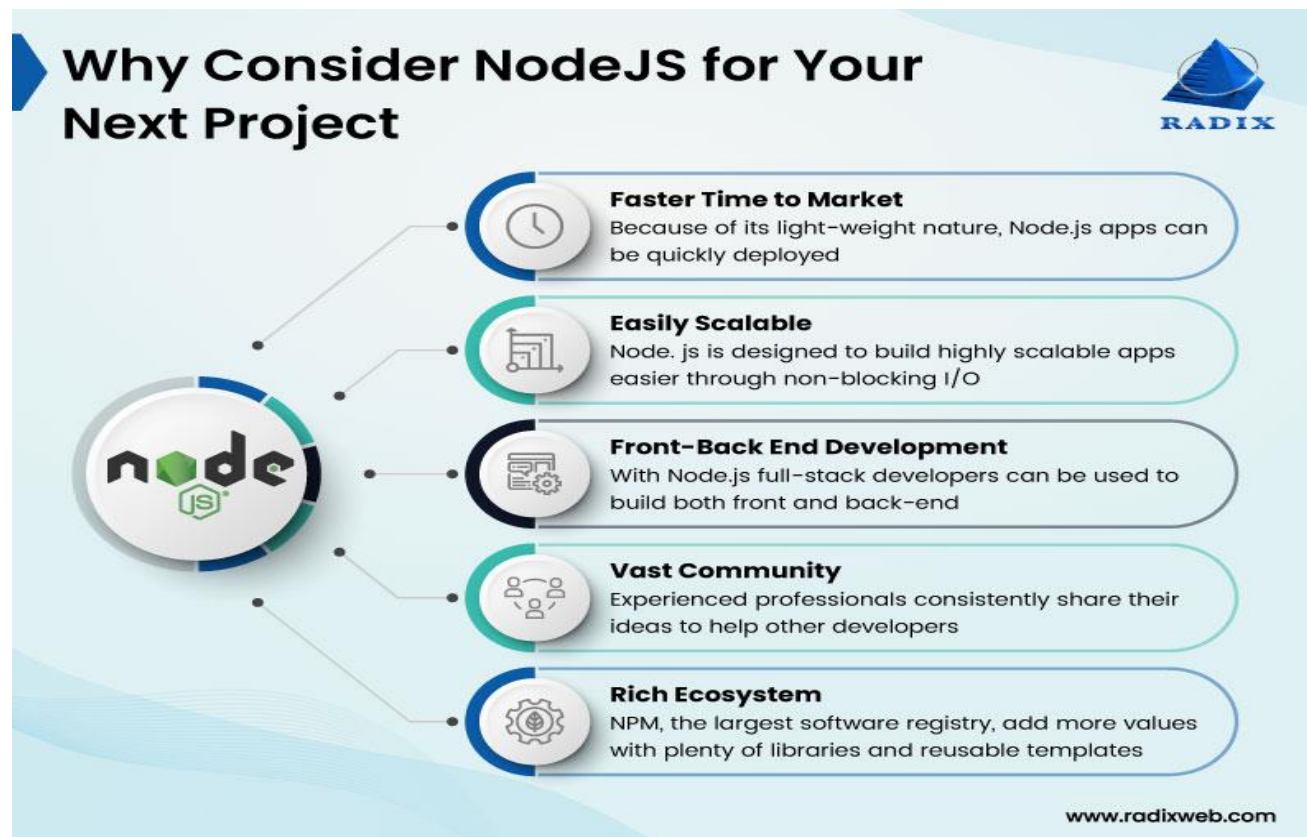


- **Purpose**

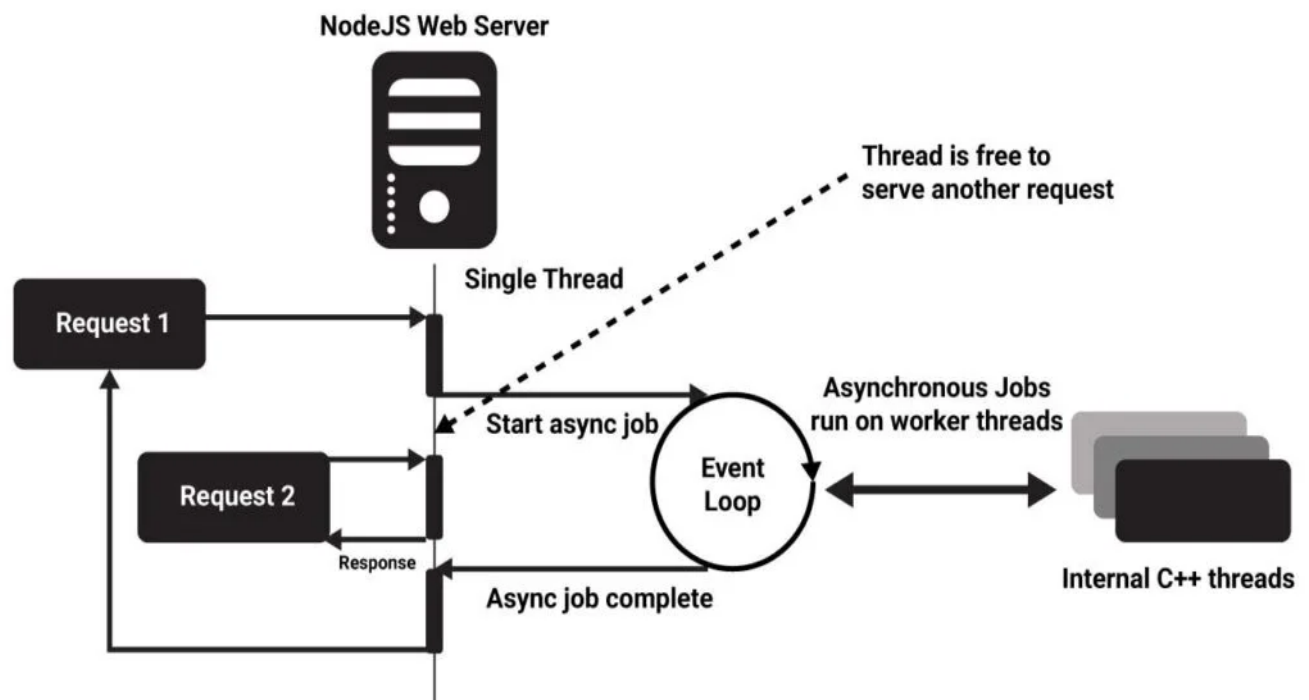
- I. Designed to build scalable network applications.
- II. Ideal for applications with high concurrency, such as real-time communication apps and APIs.

- **Event-Driven and Non-Blocking I/O**

- I. **Event-Driven:** Node.js uses events and callbacks to handle operations like reading files or making HTTP requests.
- II. **Non-Blocking, I/O:** Operations do not block the execution of other tasks. Instead, Node.js continues running other code and handles operations when they're completed.



3.Node.js Process Model



- **Single-Threaded Event Loop**

- I. The core of Node.js is a single-threaded event loop that handles asynchronous tasks.
- II. The event loop processes callbacks from asynchronous operations and manages concurrency.

- **Non-Blocking, I/O Operations**

- I. Node.js uses non-blocking I/O calls to handle tasks such as reading files or querying a database.
- II. This approach allows Node.js to perform multiple operations without waiting for each to complete sequentially.

- **Background Thread Pool**

- I. Node.js uses a thread pool managed by the libuv library for tasks that cannot be handled asynchronously (e.g., file system operations).
- II. This pool offloads tasks like file reads/writes, which are then processed without blocking the main event loop.

4. Advantages of Node.js

- **Speed**

- I. Node.js is fast due to the V8 JavaScript engine, which compiles JavaScript to native machine code.

- **Scalability**

- I. Node.js can handle a large number of simultaneous connections efficiently using its asynchronous, event-driven architecture.

- **Unified Language**

- I. JavaScript is used both on the client-side (in browsers) and server-side (Node.js), allowing for more streamlined development.

- **Ecosystem**

- I. A rich ecosystem of libraries and tools available through **npm** (Node Package Manager).

5.Traditional Web Server Model



- **Multi-Threaded Servers**

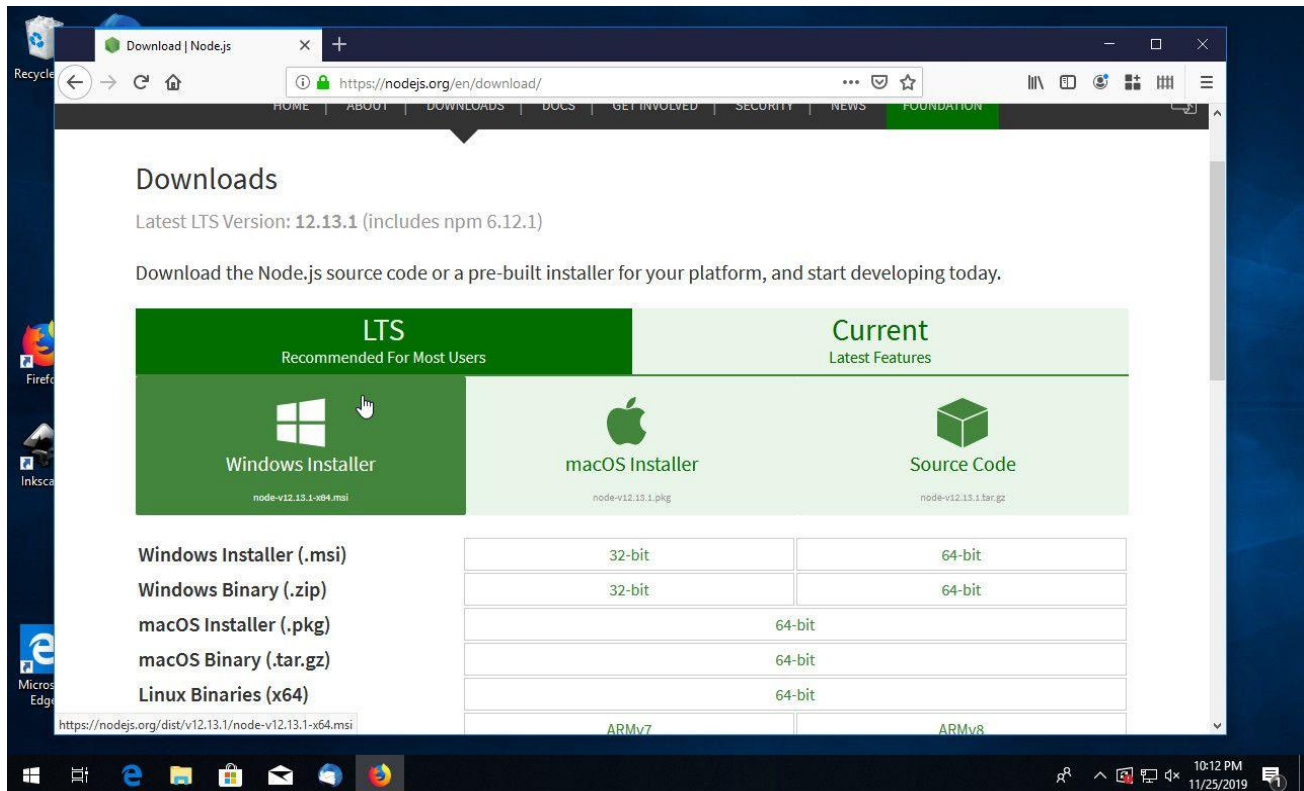
- I. Traditional servers (e.g., Apache) use a multi-threaded model where each incoming request spawns a new thread.
- II. This can lead to high memory consumption and reduced performance as the number of concurrent requests increases.

- **Limitations**

- I. High overhead with thread management and context switching.
- II. Scalability challenges as the number of requests increases.

Session 2: Setup Development Environment

1.Install Node.js on Windows



• Installation Steps

- I. Visit the [Node.js website](https://nodejs.org/en/download/) and download the Windows installer.
- II. Run the installer and follow the setup wizard.
- III. Verify installation by running `node -v` and `npm -v` in the command prompt.

```
C:\Windows\system32\cmd.exe
Microsoft Windows [Version 10.0.18363.418]
(c) 2019 Microsoft Corporation. All rights reserved.

C:\Users\win 10>node -v
v20.16.0

C:\Users\win 10>npm -v
10.8.1
```

- **Node Version Manager (nvm)**

- I. Install `nvm-windows` from [nvm-windows GitHub page](#).
- II. Use `nvm` to install and manage multiple versions of Node.js, allowing for easier switching between versions.

How to install Node.js

Node.js can be installed in different ways. This post highlights the most common and convenient ones. Official packages for all the major platforms are available at <https://nodejs.org/download/>.

One very convenient way to install Node.js is through a package manager. In this case, every operating system has its own. Other package managers for macOS, Linux, and Windows are listed in <https://nodejs.org/download/package-manager/>.

`nvm` is a popular way to run Node.js. It allows you to easily switch the Node.js version, and install new versions to try and easily rollback if something breaks. It is also very useful to test your code with old Node.js versions.

See <https://github.com/nvm-sh/nvm> for more information about this option.

2. Working in REPL, Node.js Console

- **REPL (Read-Eval-Print Loop)**

- I. Start REPL by running `node` in the command line.
- II. Use REPL to test JavaScript expressions, functions, and code snippets interactively.

III. The term REPL stands for Read Eval Print and Loop. It specifies a computer environment like a window console or a Unix/Linux shell where you can enter the commands and the system responds with an output in an interactive mode.

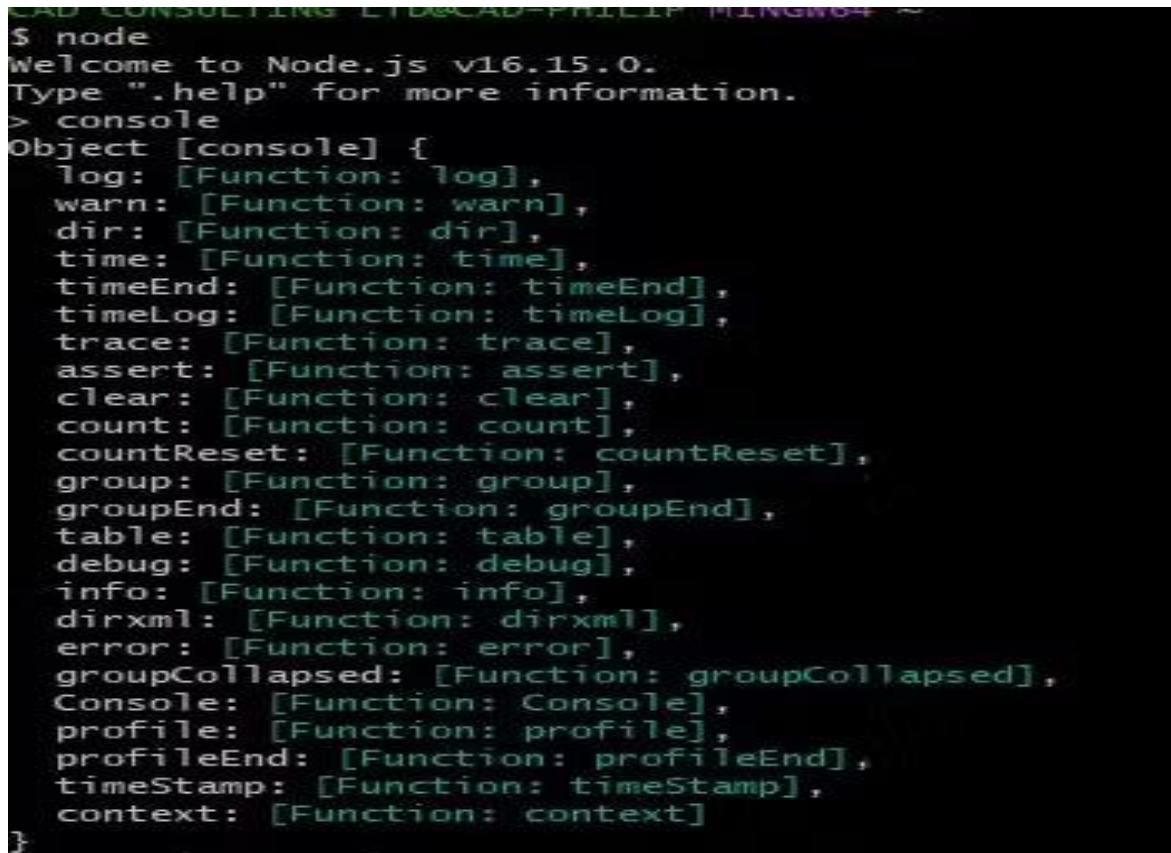
IV. **Read:** It reads user's input; parse the input into JavaScript data-structure and stores in memory.

Eval: It takes and evaluates the data structure.

Print: It prints the result.

Loop: It loops the above command until user press ctrl-c twice.

- **Console Use**



```
CAD CONSULTING LTD@CAD-PHILIP-MINGW64 ~
$ node
Welcome to Node.js v16.15.0.
Type ".help" for more information.
> console
Object [console] {
  log: [Function: log],
  warn: [Function: warn],
  dir: [Function: dir],
  time: [Function: time],
  timeEnd: [Function: timeEnd],
  timeLog: [Function: timeLog],
  trace: [Function: trace],
  assert: [Function: assert],
  clear: [Function: clear],
  count: [Function: count],
  countReset: [Function: countReset],
  group: [Function: group],
  groupEnd: [Function: groupEnd],
  table: [Function: table],
  debug: [Function: debug],
  info: [Function: info],
  dirxml: [Function: dirxml],
  error: [Function: error],
  groupCollapsed: [Function: groupCollapsed],
  Console: [Function: Console],
  profile: [Function: profile],
  profileEnd: [Function: profileEnd],
  timeStamp: [Function: timeStamp],
  context: [Function: context]
}
```

- I. Use `console.log()` to output values and debug code.
- II. Explore other console methods like `console.error()`, `console.warn()`, and `console.table()` for different types of logging.

Session 3: Node.js Modules

1.Functions, Buffer, Module

- **Functions**

Functions

- Functions are first class citizens in Node's JavaScript, similar to the browser's JavaScript.
- A function can have attributes and properties also.
- It can be treated like a class in JavaScript.

Example:

```
function Sum(x,y) {  
    console.log(x+y);  
}  
Sum(505,606);
```

- I. Fundamental building blocks in JavaScript for defining reusable code.
- II. Example:

```
function greet(name) {  
    return `Hello, ${name}!`;  
}
```

- **Buffer**

Buffers

- **Definition:** Buffer is an object property on Node's global object, which is heavily used in Node to deal with streams of binary data. It is globally available.
- JavaScript is Unicode friendly.
- But not dealing with binary data.
- Binary data is needed while handling with TCP or file system.
- For this NODEJS provide Buffer class.
- Buffer class store raw data similar as storing integer in array.
- But it will be stored at raw memory allocation outside of V8 heap.
- V8 is the default JavaScript engine which powers Node and Google Chrome

I. Used for handling raw binary data.

II. Example:

```
const buffer = Buffer.from('Hello World');  
  
console.log(buffer.toString('utf8')); // Outputs: Hello World
```

- **Module**

Node.js Modules

Read about what are Node.js modules, Read [here](#) for more

List of Node.js Modules

- ☐ fs (File System module)
- ☐ path (Directories and paths in node.js)
- ☐ os (Operating System modules)
- ☐ HTTP and HTTPS module
- ☐ util (Formatting strings and debugging)
- ☐ querystring (Parsing and formatting URL query)
- ☐ crypto (Hashing and HMAC, ciphering)
- ☐ events (Emit and listen events)
- ☐ stream (Reading and writing streams of data)
- ☐ zlib (Compression & Decompression)
- ☐ dns (DNS lookup and name resolution functions)
- ☐ buffer (work with binary data and large files)
- ☐ url (Parsing and formatting URLs)
- ☐ net (Network capabilities creating server and clients)
- ☐ tls (Implement Transport and Security layers protocols)
- ☐ child_process (Execution of external commands)
- ☐ process (Process related tasks)
- ☐ readline (reading lines from readable streams)
- ☐ timers (Scheduling and Canceling of code)

- I. Encapsulate code into reusable units, making it easier to manage and maintain.

- II. Modules can export functions, objects, or values for use in other files.

2.Core Modules

- **Built-in Modules**

Core Modules

Node.js has many built-in modules that are part of the platform and come with [Node.js](#) installation. These modules can be loaded into the program by using the **required** function.

Syntax:

```
const module = require('module_name');
```

The `require()` function will return a JavaScript type depending on what the particular module returns. The following example demonstrates how to use the Node.js `http` module to create a web server.

JavaScript

```
const http = require('http');
http.createServer(function (req, res) {
  res.writeHead(200, { 'Content-Type': 'text/html' });
  res.write('Welcome to this page!');
  res.end();
}).listen(3000);
```

- I. Common modules: `fs` (file system), `http` (HTTP server), `path` (file paths), `os` (operating system), `events` (event handling).
- II. Example of using `fs` module:

```
const fs = require('fs');
```

```
fs.readFile('file.txt', 'utf8', (err, data) => {  
  if (err) throw err;  
  console.log(data);  
});
```

3.Local Modules

- **Creating Custom Modules**

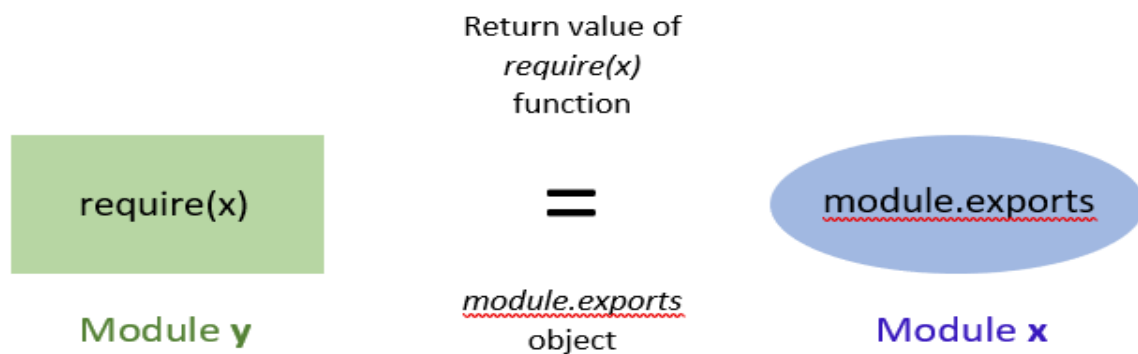
I. Define custom modules in separate files and use `module.exports` to expose functionality.

II. Example (math.js):

```
function add(a, b) {  
  return a + b;  
}  
  
module.exports = { add };
```

III. Import and use local modules with `require()`:

```
const math = require('./math');  
  
console.log(math.add(2, 3)); // Outputs: 5
```



4.Module Types

- **CommonJS Modules**

- I. Traditional module system used by Node.js.
- II. Use `module.exports` to export and `require()` to import.

- **ES6 Modules**

Import Operation	Syntax
Import entire module	<code>import * as name from 'module-name'</code>
Import default export from a module	<code>import name from 'module-name'</code>
Import single export from a module	<code>import { name } from 'module-name'</code>
Import multiple exports from a module	<code>import { nameOne , nameTwo } from 'module-name'</code>
Import a module for side effects only	<code>import './module-name'</code>

- I. Newer module system supported in recent versions of Node.js.
- II. Use `export` and `import` statements.
- III. To use, either rename files to `.mjs` or set `"type": "module"` in `package.json`.

CommonJS		ES Modules
Synchronous	Module Loading	Asynchronous
'require'/'exports'	Import/Export	'import'/'export'
Cached	Caching	Not cached by default
Server-side	Use Case	Client-side and server-side
Not supported	Dynamic Import	Supported

5.Module Exports

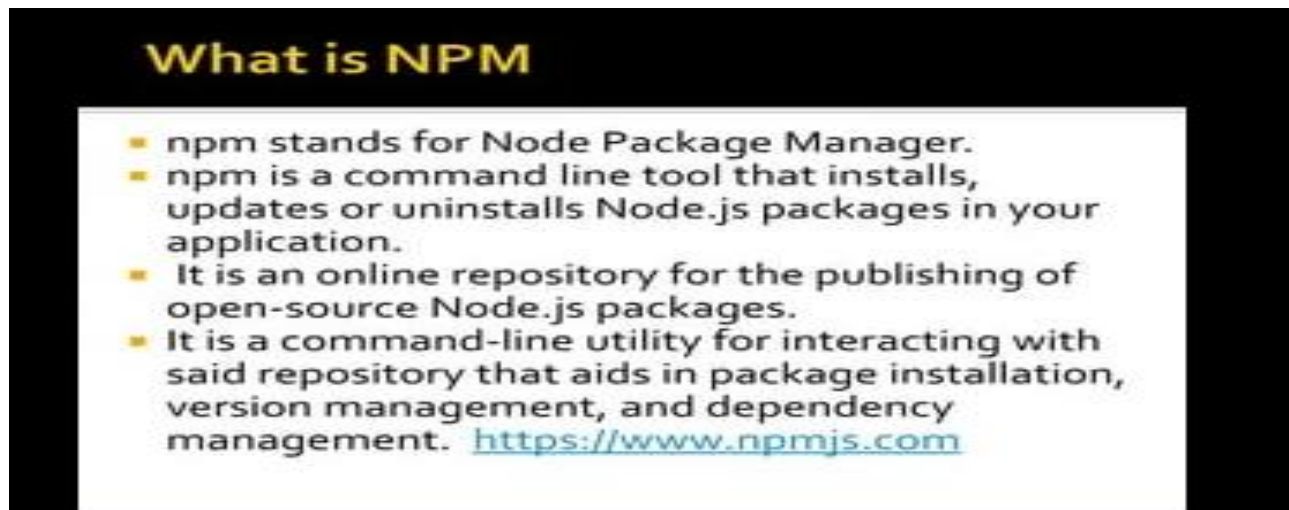
- **Exporting**

- I. Export specific functions or objects to make them available to other files.
- II. Example:

```
module.exports = {  
  greet: function(name) { return `Hello, ${name}!`; }  
};
```

Session 4: Node Package Manager (NPM)

1.What is NPM?



- **Overview**

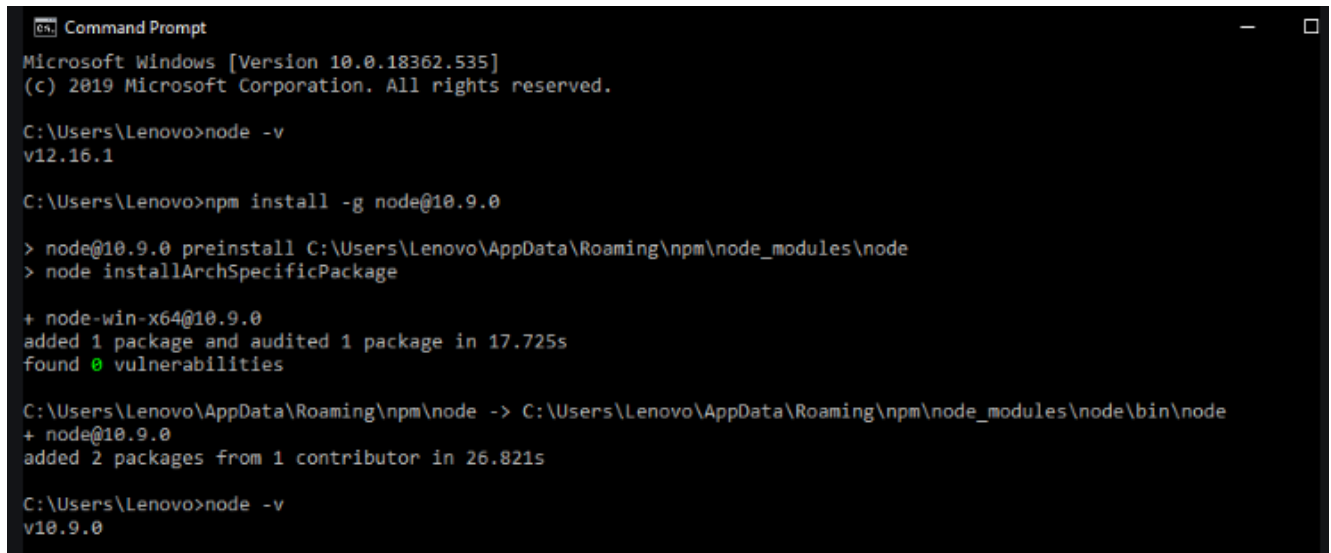
- I. NPM (Node Package Manager) is the default package manager for Node.js.
- II. Manages JavaScript libraries and tools that Node.js projects depend on.

Full Stack Development [Unit-II]

- **Features**

- I. Allows for easy installation, version management, and sharing of packages.

2.Installing Packages Locally



```
Command Prompt
Microsoft Windows [Version 10.0.18362.535]
(c) 2019 Microsoft Corporation. All rights reserved.

C:\Users\Lenovo>node -v
v12.16.1

C:\Users\Lenovo>npm install -g node@10.9.0

> node@10.9.0 preinstall C:\Users\Lenovo\AppData\Roaming\npm\node_modules\node
> node installArchSpecificPackage

+ node-win-x64@10.9.0
added 1 package and audited 1 package in 17.725s
found 0 vulnerabilities

C:\Users\Lenovo\AppData\Roaming\npm\node -> C:\Users\Lenovo\AppData\Roaming\npm\node_modules\node\bin\node
+ node@10.9.0
added 2 packages from 1 contributor in 26.821s

C:\Users\Lenovo>node -v
v10.9.0
```

- **Local Installation**

- I. Run `npm install <package-name>` to install a package in the `node_modules` directory.

- II. Example:

```
npm install express
```

- III. Installed packages are listed in `package.json` under "dependencies".

3.Adding Dependency in `package.json`

- **Manual Editing**

Full Stack Development [Unit-II]

- I. Add dependencies directly to the `package.json` file.
- II. Example:

```
"dependencies": {  
  "express": "^4.17.1",  
  "mongoose": "^5.10.9"  
}
```

- **Automatic Updates**

- I. Run `npm install` to install dependencies listed in `package.json`.

4.Installing Packages Globally

What is npm global install?

In global mode (ie, with `-g` or `--global` appended to the command), it **installs the current package context (ie, the current working directory) as a global package**. By default, npm install will install all modules listed as dependencies in package.

- **Global Installation**

- I. Use the `-g` flag to install packages globally for use across projects.
- II. Example:

```
npm install -g nodemon
```

- III. Global packages are usually stored in a system-wide directory

5.Updating Packages

^	updates to minor and patch releases only.	^0.13.0: 0.13.1, 0.14.0
~	updates to patch releases only.	~0.13.0: 0.13.1 (not 0.14.0)
>	updates to any version greater than the one specified.	>0.13.0: 0.13.1, 0.14.1, 1.1.1
<	updates to any version less than the one specified.	<3.0.0: 2.0.0, 2.9.0
>=	any version greater than or equal to the one specified.	>=3.0.0: 3.0.0, 4.1.0
<=	any version less than or equal to the one specified.	<=3.0.0: 3.0.0, 4.1.0
=	only the exact specified version.	=3.0.0: 3.0.0, (not 3.0.1)
-	range of versions.	1.0.0 - 1.10.10: 1.5.0 (not 1.11.0)
 	combination of versions.	<2.1.0 >2.6.0: 2.0.1, 3.1.0
x.x.x	only the specified version (no symbol).	=3.0.0: 3.0.0, (not 3.0.1)

• Updating Packages

- I. Run `npm update` to update packages to their latest versions based on `package.json`.
- II. Check outdated packages with `npm outdated`.
- III. Update specific packages with `npm update <package-name>`.

Session 5: Creating a Web Server

```
JS server.js > ...
1  const http = require("http");
2  const port = 3000;
3  const hostname = "localhost";
4
5  const server = http.createServer((req, res) => {
6    res.status = 200;
7    res.end("suh dude");
8  });
9
10 server.listen((hostname, port), () => {
11   console.log(`server listening on port: ${port}`);
12 });
```

1.Creating Web Server

- **Basic HTTP Server**

I. Use the `http` module to create a basic web server.

II. Example:

```
const http = require('http');

const server = http.createServer((req, res) => {
  res.writeHead(200, {'Content-Type': 'text/plain'});
  res.end('Hello World\n');
});

server.listen(3000, () => {
  console.log('Server running at http://127.0.0.1:3000/');
});
```

2.Handling HTTP Requests

- **Request Parsing**

I. Extract information from `req` object such as URL, method, and headers.

II. Example:

```
server.on('request', (req, res) => {
  console.log(req.method); // GET, POST, etc.
  console.log(req.url);    // Requested URL
});
```

3.Sending Responses

- **Response Handling**

- I. Use `res.writeHead()` to set HTTP status code and headers.
- II. Use `res.end()` to send the response body.
- III. Example:

```
res.writeHead(200, {'Content-Type': 'text/html'});  
res.end('<h1>Hello World!</h1>');
```

Session 6: File System

1.fs.readFile

- **Reading Files Asynchronously**

```
var http = require('http');  
var fs = require('fs');  
http.createServer(function (req, res) {  
  //Open a file on the server and return its content:  
  fs.readFile('demofile1.html', function(err, data) {  
    res.writeHead(200, {'Content-Type': 'text/html'});  
    res.write(data);  
    return res.end();  
  });  
}).listen(8080);
```

- I. Example of reading a file asynchronously:

```
const fs = require('fs');  
fs.readFile('example.txt', 'utf8', (err, data) => {
```

```
        if (err) throw err;

        console.log(data);
    });
```

2. Writing a File

- **Writing Files Asynchronously**

```
var fs = require('fs');

//Replace the file with a new one:
fs.writeFile('mynewfile3.txt', 'This is my text.', function (err) {
    if (err) throw err;
    console.log('Replaced!');
});
```

- I. Example of writing data to a file:

```
fs.writeFile('example.txt', 'Hello World!', (err) => {

    if (err) throw err;

    console.log('File saved!');

});
```

3. Writing a File Asynchronously

- **Non-Blocking Operations**

- I. Ensure file operations do not block the main thread.
- II. Use callback functions to handle completion of file operations.

4. Opening a File

- **Opening Files**

I. Use `fs.open()` to obtain a file descriptor.

II. Example:

```
fs.open('example.txt', 'r', (err, fd) => {  
  if (err) throw err;  
  console.log('File opened with fd:', fd);  
});
```

5.Deleting a File

- **Deleting Files**

I. Example of deleting a file:

```
fs.unlink('example.txt', (err) => {  
  if (err) throw err;  
  console.log('File deleted!');  
});
```

6.Other IO Operations

- **Additional Methods**

I. `fs.appendFile()` to append data to a file.

II. `fs.rename()` to rename or move files.

III. `fs.stat()` to get file statistics (size, modification time).

Session 7: Debugging Node.js Application

1.Core Node.js Debugger

- **Node.js Debugging Tools**

I. Use `node inspect <filename>` to start the built-in debugger.

II. Example:

```
node inspect app.js
```

- **Debugging with Chrome DevTools**

I. Use the `--inspect` flag to enable debugging with Chrome DevTools:

```
node --inspect-brk app.js
```

II. Open `chrome://inspect` in Google Chrome to start debugging session.

Session 8: Events

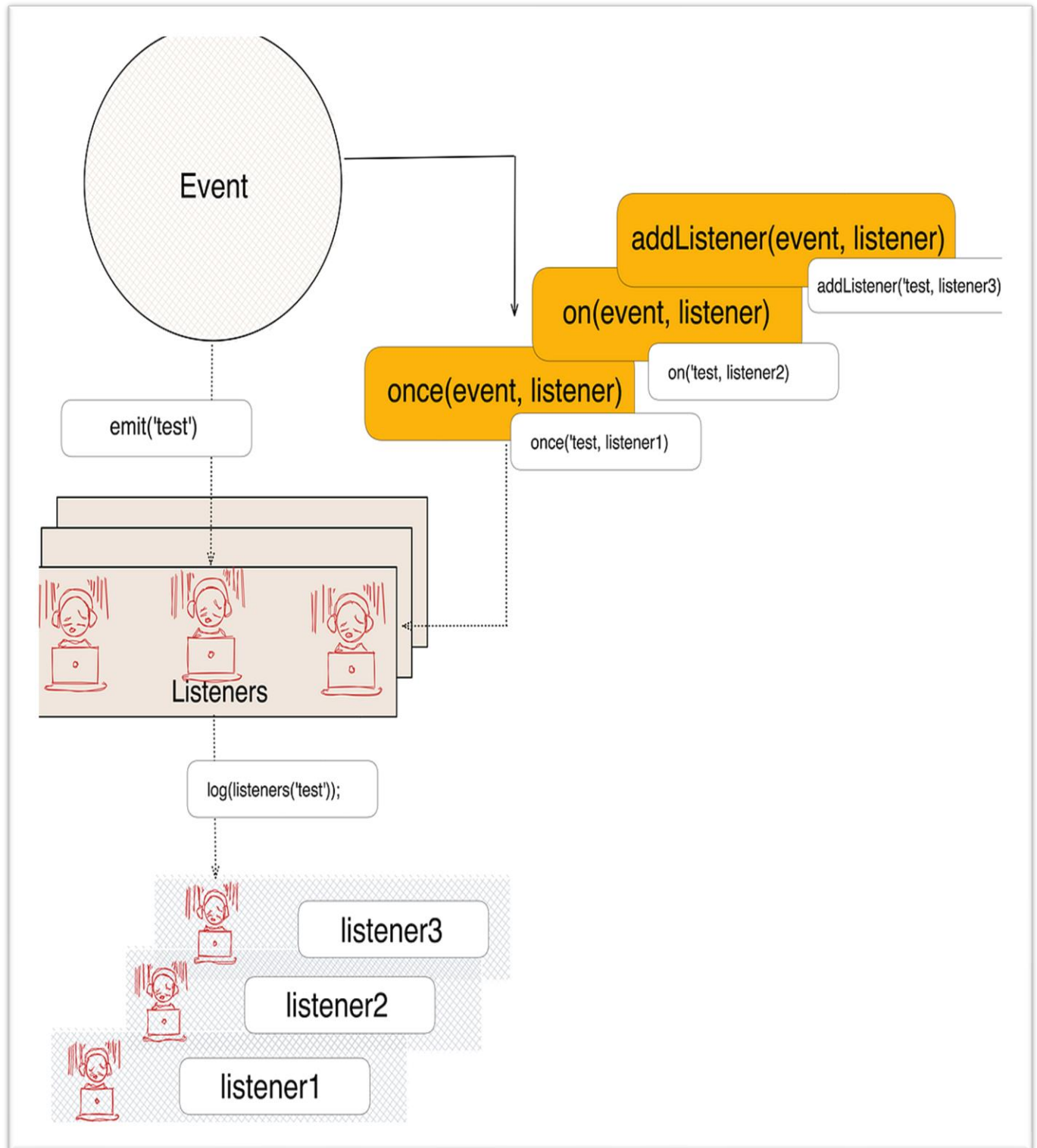
1.Event Emitter Class

- **Using EventEmitter**

I. Create and handle custom events using `EventEmitter` class:

```
const EventEmitter = require('events');  
  
class MyEmitter extends EventEmitter {}  
  
const myEmitter = new MyEmitter();  
  
myEmitter.on('event', () => {
```

```
console.log('An event occurred!');  
});myEmitter.emit('event');
```



2.Returning Event Emitter

- Custom Modules

- I. Return instances of `EventEmitter` from custom modules to handle specific events.

3.Inheriting Events

- Creating Custom Event Emitters

Creating an Event Emitter

To create an event emitter, we need to create an instance of the event emitter instance from the `events` module in NodeJS.

Note: I'm using Typescript instead of JavaScript.

```
import { EventEmitter } from 'events';  
const eventEmitter = new EventEmitter();
```

- I. Extend `EventEmitter` to create specialized event handling:

```
class MyClass extends EventEmitter {  
  constructor() {  
    super();  
  }  
}
```

Session 9: Express.js

- Express.js is a fast, flexible and minimalist web framework for Node.js. It's effectively a tool that simplifies building web applications and APIs using JavaScript on the server side. Express is an open-source that is developed and maintained by the Node.js foundation.
- Express.js offers a robust set of features that enhance your productivity and streamline your web application. It makes it easier to organize your application's functionality with middleware and routing. It adds helpful utilities to Node HTTP objects and facilitates the rendering of dynamic HTTP objects.

Q. Why learn Express?

- Express is a user-friendly framework that simplifies the development process of Node applications. It uses JavaScript as a programming language and provides an efficient way to build web applications and APIs.
- With Express, you can easily handle routes, requests, and responses, which makes the process of creating robust and scalable applications much easier. Moreover, it is a lightweight and flexible framework that is easy to learn and comes loaded with

middleware options. Whether you are a beginner or an experienced developer, Express is a great choice for building your application.

Getting Started Express

1. Installation: Install Express using npm:

```
npm install express
```

2. Basic Example of an Express App:

Node



```
const express = require('express');
const app = express();

// Define routes and middleware here
// ...

const PORT = process.env.PORT || 3000;
app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});
```

1. Configuring Routes

- **Setting Up Routes**

- I. Define routes using `express.Router()`:

```
const express = require('express');  
const router = express.Router();  
router.get('/', (req, res) => {  
    res.send('Home Page');  
});
```

- II. Register routes with the main application:

```
const app = express();  
app.use('/', router);
```

2. Working with Express

- **Middleware and Routing**

- I. Use `app.use()` to apply middleware for processing requests.

- II. Example of middleware:

```
app.use((req, res, next) => {  
    console.log('Request URL:', req.url);  
    next();  
});
```

Session 10: Serving Static Resources

Serving static files in Express

To serve static files such as images, CSS files, and JavaScript files, use the `express.static` built-in middleware function in Express.

The function signature is:

```
express.static(root, [options])
```

The `root` argument specifies the root directory from which to serve static assets. For more information on the `options` argument, see [express.static](#).

For example, use the following code to serve images, CSS files, and JavaScript files in a directory named `public`:

```
app.use(express.static('public'))
```

Now, you can load the files that are in the `public` directory:

```
http://localhost:3000/images/kitten.jpg
http://localhost:3000/css/style.css
http://localhost:3000/js/app.js
http://localhost:3000/images/bg.png
http://localhost:3000/hello.html
```

Express looks up the files relative to the static directory, so the name of the static directory is not part of the URL.

To use multiple static assets directories, call the `express.static` middleware function multiple times:

```
app.use(express.static('public'))
app.use(express.static('files'))
```

1. Serving Static Files

- **Static Middleware**
 - a. Serve static files like HTML, CSS, and JavaScript with `express.static`:

Full Stack Development [Unit-II]

```
app.use(express.static('public'));
```

- b. Place static assets in the `public` directory.

2. Working with Middleware

- **Custom Middleware**

- I. Implement middleware for tasks like logging, authentication, or request modification:

```
app.use((req, res, next) => {  
    console.log('Request received at:', new Date());  
    next();  
});
```

Session 11: Database Connectivity

```
20      sql.connect(config)
21      .then(pool => {
22          return pool.request().query(`**Check if row exists**`)
23      })
24      .then((recordset) => {
25          console.log(recordset);
26          if (recordset.length == 0) {
27              return true;
28          }
29          else {
30              return false;
31          }
32      })
33      .then((exists) => {
34          if (exists) {
35              return pool.request().query(`**Update the existing row**`);
36          }
37          else {
38              return pool.request().query(`**insert a new row**`)
39          }
40      })
41      .then(() => {
42          sql.close();
43      });
```

1.Connecting String, Configuring

- **Database Clients**

- I. Connect to databases using clients like `mongoose` for MongoDB or `pg` for PostgreSQL.
- II. Example (MongoDB with Mongoose):

```
const mongoose = require('mongoose');

mongoose.connect('mongodb://localhost/mydatabase', {
    useNewUrlParser: true,
```

```
        useUnifiedTopology: true  
    });
```

2.Working with Select Command

- **Querying Data**

- I. Perform data retrieval operations:

```
const users = await User.find();
```

3.Updating Records

- **Modifying Data**

- I. Update records using database commands:

```
await User.updateOne({ _id: id }, { $set: { name: 'New Name' }  
});
```

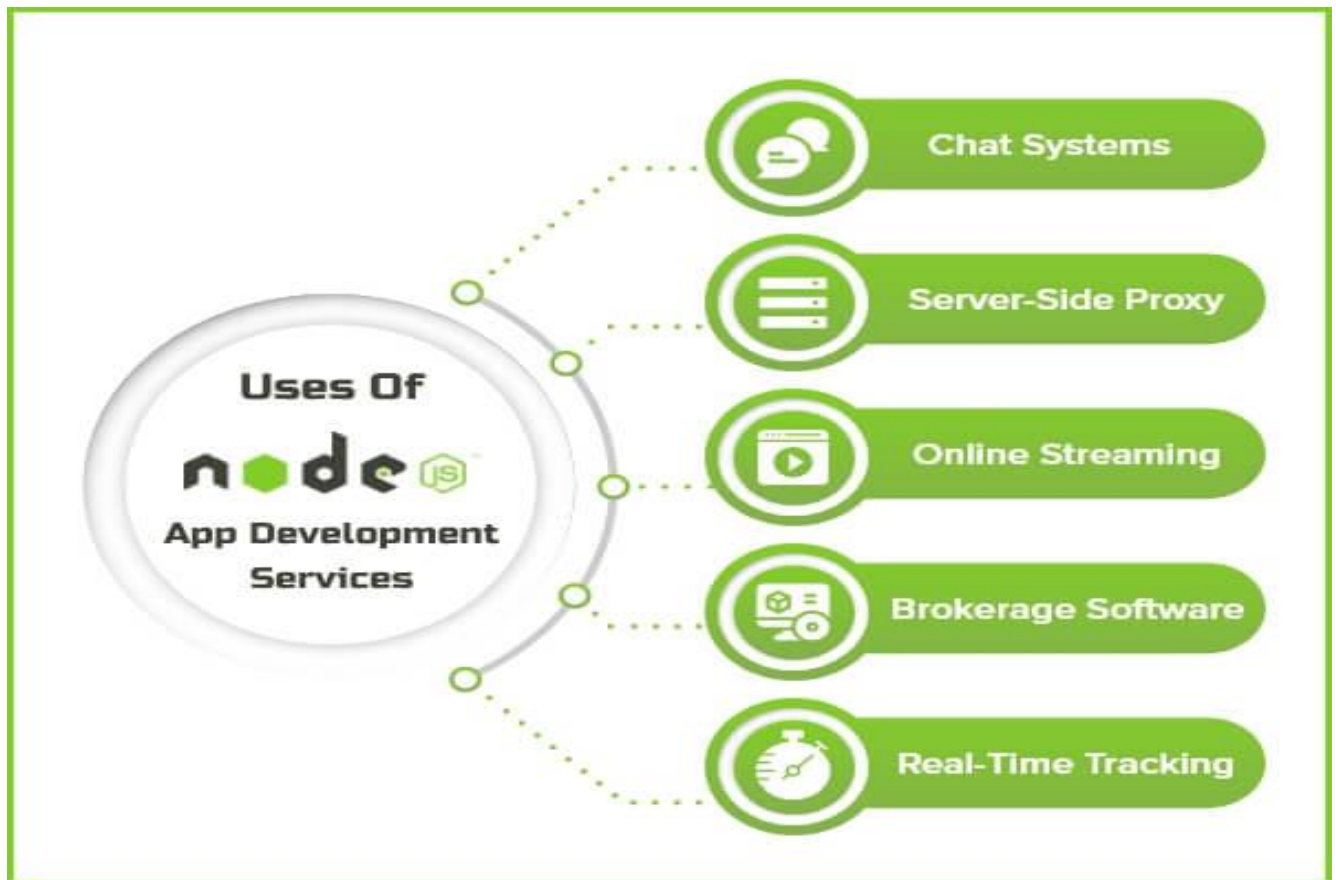
4.Deleting Records

- **Removing Data**

- I. Delete records from the database:

```
await User.deleteOne({ _id: id });
```

Session 12: Project Development



1. Project Development Using Node.js

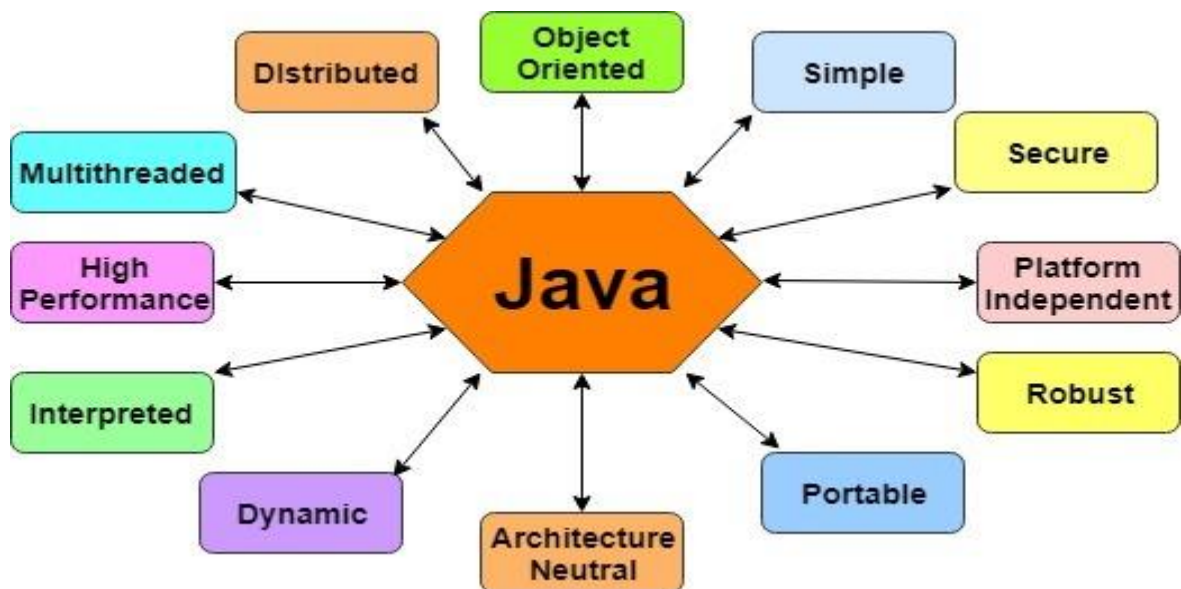
- **Building a Complete Application**

- I. Combine knowledge of server creation, routing, file handling, database interaction, and debugging.
- II. Develop a project that includes features such as user authentication, data storage, API integration, and real-time updates.
- III. Example project ideas: a blog platform, a chat application, or a task management system.

1. Introduction to Java

1.1 Overview of Java Programming Language

- I. **Java** is a high-level, object-oriented programming language developed by Sun Microsystems (now Oracle) in the mid-1990s. It is designed to be platform-independent at both the source and binary levels.



- II. Java is a class-based, object-oriented programming language that is designed to have as few implementation dependencies as possible. It is intended to let application developers write once, and run anywhere (WORA), meaning that compiled Java code can run on all platforms that support Java without the need for recompilation. Java was first released in 1995 and is widely used for developing applications for desktop, web, and mobile

devices. Java is known for its simplicity, robustness, and security features, making it a popular choice for enterprise-level applications.

- III. JAVA was developed by James Gosling at Sun Microsystems Inc in the May 1995 and later acquired by Oracle Corporation. It is a simple programming language. Java makes writing, compiling, and debugging programming easy. It helps to create reusable code and modular programs. Java is a class-based, object-oriented programming language and is designed to have as few implementation dependencies as possible. A general-purpose programming language made for developers to write once run anywhere that is compiled Java code can run on all platforms that support Java. Java applications are compiled to byte code that can run on any Java Virtual Machine. The syntax of Java is similar to c/c++.
- IV. **Example:** Java code can run on any device with the Java Virtual Machine (JVM), making it a "write once, run anywhere" language.

What Makes **Java** Most Popular Programming Language ?



- Java Is **Simple** .
- Java Is **Powerful And Secure** .
- Java Is **Object Oriented** .
- Java Is **Platform Independent**.
- Java Is **Versatile**.
- Java Supported By Huge **Community**.

www.learncomputerscienceonline.com

1.2 History and Evolution of Java

- I. Java was developed by James Gosling and his team at Sun Microsystems. The first version was released in 1996. It has undergone several updates, with major versions including Java 1.0, Java 2 (J2SE), Java 5 (1.5), Java 6, and Java 8. Each version introduced new features and improvements.
- II. **Example:** Java 8 introduced lambda expressions and the Stream API, enhancing functional programming capabilities.

1.3 Features and Advantages of Java

- I. Java is known for its simplicity, object-orientation, platform independence, robustness, and security features. It supports multithreading and networking, and has automatic memory management through garbage collection.
- II. **Example:** Java's exception handling model helps in creating robust applications by catching and managing runtime errors.

2. Java Development Environment

2.1 Installation of Java Development Kit (JDK)

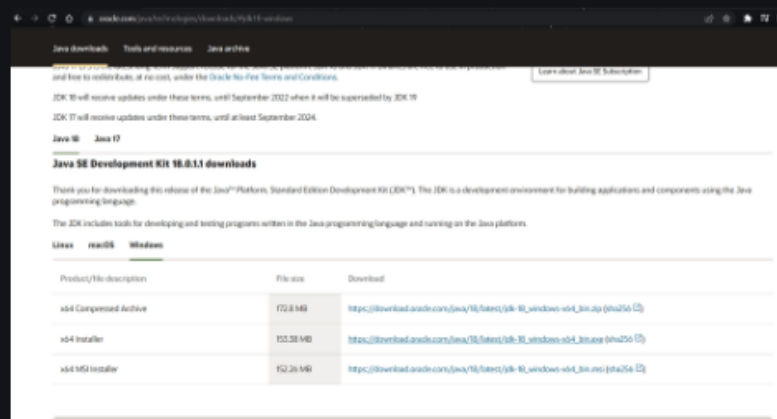
- I. The JDK includes the Java Runtime Environment (JRE) and development tools such as the compiler (`javac`), interpreter (`java`), and debugger.
- II. **Example:** Download the JDK from Oracle's website, follow the installation instructions, and set up environment variables like `JAVA_HOME` and `PATH`.

2.2 Setting up Integrated Development Environments (IDEs)

- IDEs provide tools for coding, debugging, and testing. Popular IDEs include Eclipse and IntelliJ IDEA.
- **Example:** Install Eclipse, configure it with the JDK, and use its built-in features to create, compile, and run Java applications.

Step 1: Download and Install Java Development Kit (JDK)

The very first step is to download the **Oracle Java Development Kit (JDK)** from the Official Oracle Website. For that, Head over to the [Official Website](https://www.oracle.com/in/java/technologies/javase-downloads.html).

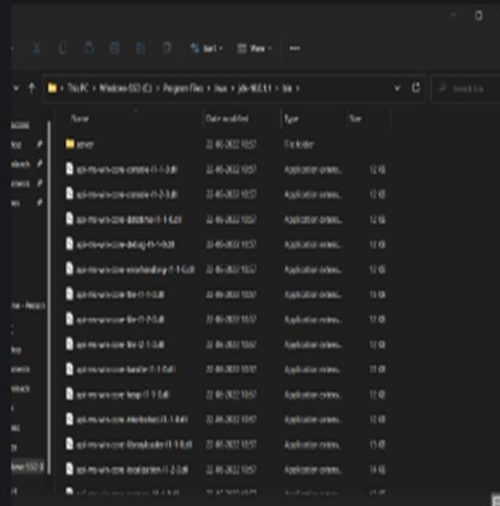


You need to identify your system specifications to choose the Product/file description. The website will contain the latest version for your corresponding system. For Windows, we'll be downloading the latest **x64 Installer of Java SE Development Kit 18**. After the download is complete, proceed to install the JDK by following the bootstrapped steps.

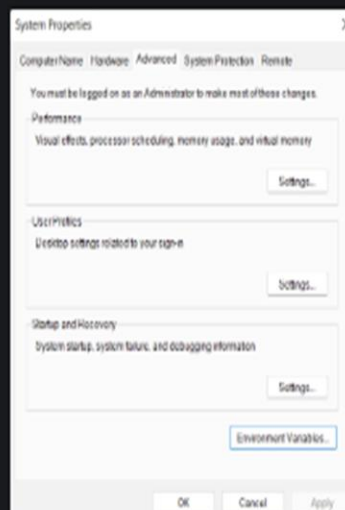


Step 2: Configure Environment Variables

After the installation is complete, we have to configure environment variables to notify the system about the directory in which JDK files are located. Proceed to `C:\Program Files\Java\jdk-{YOUR_JDK_VERSION}\bin` (replace {-} with your JDK version)

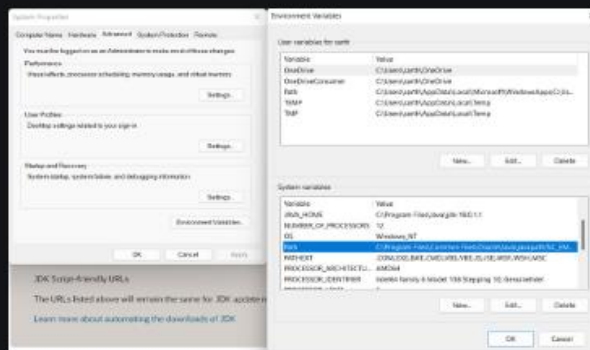


To set the Environment Variables, you need to search Environment Variables in the Task Bar and click on "Edit the system environment variables".

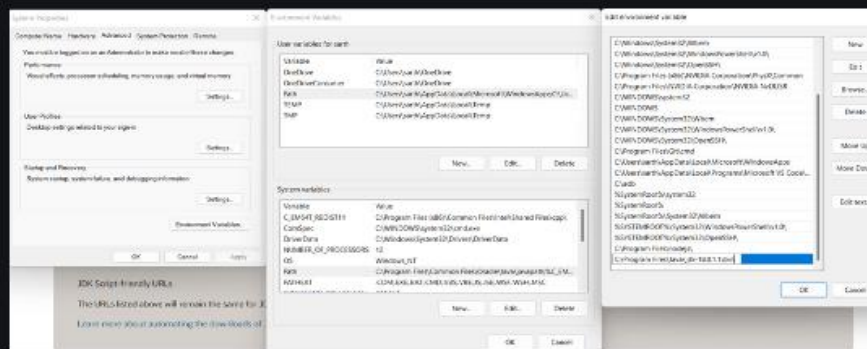


Full Stack Development [Unit-II]

Under the Advanced section, Click on "Environment Variables".



Under **System variables**, select the **"Path"** variable and click on **"Edit"**. Click on **"New"** then paste the Path Address i.e. **C:\Program Files\Java\jdk-(YOUR_JDK_VERSION)\bin**. Click on **"OK"**.

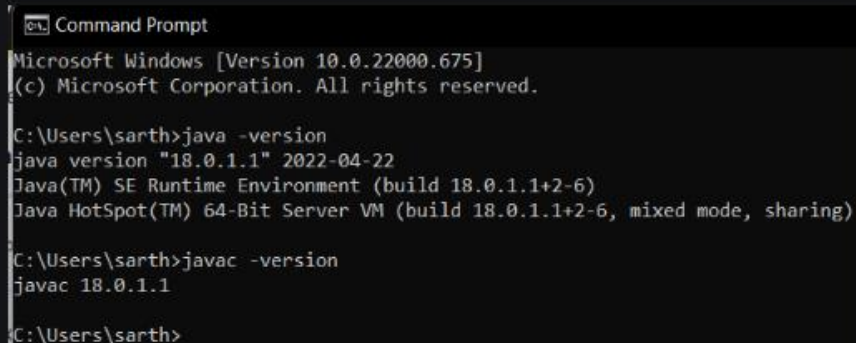


Now, in the **Environment Variables** dialogue, under **System variables**, click on **"New"** and then under **Variable name:** **JAVA_HOME** and **Variable value:** paste address i.e. **C:\Program Files\Java\jdk-[YOUR_JDK_VERSION]**. Click on **OK => OK => OK**.

Step 3: Check the Java Version

Open Command Prompt and enter the following commands

```
java -version
javac -version
```



3. Basic Syntax

3.1 Structure of a Java Program

I. A basic Java program consists of classes and methods. The `main` method is the entry point of any standalone Java application.

II. **Example:**

```
public class HelloWorld {  
    public static void main(String[] args) {  
        System.out.println("Hello, World!");  
    }  
}
```

3.2 Data Types, Variables, and Constants

Java Primitive Datatypes

Datatype	Meaning	Size in Bytes	Min Value	Max Value	Sig. Digits
int	Integer	4	-2 billion	2 billion	
double	Floating Point	8	-1.8×10^{308}	1.8×10^{308}	15
char	Character	2	Any Unicode character		
boolean	Logical	1	false	true	

String Objects

A **string** in Java is not a primitive data type. It is an object from the system defined class `String`.

- Examples of **string** constants:
"watermelon", "fig", "\$%&*^%!!", "354", " " (space), "" (null string)

The string declaration

```
String item = "apple";
```

is short for the object declaration

```
String item = new String("apple");
```

Java Constants

A **constant** is a value that cannot be changed.

- Examples of **integer** constants:
354, -37, 0, 246543 (no commas).
- Examples of **floating point** constants:
4.642, -0.034, 5.98e24 (mass of earth), 9.11e-31 (mass of electron)
- Examples of **character** constants:
'A', 'y', '8', '*', ' ' (space), '\n' (new line).

A character can be stored in memory using its **ASCII Code**. ASCII means American System of Computer Information Interchange. |

Actually, Java uses the Unicode system for storing characters which includes the ASCII codes as a subset. For more information or

Here are the Java **escape sequences**:

Escape Sequence	Meaning
\b	backspace
\f	form feed
\n	line feed
\r	carriage return
\t	tab
\"	double quote
\'	single quote
\\	backslash

- **Boolean** constants:
true, false

I. Java has primitive data types (`int`, `char`, `double`, etc.) and reference data types (objects).

Constants are defined using the `final` keyword.

II. **Example:**

```
int age = 25; // variable

final double PI = 3.14; // constant
```

Java Variables

A **variable** has these attributes:

- **Name** The string of letters in a program used to designate a variable.
A name should start with a letter and consist of letters and/or digits.
Variables in names are case sensitive (capitalization matters).
The naming convention in Java is to start a variable name with a lower case letter.
New words within a name with a name start with a capital letter
(example: numberOfCustomers).
- **Value** The binary data contained in memory. The value is interpreted according to the variable's datatype
- **Address** The location in memory where the value is stored.
- **Size** The number of bytes that the variable occupies in memory.
- **Datatype** The interpretation of the value. Some important datatypes are listed in the following table.
- **Range** The minimum and maximum values of the variable. The range of a variable depends on its size. ||

Example variable declarations:

```
int maxAge;  
int x, y, selectedIndex;  
char a, b;  
boolean flag;  
double maxVal, massInKilos;
```

Variable initialization in declaration:

```
int timeInSeconds = 245;  
char a = 'K', b = '$';  
boolean flag = true;  
double maxVal = 35.875;
```

3.3 Comments and Documentation

- I. Comments are used to document code. Single-line comments use //, multi-line comments use /* ... */, and Javadoc comments use /** ... */.

- II. **Example:**

```
// This is a single-line comment  
  
/* This is a  
    multi-line comment */  
  
/**  
    * This is a Javadoc comment
```

```
* @param args Command-line arguments
*/
```

4. Control Statements

4.1 Conditional Statements

I. Java uses `if`, `else if`, and `else` for conditional execution.

II. **Example:**

```
if (age < 18) {
    System.out.println("Minor");
} else {
    System.out.println("Adult");
}
```

4.2 Switch-Case Statements

I. The `switch` statement allows for multi-way branching based on the value of an expression.

II. **Example:**

```
int day = 3;
switch (day) {
    case 1: System.out.println("Monday"); break;
    case 2: System.out.println("Tuesday"); break;
    case 3: System.out.println("Wednesday"); break;
    default: System.out.println("Invalid day");
}
```

4.3 Looping Statements

I. Java supports `for`, `while`, and `do-while` loops for iteration.

II. **Example:**

```
// for loop
for (int i = 0; i < 5; i++) {
    System.out.println(i);
}
```

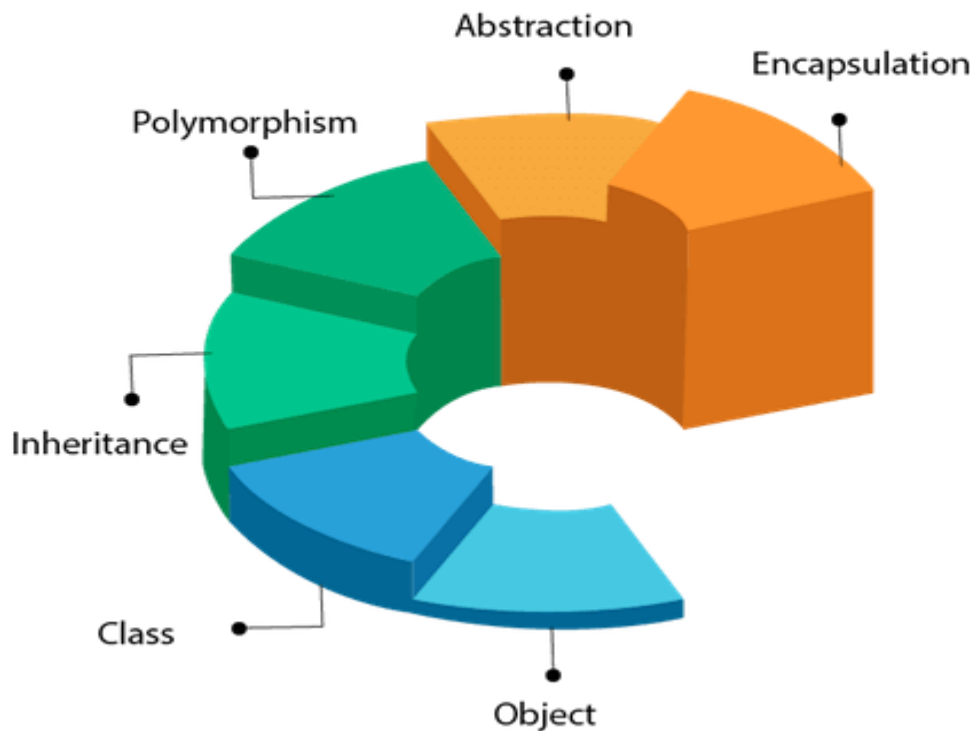
```
// while loop
int j = 0;
while (j < 5) {
    System.out.println(j);
    j++;
}
```

```
// do-while loop
int k = 0;
do {
    System.out.println(k);
    k++;
} while (k < 5);
```

5. Object-Oriented Programming (OOP)

5.1 Concepts of Objects and Classes

OOPs (Object-Oriented Programming System)



- I. Java is object-oriented, meaning it uses classes and objects to model real-world entities.

Classes define the structure, and objects are instances of classes.

II. **Example:**

```
class Car {  
    String color;  
    void start() {  
        System.out.println("Car started");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {
```



```
        Car myCar = new Car();  
  
        myCar.color = "Red";  
  
        myCar.start();  
    }  
}
```

5.2 Inheritance, Polymorphism, Encapsulation, and Abstraction

I.

- a. **Inheritance:** Allows a new class to inherit properties from an existing class.

```
Example: class Vehicle {  
  
    protected String brand = "Ford";  
  
    public void honk() {  
  
        System.out.println("Tuut, tuut!");  
  
    }  
  
}  
  
class Car extends Vehicle {  
  
    private String modelName = "Mustang";  
  
    public static void main(String[] args) {  
  
        Car myFastCar = new Car();  
  
    }  
}
```

```
        myFastCar.honk();

        System.out.println(myFastCar.brand + " " + myFastCar.modelName);

    }

}
```

- b. **Polymorphism:** Enables methods to do different things based on the object it is acting upon.

Example:

```
class Animal {

    public void animalSound() {

        System.out.println("The animal makes a sound");

    }

}

class Pig extends Animal {

    public void animalSound() {

        System.out.println("The pig says: wee wee");

    }

}
```

```
}  
  
class Dog extends Animal {  
  
    public void animalSound() {  
  
        System.out.println("The dog says: bow wow");  
  
    }  
  
}
```

- c. **Encapsulation:** Bundles data and methods that operate on the data into a single unit, typically a class.

Example :

```
public class Person {  
  
    private String name; // private = restricted access  
  
  
    // Getter  
  
    public String getName() {  
  
        return name;  
  
    }  
  
}
```

```
// Setter

public void setName(String newName) {

    this.name = newName;

}

}
```

- d. **Abstraction:** Hides complex implementation details and shows only the essential features.

The `abstract` keyword is a non-access modifier, used for classes and methods:

- **Abstract class:** is a restricted class that cannot be used to create objects (to access it, it must be inherited from another class).
- **Abstract method:** can only be used in an abstract class, and it does not have a body. The body is provided by the subclass (inherited from).

An abstract class can have both abstract and regular methods:

```
abstract class Animal {
    public abstract void animalSound();
    public void sleep() {
        System.out.println("Zzz");
    }
}
```

From the example above, it is not possible to create an object of the Animal class:

```
Animal myObj = new Animal(); // will generate an error
```

II. Example:

```
// Inheritance

class Animal {

    void eat() {

        System.out.println("Eating");

    }

}

class Dog extends Animal {

    void bark() {

        System.out.println("Barking");

    }

}

public class Main {

    public static void main(String[] args) {

        Dog dog = new Dog();

        dog.eat(); // Inherited method

        dog.bark(); // Dog's own method

    }

}
```

5.3 Constructors and Destructors

I.

- a. **Constructors:** Special methods used to initialize objects.
- b. **Destructors:** Java does not have explicit destructors but uses garbage collection to clean up unused objects.

II. Example:

```
class Person {  
    String name;  
  
    // Constructor  
    Person(String name) {  
        this.name = name;  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Person person = new Person("Alice");  
        System.out.println(person.name);  
    }  
}
```

6. Exception Handling

6.1 Understanding Exceptions and Errors

- I. Exceptions are events that disrupt the normal flow of a program, while errors are serious issues that cannot be handled by the application.
- II. **Example:** Division by zero throws an **ArithmeticException**.

6.2 Try-Catch Blocks

Full Stack Development [Unit-II]

- I. `try` blocks contain code that might throw exceptions, and `catch` blocks handle those exceptions.

- II. **Example:**

```
try {  
    int result = 10 / 0;  
} catch (ArithmeticException e) {  
    System.out.println("Cannot divide by zero");  
}
```

6.3 Custom Exception Handling

- I. Custom exceptions can be created by extending the `Exception` class.

- II. **Example:**

```
class MyException extends Exception {  
    MyException(String message) {  
        super(message);  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        try {  
            throw new MyException("Custom exception occurred");  
        } catch (MyException e) {  
            System.out.println(e.getMessage());  
        }  
    }  
}
```

```
}  
  
}
```

7. File Handling

7.1 Reading and Writing to Files

I. Java provides **FileReader** and **FileWriter** for reading from and writing to files.

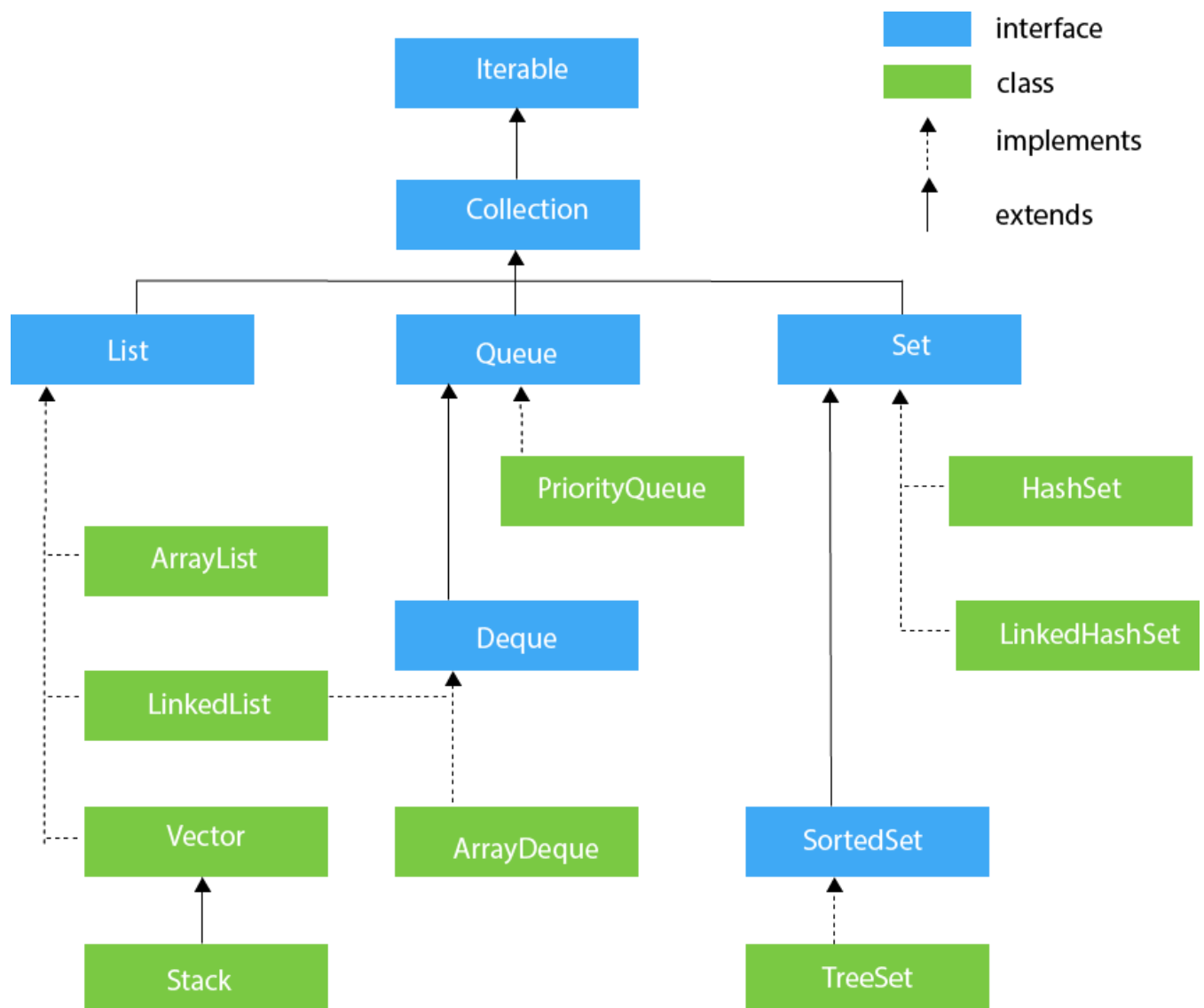
II. **Example:**

```
import java.io.*;  
  
public class FileHandling {  
    public static void main(String[] args) throws IOException {  
        // Writing to a file  
        FileWriter writer = new FileWriter("example.txt");  
        writer.write("Hello, World!");  
        writer.close();  
  
        // Reading from a file  
        FileReader reader = new FileReader("example.txt");  
        int ch;  
        while ((ch = reader.read()) != -1) {  
            System.out.print((char) ch);  
        }  
        reader.close();  
    }  
}
```


7.2 Working with Input and Output Streams

- I. Java's I/O streams handle input and output operations. `InputStream` and `OutputStream` are abstract classes for reading and writing byte data.
- II. **Example:** `BufferedReader` and `BufferedWriter` for efficient text I/O.

8. Collections Framework



8.1 Overview of Collections

- I. Java Collections Framework provides interfaces and classes for data manipulation. Major interfaces include `List`, `Set`, and `Map`.
- II. **Example:** `ArrayList` (implements `List`), `HashSet` (implements `Set`), `HashMap`

8. Collections Framework

8.1 Overview of Collections

- I. Java Collections Framework (JCF) provides a set of classes and interfaces to handle data collections. It includes three main interfaces:
 - a. **List:** An ordered collection (e.g., `ArrayList`, `LinkedList`).
 - b. **Set:** A collection that does not allow duplicate elements (e.g., `HashSet`, `TreeSet`).
 - c. **Map:** A collection of key-value pairs (e.g., `HashMap`, `TreeMap`).

8.2 Iterators and Enumerations

- I. Iterators are used to traverse the elements in a collection, while enumerations are a legacy interface for iteration.
- II. **Example:**

```
// Using Iterator
List<String> list = new ArrayList<>();

list.add("A");

list.add("B");

Iterator<String> iterator = list.iterator();
```

Full Stack Development [Unit-II]

```
while (iterator.hasNext()) {  
    System.out.println(iterator.next());  
}  
  
// Using Enumeration (legacy)  
Vector<String> vector = new Vector<>();  
vector.add("X");  
vector.add("Y");  
Enumeration<String> enumeration = vector.elements();  
while (enumeration.hasMoreElements()) {  
    System.out.println(enumeration.nextElement());  
}
```

Property	Enumeration	Iterator	ListIterator
Applicable for	Only legacy classes	Any Collection classes	Only List classes
Movement	Only forward direction(single direction)	Only forward direction(single direction)	Both forward and backward direction(bi directional)
Accessibility	Only read access	Both read and remove	Read ,remove, replace and addition of new objects
How to get it?	By using elements() method of Vector class	By using iterator() method of Collection interface	By using listIterator() method of List interface
Methods	2 methods hasMoreElements() nextElement()	3 methods hasNext () next() remove()	9 methods
Is it legacy	"yes" (1.0v)	"no" (1.2V)	"no" (1.2V)

8.3 ArrayList, LinkedList, HashMap

Property	HashMap	TreeMap	LinkedHashMap	HashTable
Iteration Order	Random	Sorted according to natural order of keys	Sorted according to the insertion order.	Random
Efficiency: Get, Put, Remove, ContainsKey	$O(1)$	$O(\log(n))$	$O(1)$	$O(1)$
Null keys/values	allowed	Not-allowed*	allowed	Not-allowed
Interfaces	Map	Map, SortedMap, NavigableMap	Map	Map
Synchronized	Not instead use Collection.synchronizedMap(new HashMap())			Yes but prefer to use ConcurrentHashMap
Implementation	Buckets	Red-Black tree	HashTable and LinkedList using doubly linked list of buckets	Buckets
Comments	Efficient	Extra cost of maintaining TreeMap	Advantage of TreeMap without extra cost.	Obsolete

I.

- a. **ArrayList:** Implements `List`, backed by an array, provides fast random access.
- b. **LinkedList:** Implements `List`, backed by a doubly linked list, offers efficient insertions and deletions.
- c. **HashMap:** Implements `Map`, provides a key-value storage with constant-time performance for basic operations.

II. Example:

```
// ArrayList Example

List<String> arrayList = new ArrayList<>();

arrayList.add("Element1");

arrayList.add("Element2");


// LinkedList Example

LinkedList<String> linkedList = new LinkedList<>();

linkedList.add("Node1");

linkedList.add("Node2");


// HashMap Example

Map<String, Integer> hashMap = new HashMap<>();

hashMap.put("Key1", 1);

hashMap.put("Key2", 2);
```

9. Multithreading

9.1 Introduction to Threads

- I. Threads are lightweight processes that allow concurrent execution. Java supports multithreading through the `Thread` class and `Runnable` interface.

II. Example:

```
class MyThread extends Thread {

    public void run() {
```

```
        System.out.println("Thread is running");
    }
}

public class Main {
    public static void main(String[] args) {
        MyThread t = new MyThread();
        t.start();
    }
}
```



Thread

- Thread is the smallest unit of executable code that performs a particular task.
- An application can be divided into multiple tasks and each task can be assigned to a thread.
- Many threads executing simultaneously is termed as Multithreading.
- Appears that the processes are running concurrently, but it is not so.

9.2 Synchronization and Coordination

- I. Synchronization prevents data corruption when multiple threads access shared resources.

Coordination involves techniques like `wait()`, `notify()`, and `notifyAll()`.

II. **Example:**

```
class Counter {  
    private int count = 0;  
  
    public synchronized void increment() {  
        count++;  
    }  
  
    public synchronized int getCount() {  
        return count;  
    }  
}
```

9.3 Thread Safety and Deadlock

I.

- a. **Thread Safety:** Ensures that multiple threads can operate on shared data without causing inconsistencies.
- b. **Deadlock:** Occurs when two or more threads are blocked forever, each waiting on the other to release resources.

II. **Example of Deadlock:**

```
class Resource1 {  
    synchronized void method1(Resource2 r2) {  
        System.out.println("Resource1: method1");  
        r2.last();  
    }  
  
    synchronized void last() {  
        System.out.println("Resource1: last");  
    }  
}  
  
class Resource2 {  
    synchronized void method1(Resource1 r1) {  
        System.out.println("Resource2: method1");  
        r1.last();  
    }  
  
    synchronized void last() {  
        System.out.println("Resource2: last");  
    }  
}
```

10. Introduction to Java API

10.1 Working with Java Standard Libraries

- I. Java API includes a vast set of libraries for tasks like data manipulation, file I/O, networking, and more.

II. Example:

```
// Using Java Standard Libraries

import java.util.ArrayList;

import java.util.HashMap;


public class ApiExample {

    public static void main(String[] args) {

        ArrayList<String> list = new ArrayList<>();

        list.add("Hello");

        System.out.println(list);


        HashMap<String, Integer> map = new HashMap<>();

        map.put("Key1", 100);

        System.out.println(map);

    }

}
```

10.2 Utilizing Predefined Classes and Methods

- I. Java provides predefined classes and methods for common tasks, such as String, Math, Date, etc.

II. Example:

```
public class PredefinedClasses {

    public static void main(String[] args) {

        String str = "Hello, World!";

        System.out.println(str.toUpperCase());

    }

}
```

```
        int result = Math.max(10, 20);  
        System.out.println(result);  
    }  
}
```

11. Applets and Swing

11.1 Basics of GUI Programming with AWT and Swing

Java AWT	Java Swing
AWT components are platform-dependent.	Java swing components are platform-independent.
AWT components are heavyweight.	Swing components are lightweight.
AWT doesn't support pluggable look and feel.	Swing supports pluggable look and feel.
AWT provides less components than Swing.	Swing provides more powerful components such as tables, lists, scrollpanes, colorchooser, tabbedpane, etc.
AWT doesn't follow MVC(Model View Controller) where model represents data, view represents presentation and controller acts as an interface between model and view.	Swing follows MVC.

- I. AWT (Abstract Window Toolkit) and Swing are libraries for creating graphical user interfaces in Java.
- II. **Example:**

```
import javax.swing.JButton;  
import javax.swing.JFrame;  
import javax.swing.JPanel;
```

```
public class SwingExample {  
    public static void main(String[] args) {  
        JFrame frame = new JFrame("Swing Example");  
        JPanel panel = new JPanel();  
        JButton button = new JButton("Click Me");  
        panel.add(button);  
        frame.add(panel);  
        frame.setSize(300, 200);  
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);  
        frame.setVisible(true);  
    }  
}
```

11.2 Designing and Implementing Graphical Interfaces

- I. Designing GUIs involves creating components such as buttons, text fields, and handling user interactions.

- II. **Example:**

```
import javax.swing.JButton;  
import javax.swing.JFrame;  
import javax.swing.JLabel;  
import javax.swing.JPanel;  
import java.awt.event.ActionEvent;  
import java.awt.event.ActionListener;  
  
public class GUIExample {  
    public static void main(String[] args) {
```

```
JFrame frame = new JFrame("GUI Example");

JPanel panel = new JPanel();

JLabel label = new JLabel("Click the button");

JButton button = new JButton("Click Me");

button.addActionListener(new ActionListener() {

    public void actionPerformed(ActionEvent e) {

        label.setText("Button Clicked!");

    }

});

panel.add(label);

panel.add(button);

frame.add(panel);

frame.setSize(300, 200);

frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

frame.setVisible(true);

}
```

12. Networking

12.1 Basics of Networking in Java

- I. Java provides APIs for networking, including `Socket` and `ServerSocket` classes for communication over TCP/IP.
- II. **Example:**

```
import java.io.*;
```

Full Stack Development [Unit-II]

```
import java.net.*;

public class SimpleServer {

    public static void main(String[] args) throws IOException {

        ServerSocket serverSocket = new ServerSocket(8080);

        System.out.println("Server is running...");

        Socket socket = serverSocket.accept();

        PrintWriter out = new PrintWriter(socket.getOutputStream(),
true);

        out.println("Hello, Client!");

        socket.close();

    }

}

public class SimpleClient {

    public static void main(String[] args) throws IOException {

        Socket socket = new Socket("localhost", 8080);

        BufferedReader in = new BufferedReader(new
InputStreamReader(socket.getInputStream()));

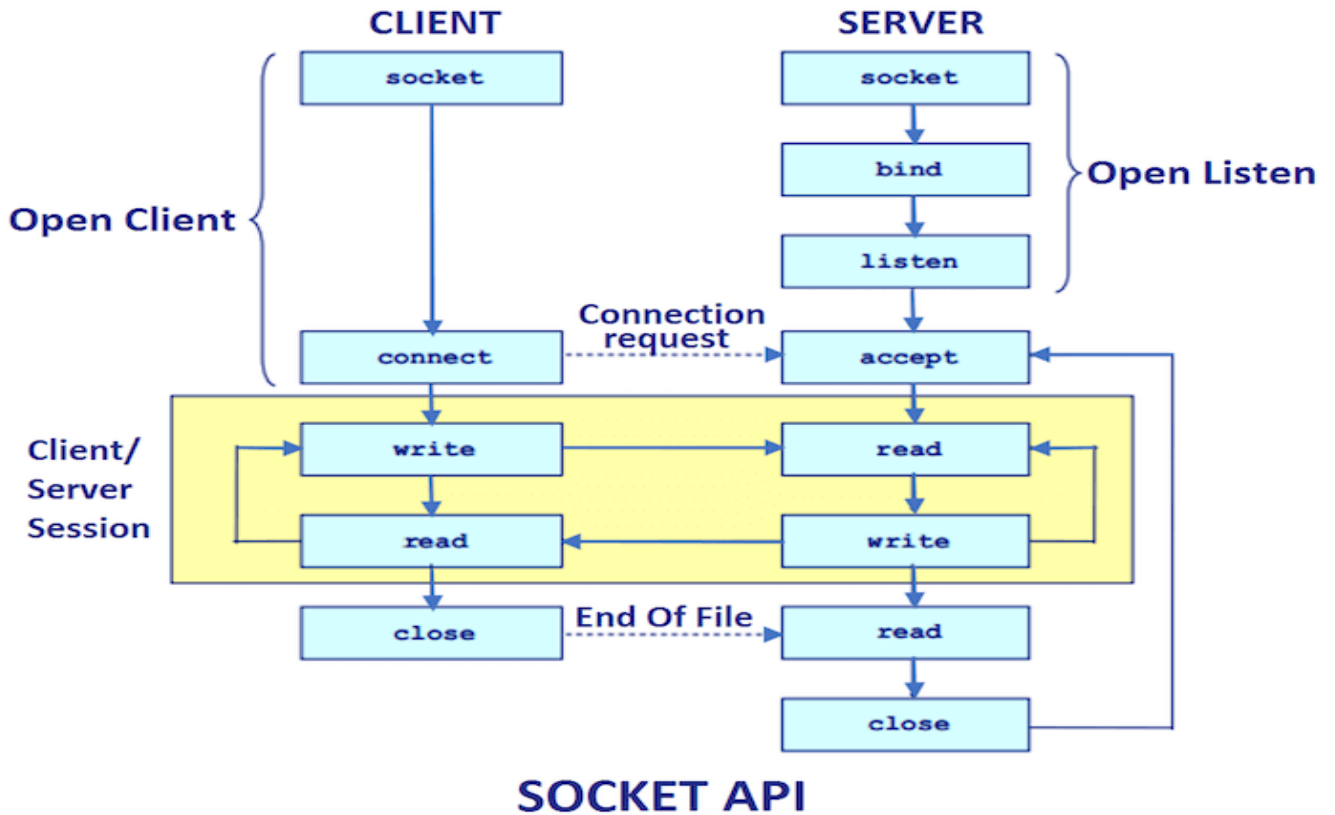
        String response = in.readLine();

        System.out.println("Server says: " + response);

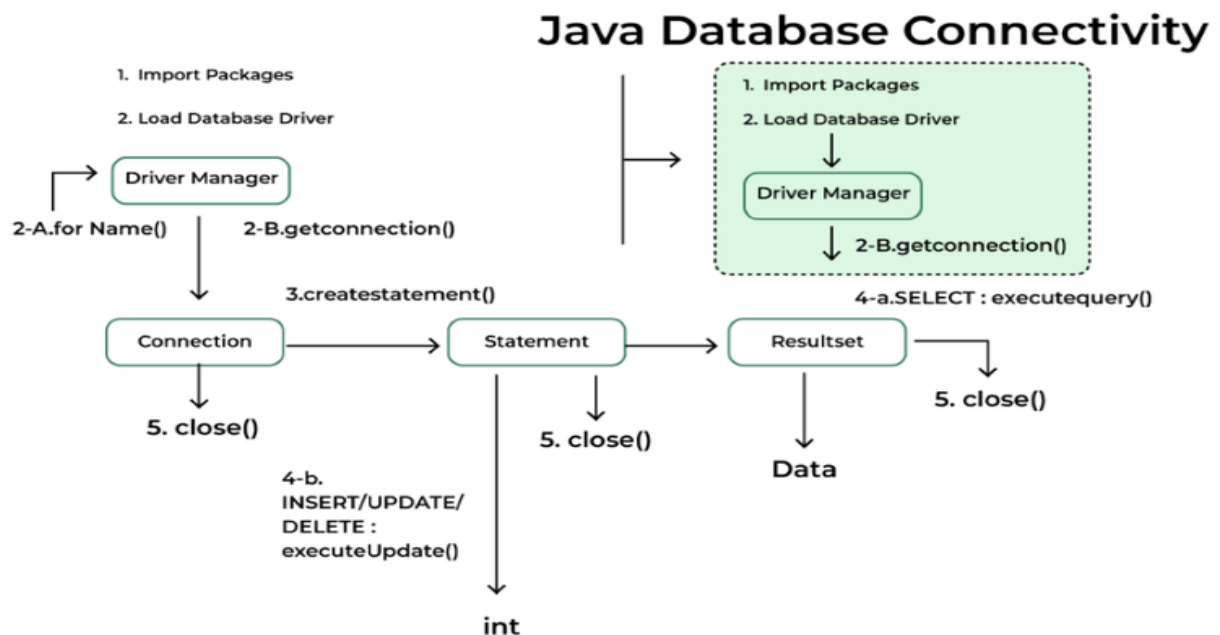
        socket.close();

    }

}
```



13. Database Connectivity (JDBC)



13.1 Overview of JDBC

- I. JDBC (Java Database Connectivity) is an API for connecting and executing queries on a database.
- II. **Example:**

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.Statement;

public class JDBCExample {
    public static void main(String[] args) {
        try {
            Connection connection =
DriverManager.getConnection("jdbc:mysql://localhost:3306/mydatabase",
"user", "password");

            Statement statement = connection.createStatement();
            ResultSet resultSet = statement.executeQuery("SELECT * FROM
users");

            while (resultSet.next()) {
                System.out.println("User ID: " +
resultSet.getInt("id"));
                System.out.println("Username: " +
resultSet.getString("username"));
            }
        }
    }
}
```

```
        connection.close();  
    } catch (Exception e) {  
        e.printStackTrace();  
    }  
}  
}
```

13.2 Connecting to Databases and Executing SQL Queries

```
// An object that represents a precompiled SQL statement.  
PreparedStatement stmt = null;  
try {  
    conn = crunchifyDS.getConnection();  
    stmt = conn.prepareStatement("select * from company");  
  
    // A table of data representing a database result set, which is  
    // usually generated by executing a statement that queries the  
    // database.  
    ResultSet rs = stmt.executeQuery();  
    while (rs.next()) {  
        System.out.println("Comapny: " + rs.getString("company"));  
    }  
} catch (SQLException e) {  
    e.printStackTrace();  
} finally {  
    // Close Statement and Connection  
    if (stmt != null) {  
        stmt.close();  
    }  
    if (conn != null) {  
        conn.close();  
    }  
}
```

- I. Use Connection to establish a connection, Statement or PreparedStatement to execute queries, and ResultSet to handle results.

II. Example:


```
// PreparedStatement Example

import java.sql.Connection;

import java.sql.DriverManager;

import java.sql.PreparedStatement;


public class PreparedStatementExample {

    public static void main(String[] args) {

        try {

            Connection connection =

DriverManager.getConnection("jdbc:mysql://localhost:3306/mydatabase",

"user", "password");

            String query = "INSERT INTO users (username) VALUES (?)";

            PreparedStatement preparedStatement =

connection.prepareStatement(query);

            preparedStatement.setString(1, "newuser");

            preparedStatement.executeUpdate();

            connection.close();

        } catch (Exception e) {

            e.printStackTrace();

        }

    }

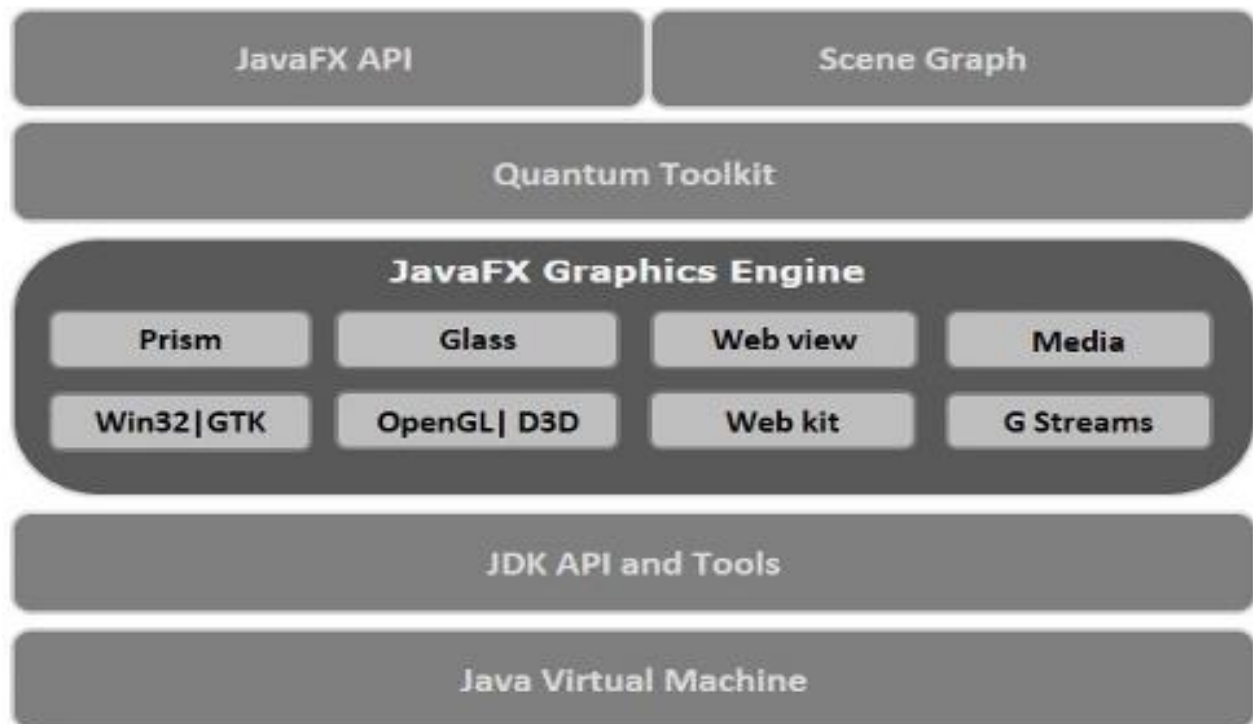
}
```

14. Introduction to JavaFX

- JavaFX has numerous built-in elements that are interconnected with each other. JavaFX library comprises a valuable collection of APIs, classes, and interfaces that are more than

sufficient to produce rich internet applications and GUI applications with intense graphics that can run consistently over multiple platforms.

- The subsequent figure displays the complete architecture of the JavaFX platform. Here you can examine the elements that support JavaFX API.



JavaFX API – The topmost layer of JavaFX architecture holds a JavaFX public API that implements all the required classes that are capable of producing a full-featured JavaFX application with rich graphics. The list of all the important packages of this API is as follows.

- javafx.animation:** It includes classes that are used to combine transition-based animations such as fill, fade, rotate, scale and translation, to the JavaFX nodes (collection of nodes makes a scene graph).

- II. `javafx.css` – It comprises classes that are used to append CSS-like styling to the JavaFX GUI applications.
- III. `javafx.geometry` – It contains classes that are used to represent 2D figures and execute methods on them.
- IV. `javafx.scene` – This package of JavaFX API implements classes and interfaces to establish the scene graph. In extension, it also renders sub-packages such as canvas, chart, control, effect, image, input, layout, media, paint, shape, text, transform, web, etc. These are the diverse elements that sustain this precious API of JavaFX.
- V. `javafx.application` – This package includes a collection of classes that are responsible for the life cycle of the JavaFX application.
- VI. `javafx.event` – It includes classes and interfaces that are used to perform and manage JavaFX events.
- VII. `javafx.stage` – This package of JavaFX API accommodates the top-level container classes used for the JavaFX application.
- VIII. Scene Graph – A Scene Graph is the starting point of the development of any of the GUI Applications. In JavaFX, all the GUI Applications are made using a Scene Graph only. The Scene Graph includes the primitives of the rich internet applications that are known as nodes. In simple words, a single component in a scene graph is known as a node. In general, a scene graph is made up of a collection of nodes. All these nodes are organized in the form of a hierarchical tree that describes all of the visual components of the application's user interface (UI). A node instance can be appended to a scene graph only once. The nodes of a scene graph can have numerous segments like Effects, Opacity,

Transforms, Event Handlers, Application Specific States. The nodes are of three general types.

14.1 Basics of JavaFX for Modern UI Development

- I. JavaFX is a framework for creating modern and rich user interfaces. It replaces Swing as the standard GUI toolkit for Java.

- II. **Example:**

```
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.layout.VBox;
import javafx.stage.Stage;

public class JavaFXExample extends Application {
    public static void main(String[] args) {
        launch(args);
    }
    @Override
    public void start(Stage primaryStage) {
        VBox vbox = new VBox();
        Button button = new Button("Click Me");
        button.setOnAction(e -> System.out.println("Button Clicked"));

        vbox.getChildren().add(button);
        Scene scene = new Scene(vbox, 300, 200);
        primaryStage.setScene(scene);
```

```
        primaryStage.setTitle("JavaFX Example");  
        primaryStage.show();  
    }  
}
```

15. Unit Testing

15.1 Introduction to JUnit for Testing Java Applications

- I. JUnit is a framework for writing and running tests in Java. It allows for the creation of automated tests to verify the correctness of code.

- II. **Example:**

```
import static org.junit.jupiter.api.Assertions.assertEquals;  
import org.junit.jupiter.api.Test;  
  
public class MathUtilsTest {  
    @Test  
    void testAdd() {  
        MathUtils mathUtils = new MathUtils();  
        assertEquals(5, mathUtils.add(2, 3));  
    }  
}  
  
class MathUtils {  
    int add(int a, int b) {  
        return a + b;  
    }  
}
```

16. Introduction to Maven and Ant

Ant	Maven
Ant has no formal rules, so you need to include the project structure information in your build.xml file.	Maven has rules for storing source code, compiled code, etc., so you don't need to provide information about the project structure in the pom.xml file.
Ant has no life cycle. So, you have to maintain order.	Maven has a life cycle. So, you do not need to maintain the code.
Ant is primarily a build tool.	Maven is a project management tool.
Ant scripts cannot be reused.	Maven plugins can be reused.

16.1 Overview of Building Automation Tools

- I. Maven and Ant are tools for building and managing Java projects. Maven uses a declarative approach with XML configuration, while Ant uses a more procedural approach with XML.

- II. **Example (Maven):**

```

<!-- pom.xml -->

<project xmlns="http://maven.apache.org/POM/4.0.0"
          xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
          xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/POM/4.0.0">
    <modelVersion>4.0.0</modelVersion>
    <groupId>com.example</groupId>
    <artifactId>myproject</artifactId>
    <version>1.0-SNAPSHOT</version>

```

```
<dependencies>

  <dependency>

    <groupId>junit</groupId>

    <artifactId>junit</artifactId>

    <version>4.13.2</version>

    <scope>test</scope>

  </dependency>

</dependencies>

</project>
```

III. Example (Ant):

```
<!-- build.xml -->

<project name="MyProject" basedir="." default="compile">

  <target name="compile">

    <javac srcdir="src" destdir="bin">

      <classpath>

        <fileset dir="lib">

          <include name="*.jar"/>

        </fileset>

      </classpath>

    </javac>

  </target>

</project>
```

Maven is a powerful project management tool based on the Project Object Model. It helps in managing project builds, documentation, dependency, releases, etc. **Ant:** Ant is a command-line toolbox without any coding conventions or project structures, making it flexible and more manageable to use.

17. Basic Design Patterns

17.1 Understanding Common Design Patterns

- I. Design patterns are typical solutions to common problems in software design. Some common patterns are:
 - a. **Singleton:** Ensures a class has only one instance and provides a global point of access.
 - b. **Factory:** Provides an interface for creating objects without specifying their concrete classes.
 - c. **Observer:** Defines a one-to-many dependency between objects so that when one object changes state, all dependents are notified.
 - d. **Decorator:** Adds additional responsibilities to an object dynamically.

II. Example:

```
// Singleton Pattern

public class Singleton {

    private static Singleton instance;

    private Singleton() {}

    public static synchronized Singleton getInstance() {

        if (instance == null) {

            instance = new Singleton();

        }

        return instance;

    }

}
```


Advanced Java Syllabus

Module 1: Java Database Connectivity (JDBC)

- **1.1 Overview of JDBC**

- I. JDBC is an API for interacting with databases from Java applications.
- II. **JDBC Architecture:** Consists of JDBC API, JDBC Driver Manager, and JDBC Drivers.

- **1.2 Connecting to Databases**

- I. Establish connections using `DriverManager` or `DataSource`.
- II. **Connection Pooling:** Enhances performance by reusing database connections.
- III. **Example:**

```
// Connection Pooling Example with HikariCP
HikariConfig config = new HikariConfig();
config.setJdbcUrl("jdbc:mysql://localhost:3306/mydatabase");
config.setUsername("user");
config.setPassword("password");
HikariDataSource ds = new HikariDataSource(config);
Connection connection = ds.getConnection();
```

- **1.3 Executing SQL Queries**

- I. Use `PreparedStatement` and `CallableStatement` for executing SQL queries and handling stored procedures.
- II. **Batch Processing:** Allows executing multiple statements in one go.

III. Example:

```
// Batch Processing Example

Connection connection =

DriverManager.getConnection("jdbc:mysql://localhost:3306/mydataba
se", "user", "password");

PreparedStatement pstmt = connection.prepareStatement("INSERT
INTO users (username) VALUES (?)");

pstmt.setString(1, "user1");

pstmt.addBatch();

pstmt.setString(1, "user2");

pstmt.addBatch();

pstmt.executeBatch();
```

Module 2: Java Servlets

What is Java Servlet?

Java Servlets are the Java programs that run on the Java-enabled web server or application server. They are used to handle the request obtained from the web server, process the request, produce the response, and then send a response back to the web server.

Properties of Java Servlet

The properties of Servlets are as follows:

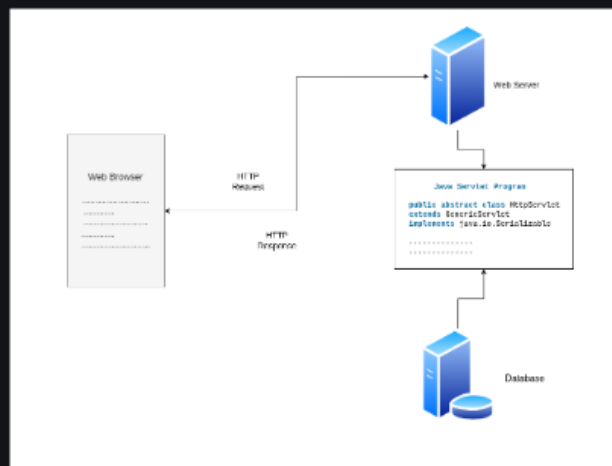
- Servlets work on the server side.
- Servlets are capable of handling complex requests obtained from the web server.

• 2.1 Introduction to Servlets

- I. Servlets are Java programs that run on a server and handle client requests.
- II. **Servlet Lifecycle:** Includes initialization (`init`), request handling (`service`), and destruction (`destroy`).

Java Servlets Architecture

Servlet Architecture can be depicted from the image itself as provided below as follows:



Execution of Java Servlets

Execution of Servlets basically involves Six basic steps:

1. The Clients send the request to the Web Server.
2. The Web Server receives the request.
3. The Web Server passes the request to the corresponding servlet.
4. The Servlet processes the request and generates the response in the form of output.
5. The Servlet sends the response back to the webserver.
6. The Web Server sends the response back to the client and the client browser displays it on the screen.

• 2.2 Servlet Configurations and Parameters

- I. Initialization parameters can be set in the `web.xml` file or annotations.
- II. **Example:**

```
<!-- web.xml -->

<servlet>

    <servlet-name>MyServlet</servlet-name>

    <servlet-class>com.example.MyServlet</servlet-class>

    <init-param>

        <param-name>paramName</param-name>

        <param-value>paramValue</param-value>

    </init-param>

</servlet>
```

- **2.3 Handling Form Data**

- I. Servlets handle form data using `HttpServletRequest` methods like

`getParameter.`

- II. **Example:**

```
// Handling Form Data

protected void doPost(HttpServletRequest request,
    HttpServletResponse response) throws ServletException,
    IOException {

    String username = request.getParameter("username");

    // Process the username

}
```

Module 3: JavaServer Pages (JSP)

- **3.1 Introduction to JSP**

- I. JSP is a technology for creating dynamic web pages with Java.
- II. **JSP Lifecycle:** Involves translation, compilation, initialization, request processing, and destruction.
- III. It is a server-side technology which is used for creating web applications. It is used to create dynamic web content. JSP consists of both HTML tags and JSP tags. In this, JSP tags are used to insert JAVA code into HTML pages. It is an advanced version of Servlet Technology i.e. a web-based technology that helps us to create dynamic and platform-independent web pages.
- IV. In this, Java code can be inserted in HTML/ XML pages or both. JSP is first converted into a servlet by the JSP container before processing the client's request. JSP has various features like JSP Expressions, JSP tags, JSP Expression Language, etc.

How JSP more advantageous than Servlet?

- They are easy to maintain.
- No recompilation or redeployment is required.
- Less coding is required in JSP.
- JSP has access to the entire API of JAVA.
- JSP are extended version of Servlet.

• 3.2 Directives and Actions

- I. JSP directives provide information about an entire JSP page, while actions perform tasks within the page.
- II. **Example:**

```
<%@ page language="java" contentType="text/html; charset=UTF-8"
%>

<jsp:useBean id="myBean" class="com.example.MyBean"/>
```

• 3.3 JSP Tag Libraries

I. Tag libraries like JSTL provide reusable components for JSP pages.

II. **Example:**

```
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>

<c:forEach var="item" items="${itemsList}">

    ${item}

</c:forEach>
```

Advantage of JSTL

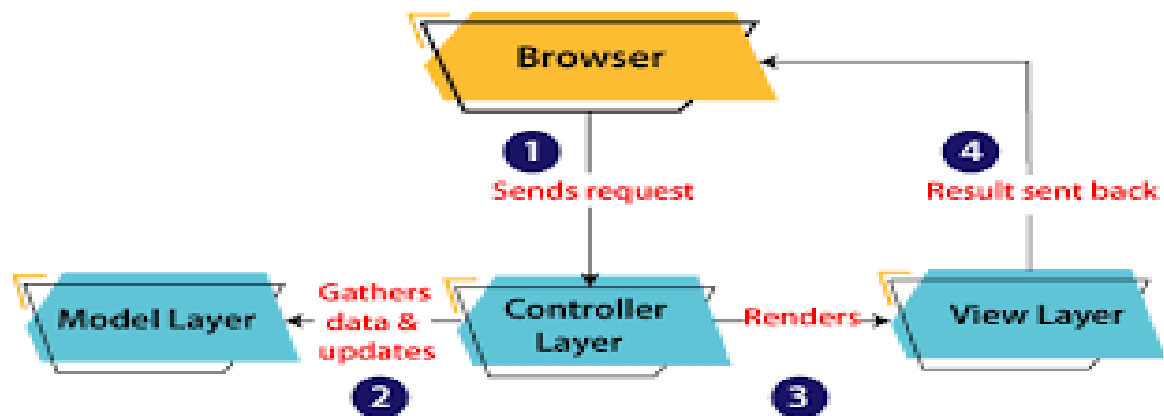
1. **Fast Development** JSTL provides many tags that simplify the JSP.
2. **Code Reusability** We can use the JSTL tags on various pages.
3. **No need to use scriptlet tag** It avoids the use of scriptlet tag.

JSTL Tags

There JSTL mainly provides five types of tags:

Tag Name	Description
Core tags	The JSTL core tag provide variable support, URL management, flow control, etc. The URL for the core tag is http://java.sun.com/jsp/jstl/core . The prefix of core tag is c .
Function tags	The functions tags provide support for string manipulation and string length. The URL for the functions tags is http://java.sun.com/jsp/jstl/functions and prefix is fn .
Formatting tags	The Formatting tags provide support for message formatting, number and date formatting, etc. The URL for the Formatting tags is http://java.sun.com/jsp/jstl/fmt and prefix is fmt .
XML tags	The XML tags provide flow control, transformation, etc. The URL for the XML tags is http://java.sun.com/jsp/jstl/xml and prefix is x .
SQL tags	The JSTL SQL tags provide SQL support. The URL for the SQL tags is http://java.sun.com/jsp/jstl/sql and prefix is sql .

Module 4: Model-View-Controller (MVC) Architecture



- **4.1 MVC Design Pattern**

Advantages of MVC Architecture

The advantages of MVC architecture are as follows:

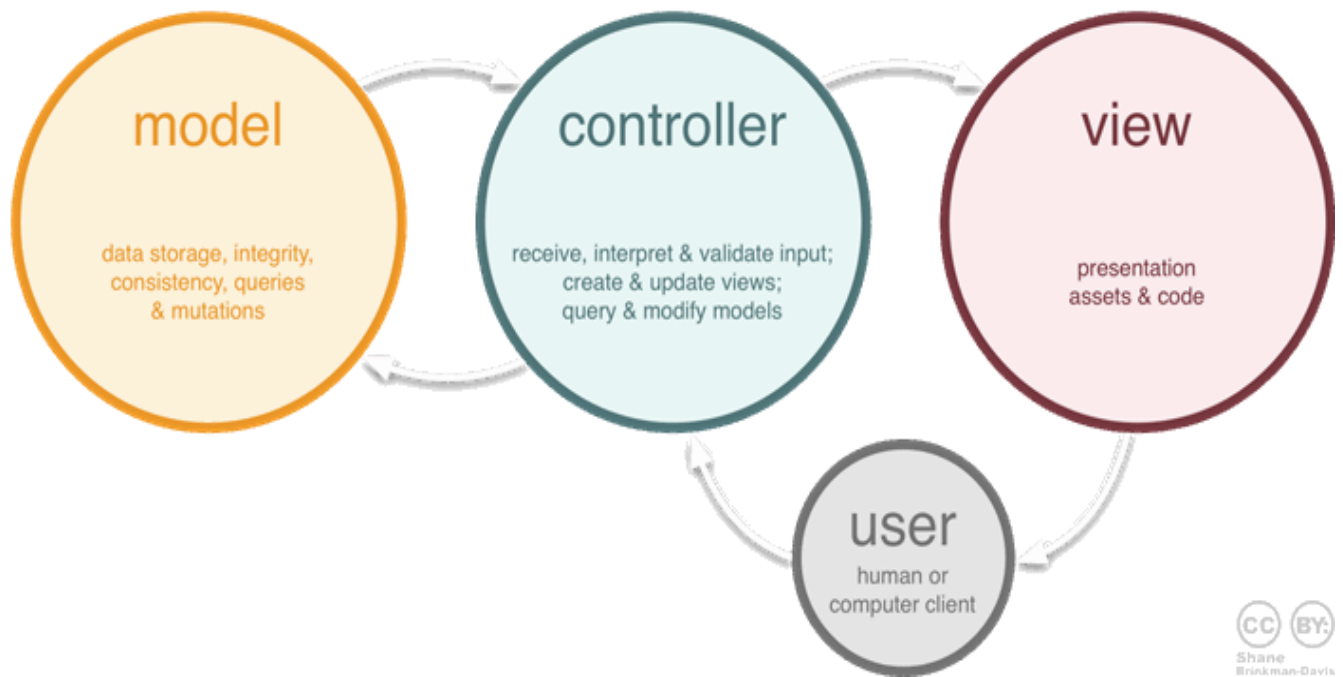
- MVC has the feature of scalability that in turn helps the growth of application.
- The components are easy to maintain because there is less dependency.
- A model can be reused by multiple views that provides reusability of code.
- The developers can work with the three layers (Model, View, and Controller) simultaneously.
- Using MVC, the application becomes more understandable.
- Using MVC, each layer is maintained separately therefore we do not require to deal with massive code.
- The extending and testing of application is easier

I. MVC separates an application into three interconnected components:

- Model:** Manages the data and business logic.

- b. **View:** Handles the presentation layer.
- c. **Controller:** Manages user input and interacts with the model.

II. **Example:** Implementing a basic MVC structure in a web application.



Module 5: Session Management

- **5.1 HttpSession and Cookies**

- I. `HttpSession` manages user session data, while cookies store small amounts of data on the client side.

II. **Example:**

```
// Managing Session  
HttpSession session = request.getSession();  
session.setAttribute("username", "JohnDoe");
```



```
// Managing Cookies  
  
Cookie cookie = new Cookie("user", "JohnDoe");  
  
response.addCookie(cookie);
```

Module 6: JavaServer Faces (JSF)

JavaServer Faces (JSF) is a new standard Java framework for building Web applications. It simplifies development by providing a component-centric approach to developing Java Web user interfaces. JavaServer Faces also appeals to a diverse audience of Java/Web developers.

- **6.1 Introduction to JSF**

- JavaServer Faces (JSF) is a new standard Java framework for building Web applications. It simplifies development by providing a component-centric approach to developing Java Web user interfaces. JavaServer Faces also appeals to a diverse audience of Java/Web developers.
- "Corporate developers" and Web designers will find that JSF development can be as simple as dragging and dropping user interface (UI) components onto a page, while "systems developers" will find that the rich and robust JSF API offers them unsurpassed power and programming flexibility.
- JSF also ensures that applications are well designed with greater maintainability by integrating the well-established Model-View-Controller (MVC) design pattern into its architecture.

- Finally, since JSF is a Java standard developed through Java Community Process (JCP), development tools vendors are fully empowered to provide easy to use, visual, and productive develop environments for JavaServer Faces

- I. JSF is a framework for building component-based web applications.
- II. **JSF Architecture:** Includes components, managed beans, and navigation rules.
- III. **Example:**

```
// Managed Bean
@ManagedBean
public class HelloBean {
    private String name;

    public String getName() {
        return name;
    }

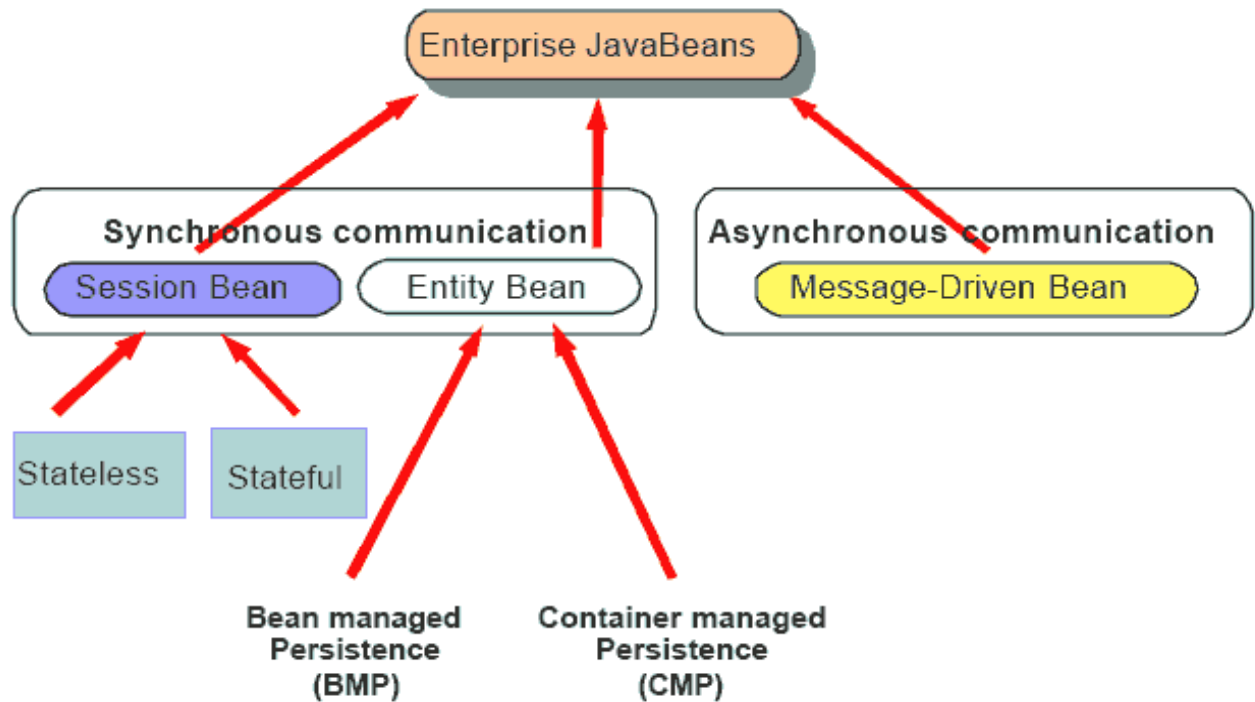
    public void setName(String name) {
        this.name = name;
    }

    public String greet() {
        return "Hello " + name;
    }
}
```

Module 7: Enterprise JavaBeans (EJB)

- 7.1 Introduction to EJB

Enterprise JavaBeans



- I. EJBs are server-side components that encapsulate business logic.
- II. **Types of EJBs:** Session beans, entity beans, and message-driven beans.
- III. **Example:**

```
@Stateless

public class MyBean implements MyBeanLocal {

    public String sayHello() {

        return "Hello";

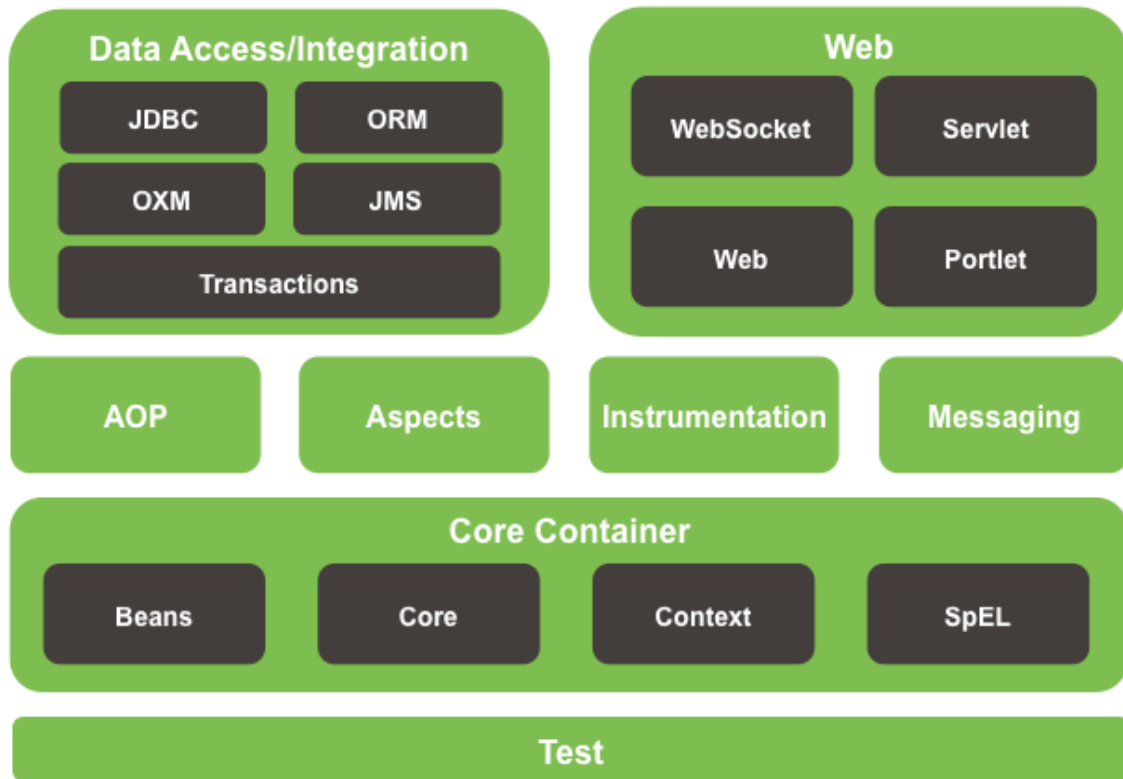
    }

}
```

Module 8: Spring Framework



Spring Framework Runtime



- **8.1 Introduction to Spring**

- I. Spring is a comprehensive framework for enterprise Java development, emphasizing Dependency Injection (DI) and Aspect-Oriented Programming (AOP).
- II. **Example:**

```
// Spring Bean Configuration
```

```
@Configuration

public class AppConfig {

    @Bean

    public MyBean myBean() {

        return new MyBean();

    }

}

@Component

public class MyBean {

    // Bean methods

}
```

Features of the Spring Framework

The features of the Spring framework such as IoC, AOP, and transaction management, make it unique among the list of frameworks. Some of the most important features of the Spring framework are as follows:

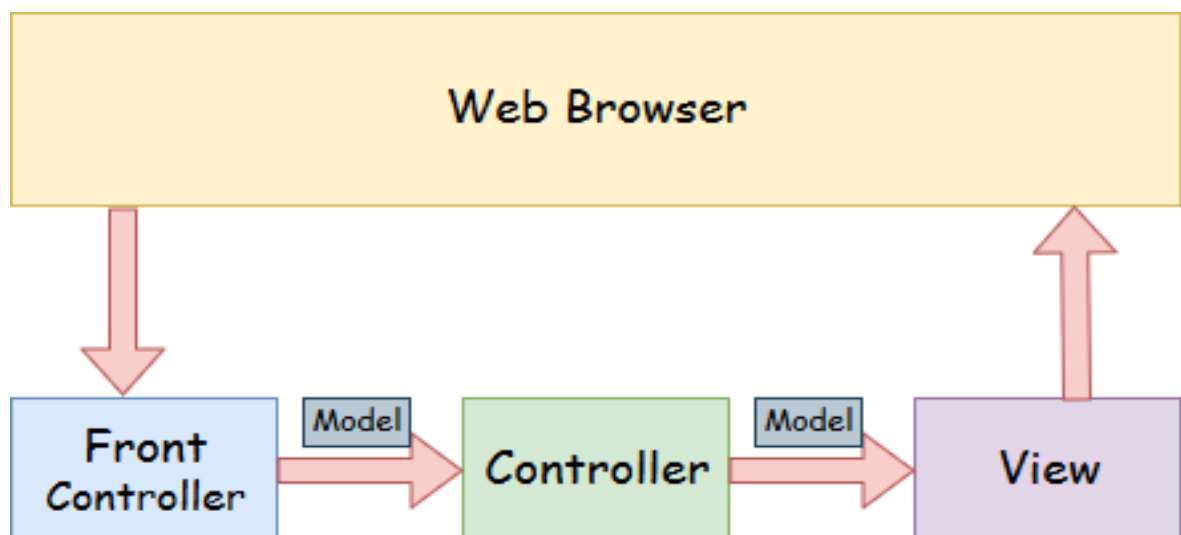
- a. **IoC container:** Refers to the core container that uses the DI or IoC pattern to implicitly provide an object reference in a class during runtime. This pattern acts as an alternative to the service locator pattern. The IoC container contains assembler code that handles the configuration management of application objects. The Spring framework provides two packages, namely `org.springframework.beans` and `org.springframework.context` which helps in providing the functionality of the IoC container
- b. **Data access framework:**
Allows the developers to use persistence APIs, such as JDBC and Hibernate, for storing persistence data in database. It helps in solving various problems of the developer, such as how to interact with a database connection, how to make sure that the connection is

closed, how to deal with exceptions, and how to implement transaction management. It also enables the developers to easily write code to access the persistence data throughout the application.

- c. **Spring MVC framework:** Allows you to build Web applications based on MVC architecture. All the requests made by a user first go through the controller and are then dispatched to different views, that is, to different JSP pages or Servlets. The form handling and form validating features of the Spring MVC framework can be easily integrated with all popular view technologies such as JSP, Jasper Report, FreeMarker, and Velocity.
- d. **Transaction management:** Helps in handling transaction management of an application without affecting its code. This framework provides Java Transaction API (JTA) for global transactions managed by an application server and local transactions managed by using the JDBC Hibernate, Java Data Objects (JDO), or other data access APIs. It enables the developer to model a wide range of transactions on the basis of Spring's declarative and programmatic transaction management.
- e. **Spring Web Service:** Generates Web service endpoints and definitions based on Java classes, but it is difficult to manage them in an application. To solve this problem, Spring Web Service provides layered-based approaches that are separately managed by Extensible Markup Language (XML) parsing (the technique of reading and manipulating XML). Spring provides effective mapping for transmitting incoming XML message request to an object and the developer to easily distribute XML message (object) between two machines.
- f. **JDBC abstraction layer:** Helps the users in handling errors in an easy and efficient manner. The JDBC programming code can be reduced when this abstraction layer is implemented in a Web application. This layer handles exceptions such as `DriverNotFound`. All `SQLExceptions` are translated into the `DataAccessException` class.

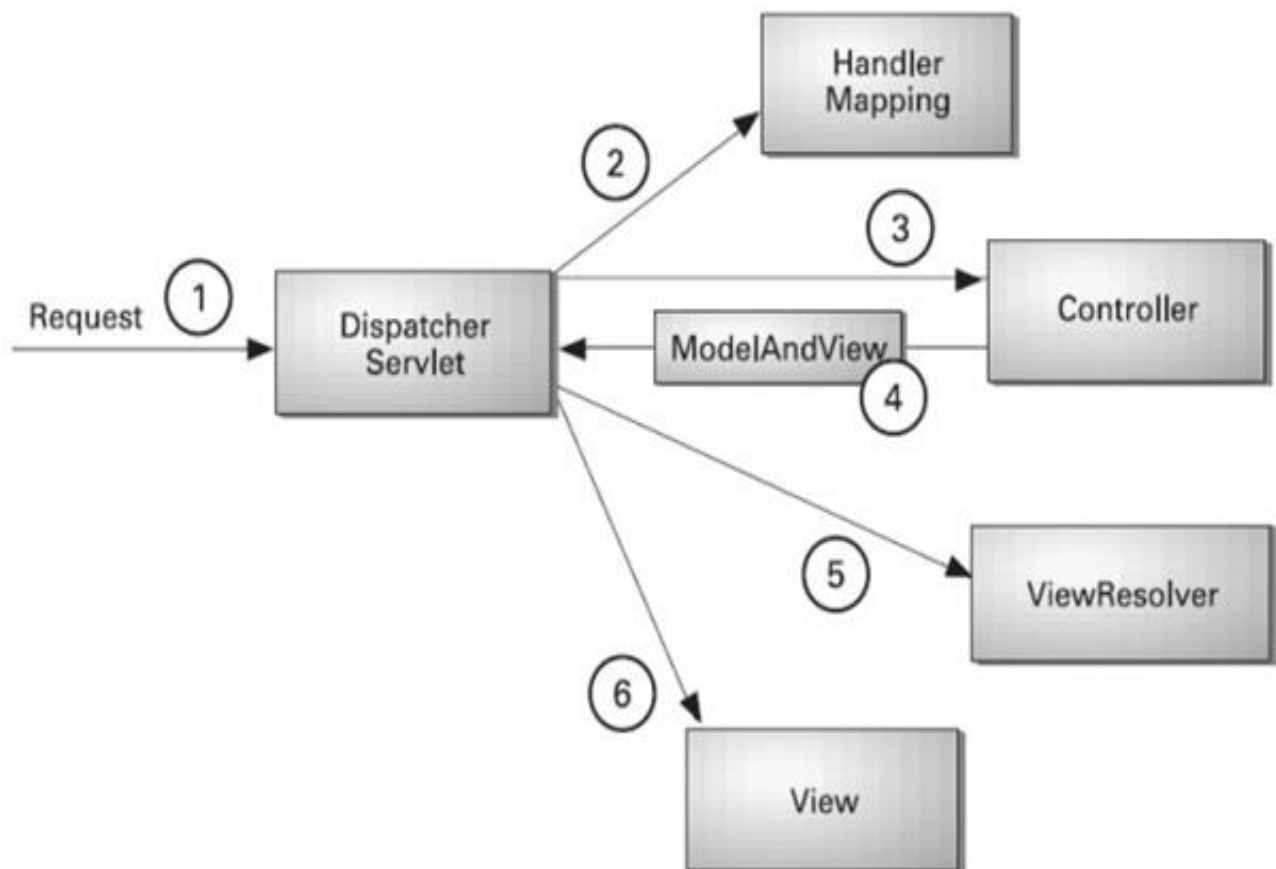
Spring's data access exception is not JDBC specific and hence Data Access Objects (DAO) are not bound to JDBC only.

- g. **Spring TestContext framework:** Provides facilities of unit and integration testing for the Spring applications. Moreover, the Spring TestContext framework provides specific integration testing functionalities such as context management and caching DI of test fixtures, and transactional test management with default rollback semantics.
- **8.2 Spring MVC**
 - A Spring MVC is a Java framework which is used to build web applications. It follows the Model-View-Controller design pattern. It implements all the basic features of a core spring framework like Inversion of Control, Dependency Injection.
 - A Spring MVC provides an elegant solution to use MVC in spring framework by the help of **DispatcherServlet**. Here, **DispatcherServlet** is a class that receives the incoming request and maps it to the right resource such as controllers, models, and views.



- I. Spring MVC is a web framework based on the MVC design pattern.
- II. **Example:**

```
@Controller  
  
public class MyController {  
    @RequestMapping("/hello")  
    public String sayHello(Model model) {  
        model.addAttribute("message", "Hello, World!");  
        return "hello";  
    }  
}
```



Module 9: Hibernate

- Hibernate is a Java framework, licensed under the open-source GNU Lesser General Public License (LGPL), and is available for free download. Developed in 2001 by Gavin King, hibernate was introduced as a groundbreaking alternative to the EJB2-style entity bean approach.
- By mapping Java objects to database tables, it streamlines data persistence and retrieval without the need for complex SQL queries. With features like automatic transaction management and caching, hibernate simplifies and optimizes database interactions, making it an indispensable framework for efficient data management in Java applications

• 9.1 Introduction to Hibernate

- I. Hibernate is an Object-Relational Mapping (ORM) framework for managing database interactions.
- II. **Example:**

```
// Hibernate Configuration  
Configuration cfg= new Configuration();  
cfg.configure("hibernate.cfg.xml");  
SessionFactory factory = cfg.buildSessionFactory();
```

• 9.2 Mapping in Hibernate

- I. Hibernate maps Java classes to database tables and supports querying with HQL (Hibernate Query Language).

II. Example:

```
@Entity

public class User {

    @Id

    @GeneratedValue(strategy = GenerationType.IDENTITY)

    private Long id;

    private String username;

    // Getters and setters

}
```

Hibernate vs. JDBC comparison

TOOL	HIBERNATE JPA IMPLEMENTATION	JDBC
Purpose	Object-relational mapping (ORM)	Database Connectivity
Query Language	Hibernate Query Language (HQL) and the Java Persistence Query Language (JPQL)	Structured Query Language (SQL)
License	LGPL 2.1 and ASL 2.0	Oracle license
Packaging	Standalone JAR file along with dependent libraries	Part of Java SE
Caching	Built-in second-level caching	No caching
Competitors	TopLink, EclipseLink, OpenJPA and ActiveJDBC	ODBC, JDO
Maintainer	JBoss by Red Hat	Oracle
Release	Initial Hibernate release in 2001; JPA 1.0 released in 2006	Released in 1997 as part of Java 1.1

Module 10: RESTful Web Services

• 10.1 Introduction to REST

- I. REST (Representational State Transfer) is an architectural style for designing networked applications.

II. Example:

```
@Path("/users")

public class UserService {

    @GET

    @Produces(MediaType.APPLICATION_JSON)

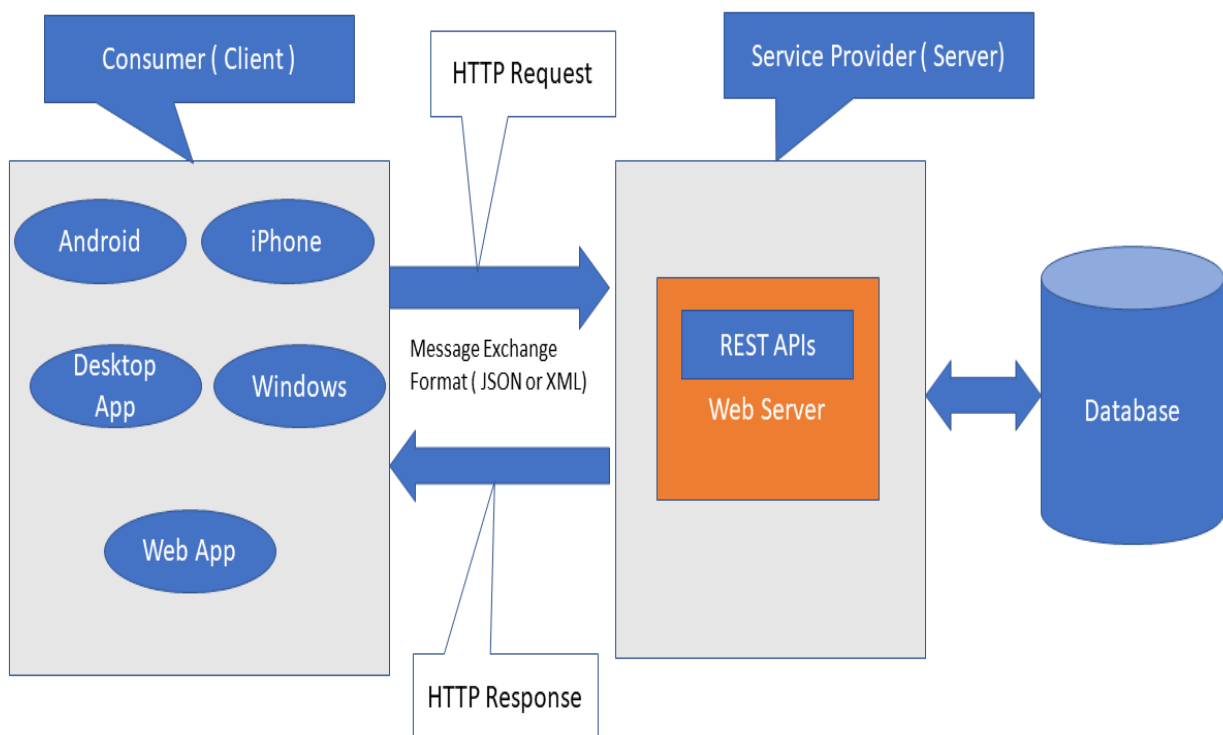
    public List<User> getUsers() {

        // Return list of users

    }

}
```

REST – Architecture

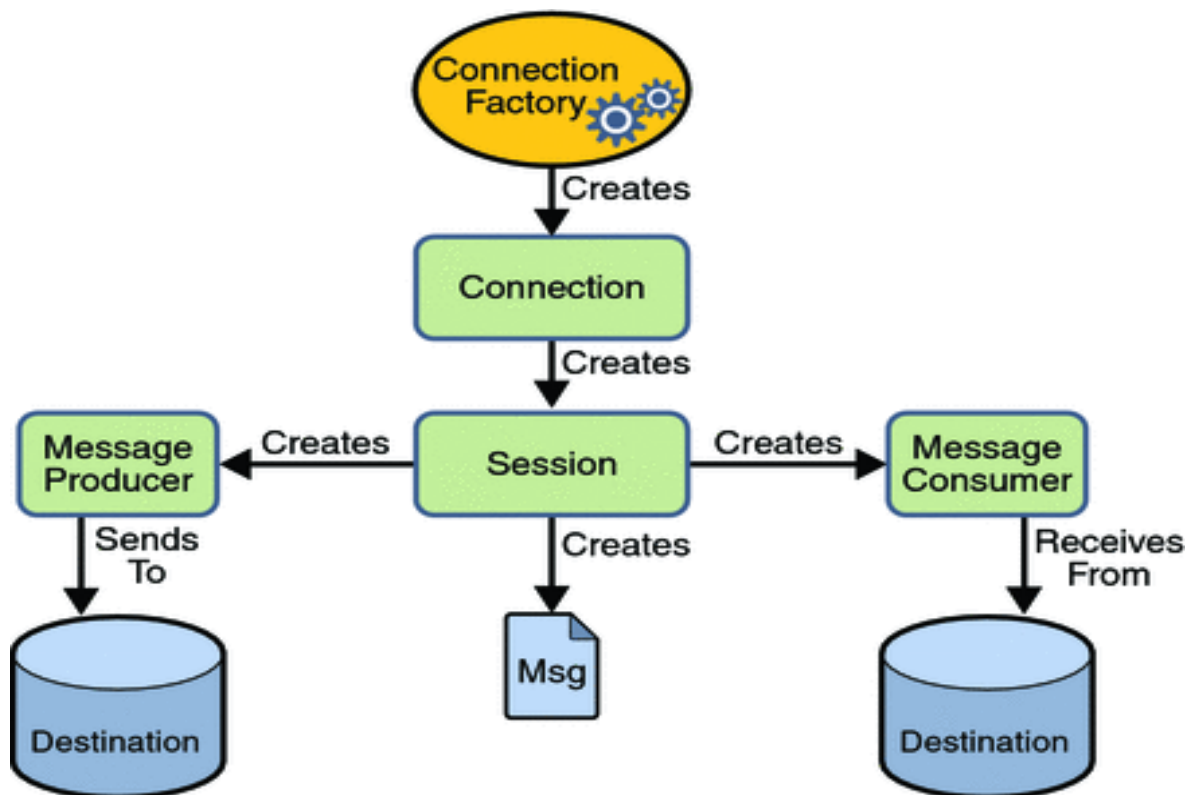


Module 11: Java Message Service (JMS)

- 11.1 Overview of JMS

Advantage of JMS

- Asynchronous:** To receive the message, client is not required to send request. Message will arrive automatically to the client.
- Reliable:** It provides assurance that message is delivered.



- JMS is an API for messaging between Java applications. It supports point-to-point and publish/subscribe models.
- Example:**

```
// JMS Example

ConnectionFactory factory = new
ActiveMQConnectionFactory("tcp://localhost:61616");

Connection connection = factory.createConnection();

Session session = connection.createSession(false,
Session.AUTO_ACKNOWLEDGE);

Destination destination = session.createQueue("TEST.FOO");

MessageProducer producer = session.createProducer(destination);

TextMessage message = session.createTextMessage("Hello World");

producer.send(message);
```

Module 12: Java Authentication and Authorization Service (JAAS)

➤ JAAS can be used for two purposes:

- for *authentication* of users, to reliably and securely determine who is currently executing Java code, regardless of whether the code is running as an application, an applet, a bean, or a servlet; and
- for *authorization* of users to ensure they have the access control rights (permissions) required to do the actions performed.

• 12.1 Security in Java

I. JAAS provides a way for Java applications to authenticate and authorize users.

II. **Example:**

```
// JAAS LoginModule Example

LoginContext lc = new LoginContext("MyLoginModule");

lc.login();
```

Module 13: Design Patterns in Java

Advantage of design pattern:

- They are reusable in multiple projects.
- They provide the solutions that help to define the system architecture.
- They capture the software engineering experiences.
- They provide transparency to the design of an application.
- They are well-proved and testified solutions since they have been built upon the knowledge and experience of expert software developers.

Creational Design Patterns	Structural Design Patterns	Behavioural Design Patterns
• Singleton Pattern • Factory Pattern • Abstract Factory Pattern • Builder Pattern • Prototype Pattern	• Adapter Pattern • Composite Pattern • Proxy Pattern • Flyweight Pattern • Façade Pattern • Bridge Pattern • Decorator Pattern	• Template Method Pattern • Mediator Pattern • Chain of Responsibility Pattern • Observer Pattern • Strategy Pattern • Command Pattern • State Pattern • Visitor Pattern • Interpreter Pattern • Iterator Pattern • Memento Pattern

• 13.1 Common Design Patterns

I. Design patterns are standard solutions to common problems in software design.

II. **Example:**

```
// Factory Pattern  
  
public class ShapeFactory {
```

```
public Shape getShape(String shapeType) {  
    if (shapeType == null) {  
        return null;  
    }  
    if (shapeType.equalsIgnoreCase("CIRCLE")) {  
        return new Circle();  
    } else if (shapeType.equalsIgnoreCase("SQUARE")) {  
        return new Square();  
    }  
    return null;  
}
```

Module 14: Introduction to Microservices

- **14.1 Microservices Architecture**

- I. Microservices architecture involves building applications as a set of loosely coupled services, each responsible for a specific function.
- II. **Example:** Implementing microservices using Spring Boot and Docker.

1. Introduction to Python

Overview of Python

- **High-Level Language:** Python abstracts complex low-level operations, making it easier to write and understand code.
- **Interpreted Language:** Python code is executed line-by-line, which aids in debugging and development speed.
- **Dynamic Typing:** Variable types are determined at runtime, allowing for more flexible code.
- **Readable Syntax:** Python emphasizes readability with its clear and concise syntax, resembling plain English.
- **Versatility:** Suitable for web development, data analysis, artificial intelligence, scientific computing, automation, and more.

Installing Python

- **Download from Python.org:**
 - Visit python.org to download the latest version.
 - Follow installation instructions specific to your operating system (Windows, macOS, Linux).
- **Setting Up Environment Variables:**
 - Ensure Python is added to your system's PATH to run Python from the terminal.
- **Introduction to IDEs (Integrated Development Environments):**
 - **PyCharm:** Feature-rich IDE with advanced debugging and project management.

Full Stack Development [Unit-II]

- **VSCode:** Lightweight, extensible editor with Python extensions.
- **Jupyter Notebook:** Interactive environment ideal for data science and exploratory programming.
- **IDLE:** Simple, built-in Python IDE suitable for beginners.

Running Python Programs

- **Terminal/Command Prompt:**
 - Navigate to the directory containing your Python script.
 - Execute scripts using the command: `python script_name.py`.
- **Interactive Mode:**
 - Launch the Python shell by typing `python` in the terminal.
 - Execute Python commands line-by-line for quick testing and experimentation.

Example: Running a Python Script

```
# To run a Python file from the terminal
```

```
$ python hello_world.py
```

2. Python Basics

Variables and Data Types

- **Variables:**
 - Containers for storing data values.

Full Stack Development [Unit-II]

- No need to declare the type explicitly; Python infers it.
- **Data Types:**
 - **Integer (int):** Whole numbers (e.g., 5, -3).
 - **Float (float):** Decimal numbers (e.g., 3.14, -0.001).
 - **String (str):** Sequence of characters enclosed in quotes (e.g., "Hello").
 - **Boolean (bool):** True or False values.
 - **List (list):** Ordered, mutable collections (e.g., [1, 2, 3]).
 - **Tuple (tuple):** Ordered, immutable collections (e.g., (1, 2, 3)).
 - **Dictionary (dict):** Key-value pairs (e.g., {"name": "Alice"}).
 - **Set (set):** Unordered collections of unique elements (e.g., {1, 2, 3}).

Example: Variables and Data Types

```
name = "Alice"    # str
```

```
age = 30          # int
```

```
height = 5.6      # float
```

```
is_student = False # bool
```

Operators

- **Arithmetic Operators:** Perform mathematical operations.
 - + (Addition), - (Subtraction), * (Multiplication), / (Division), // (Floor Division), % (Modulus), ** (Exponentiation).

Full Stack Development [Unit-II]

- **Comparison Operators:** Compare two values and return a Boolean.

- `==, !=, >, <, >=, <=`.

- **Logical Operators:** Combine conditional statements.

- `and, or, not`.

Example: Operators

```
result = (5 + 3) * 2 # Arithmetic operation: (8) * 2 = 16
```

```
print(5 > 3 and 3 < 8) # Logical operation: True and True = True
```

3. Control Flow

If-Else Statements

- **Conditional Execution:** Execute code blocks based on conditions.
- **Syntax:**

```
if condition:
```

```
    # code if condition is True
```

```
elif another_condition:
```

```
    # code if another_condition is True
```

```
else:
```

```
    # code if all above conditions are False
```

Full Stack Development [Unit-II]

Example: If-Else Statement

```
age = 20
```

```
if age >= 18:
```

```
    print("You are an adult.")
```

```
else:
```

```
    print("You are a minor.")
```

For and While Loops

- **For Loops:** Iterate over a sequence (e.g., list, range).

```
for item in sequence:
```

```
    # code to execute
```

- **While Loops:** Repeat as long as a condition is True.

```
while condition:
```

```
    # code to execute
```

Example: For and While Loops

```
for i in range(5):
```

```
    print(i)
```

```
# while loop
```

```
count = 0
```

```
while count < 5:
```

Full Stack Development [Unit-II]

```
print(count)
```

```
count += 1
```

Break and Continue Statements

- **Break:** Exit the current loop immediately.
- **Continue:** Skip the rest of the current loop iteration and continue with the next iteration.

Example: Break Statement

```
for i in range(10):
```

```
    if i == 5:
```

```
        break # Exits the loop when i is 5
```

```
    print(i)
```

Example: Continue Statement

```
for i in range(10):
```

```
    if i % 2 == 0:
```

```
        continue # Skips even numbers
```

```
    print(i)
```

4. Functions

Defining and Calling Functions

- **Purpose:** Encapsulate reusable code blocks.

Full Stack Development [Unit-II]

- **Syntax:**

```
def function_name(parameters):
```

```
    # function body
```

```
    return value
```

- **Calling a Function:**

```
function_name(arguments)
```

Example: Defining and Calling Functions

```
def greet(name):
```

```
    return f"Hello, {name}"
```

```
print(greet("Alice"))
```

****Default Arguments, *args, and kwargs**

- **Default Arguments:** Assign default values to parameters.

```
def function(param1, param2=default_value):
```

```
    # function body
```

- ***args:** Accept a variable number of non-keyword arguments.

```
def function(*args):
```

```
    # args is a tuple of arguments
```

- ***kwargs:** Accept a variable number of keyword arguments.

Full Stack Development [Unit-II]

```
def function(**kwargs):
```

```
    # kwargs is a dictionary of arguments
```

****Example: Default Arguments, *args, and kwargs**

```
def add(a, b=0):
```

```
    return a + b
```

```
def dynamic_args(*args):
```

```
    return sum(args)
```

```
def dynamic_kwargs(**kwargs):
```

```
    return kwargs
```

```
print(add(5))          # Uses default value for b: 5 + 0 = 5
```

```
print(dynamic_args(1, 2, 3)) # Sums all arguments: 6
```

```
print(dynamic_kwargs(name="Bob", age=25)) # {'name': 'Bob', 'age': 25}
```

Lambda Functions

- **Anonymous Functions:** Functions without a name, defined using the lambda keyword.
- **Syntax:**

lambda parameters: expression

Example: Lambda Functions

```
square = lambda x: x ** 2
```

```
print(square(4)) # Output: 16
```

```
# Using lambda with map

numbers = [1, 2, 3, 4]

squared = list(map(lambda x: x ** 2, numbers))

print(squared) # Output: [1, 4, 9, 16]
```

5. Data Structures

Lists

- **Definition:** Ordered, mutable collections of items.
- **Characteristics:**
 - Can contain mixed data types.
 - Supports indexing and slicing.
- **Common Methods:**
 - `append()`: Add an item to the end.
 - `pop()`: Remove and return an item.
 - `sort()`: Sort the list.
 - `reverse()`: Reverse the list.

Example: Lists

Full Stack Development [Unit-II]

```
my_list = [1, 2, 3, 4]

my_list.append(5) # [1, 2, 3, 4, 5]

my_list.pop()    # Removes 5, list becomes [1, 2, 3, 4]

my_list.sort(reverse=True) # [4, 3, 2, 1]

print(my_list)
```

Tuples

- **Definition:** Ordered, immutable collections of items.
- **Characteristics:**
 - Cannot be modified after creation.
 - Useful for fixed collections of items.
- **Usage:**
 - Return multiple values from functions.
 - Use as keys in dictionaries (if containing only immutable types).

Example: Tuples

```
my_tuple = (1, 2, 3)

print(my_tuple[0]) # Output: 1

# Attempting to modify will raise an error

# my_tuple[0] = 10 # TypeError
```

Dictionaries

- **Definition:** Unordered, mutable collections of key-value pairs.
- **Characteristics:**
 - Keys must be unique and immutable types.
 - Values can be of any data type.
- **Common Methods:**
 - `keys()`: Returns a view of keys.
 - `values()`: Returns a view of values.
 - `items()`: Returns a view of key-value pairs.
 - `get()`: Retrieves value for a key with a default option.

Example: Dictionaries

```
my_dict = {"name": "Alice", "age": 25}

print(my_dict["name"]) # Output: Alice

my_dict["age"] = 26    # Update value

my_dict["city"] = "NY" # Add new key-value pair

print(my_dict)
```

Sets

- **Definition:** Unordered collections of unique elements.
- **Characteristics:**

Full Stack Development [Unit-II]

- No duplicate elements.
- Efficient for membership testing.
- **Common Methods:**
 - `add()`: Add an element.
 - `remove()`: Remove an element (raises `KeyError` if not found).
 - `discard()`: Remove an element if present.
 - `union()`, `intersection()`, `difference()`: Set operations.

Example: Sets

```
my_set = {1, 2, 3, 4}

my_set.add(5)      # {1, 2, 3, 4, 5}

my_set.remove(3)   # {1, 2, 4, 5}

my_set.discard(10) # No error, even if 10 is not present

print(my_set)

# Set operations

set_a = {1, 2, 3}

set_b = {3, 4, 5}

print(set_a.union(set_b))    # {1, 2, 3, 4, 5}

print(set_a.intersection(set_b)) # {3}

print(set_a.difference(set_b)) # {1, 2}
```

6. File Handling

Opening, Reading, and Writing Files

- **Opening Files:** Use `open()` with mode ('r', 'w', 'a', 'rb', etc.).
- **Reading Files:**
 - `read()`: Read the entire file.
 - `readline()`: Read one line at a time.
 - `readlines()`: Read all lines into a list.
- **Writing Files:**
 - `write()`: Write a string to the file.
 - `writelines()`: Write a list of strings to the file.
- **Closing Files:** Always close files to free resources (`file.close()`).

Using Context Managers with with Keyword

- **Benefits:**
 - Automatically handles opening and closing files.
 - Ensures files are properly closed even if errors occur.

- **Syntax:**

with `open("filename", "mode")` as file:

```
# Perform file operations
```

Full Stack Development [Unit-II]

Example: File Handling

Writing to a file

with open("example.txt", "w") as file:

file.write("Hello, World!\n")

file.write("This is a second line.")

Reading from a file

with open("example.txt", "r") as file:

content = file.read()

print(content)

Output:

Hello, World!

This is a second line.

Additional File Operations

- **Appending to Files:**

with open("example.txt", "a") as file:

file.write("\nAppending a new line.")

- **Reading Line by Line:**

with open("example.txt", "r") as file:

for line in file:

```
print(line.strip())
```

7. Object-Oriented Programming (OOP)

Classes and Objects

- **Class:** Blueprint for creating objects (instances).
- **Object:** An instance of a class containing data and methods.
- **Attributes:** Variables that belong to a class or instance.
- **Methods:** Functions that belong to a class.

Syntax:

```
class ClassName:

    def __init__(self, parameters):

        # Initialize attributes

    def method_name(self, parameters):

        # Define method behavior
```

Example: Classes and Objects

```
class Person:

    def __init__(self, name, age):

        self.name = name # Instance attribute

        self.age = age # Instance attribute
```

Full Stack Development [Unit-II]

```
def greet(self):  
  
    print(f"Hello, my name is {self.name} and I am {self.age} years old.")  
  
# Creating an object  
  
person1 = Person("Alice", 30)  
  
person1.greet() # Output: Hello, my name is Alice and I am 30 years old.
```

Inheritance

- **Definition:** Creating a new class that inherits attributes and methods from an existing class.
- **Benefits:**
 - Reusability of code.
 - Logical hierarchy and organization.
- **Syntax**
-

```
class ChildClass(ParentClass):  
  
    def __init__(self, additional_parameters):  
  
        super().__init__(parent_parameters)  
  
    # Initialize additional attributes
```

Example: Inheritance

```
class Employee(Person):  
  
    def __init__(self, name, age, salary):
```

Full Stack Development [Unit-II]

```
super().__init__(name, age) # Initialize attributes from Person

self.salary = salary      # Additional attribute

def display_salary(self):

    print(f"{self.name} earns ${self.salary} per year.")

# Creating an Employee object

emp = Employee("Bob", 25, 50000)

emp.greet()              # Inherited method

emp.display_salary() # Output: Bob earns $50000 per year.
```

Polymorphism and Encapsulation

- **Polymorphism:**
 - **Definition:** Ability to present the same interface for different underlying forms (data types).
 - **Example:** Different classes having methods with the same name.

```
class Cat:

    def sound(self):

        print("Meow")

class Dog:

    def sound(self):

        print("Bark")
```



```
animals = [Cat(), Dog()]
```

```
for animal in animals:
```

```
    animal.sound()
```

```
# Output:
```

```
# Meow
```

```
# Bark
```

- **Encapsulation:**

- **Definition:** Restricting access to certain components of an object.

- **Private Attributes and Methods:** Prefix with double underscores (__).

```
class Secret:
```

```
    def __init__(self):
```

```
        self.__private = "This is private"
```

```
    def get_private(self):
```

```
        return self.__private
```

```
s = Secret()
```

```
# print(s.__private) # AttributeError
```

```
print(s.get_private()) # Output: This is private
```

8. Exception Handling

Try, Except, Finally

- **Purpose:** Handle errors gracefully without crashing the program.
- **Structure:**

try:

 # Code that may raise an exception

except SpecificException:

 # Handle specific exception

except AnotherException as e:

 # Handle another exception with error details

else:

 # Execute if no exceptions occur

finally:

 # Execute regardless of exceptions

Example: Try, Except, Finally

try:

 x = 10 / 0

except ZeroDivisionError:

Full Stack Development [Unit-II]

```
print("Cannot divide by zero!")
```

finally:

```
print("This will always run.")
```

Output:

Cannot divide by zero!

This will always run.

Custom Exceptions

- **Definition:** User-defined exceptions to handle specific error scenarios.
- **Creating Custom Exceptions:**

```
class MyError(Exception):
```

- **Raising Custom Exceptions:**

```
raise MyError("This is a custom error")
```

Example: Custom Exceptions

```
class MyError(Exception):
```

```
    """Custom exception for specific error cases."""
```

```
    def __init__(self, message):
```

```
        super().__init__(message)
```

```
        self.message = message
```

```
try:
```

Full Stack Development [Unit-II]

```
raise MyError("This is a custom error")
```

```
except MyError as e:
```

```
print(e) # Output: This is a custom error
```

Best Practices in Exception Handling

- **Catch Specific Exceptions:** Avoid using bare except: clauses.
 - **Use Finally for Cleanup:** Ensure resources are released properly.
 - **Avoid Silent Failures:** Always handle exceptions appropriately or log them.
-

9. Modules and Packages

Importing Modules

- **Standard Library Modules:** Pre-installed modules that come with Python (e.g., math, datetime).
- **Third-Party Modules:** External modules installed via package managers like pip (e.g., requests, numpy).
- **Import Statements:**

- Import entire module:

```
import math
```

```
print(math.sqrt(16)) # Output: 4.0
```

- Import specific functions:

```
from math import sqrt
```

Full Stack Development [Unit-II]

```
print(sqrt(16)) # Output: 4.0
```

- Import with alias:

```
import numpy as np
```

```
array = np.array([1, 2, 3])
```

Creating and Using Packages

- **Definition:** A way to organize related modules into a directory hierarchy.
- **Structure**

```
mypackage/
```

```
__init__.py
```

```
module1.py
```

```
module2.py
```

- **__init__.py:** Makes Python treat directories as packages. Can be empty or execute initialization code.
- **Using Packages:**

```
from mypackage.module1 import function1
```

```
function1()
```

Example: Creating and Using Packages

```
# Directory structure:
```

```
# mypackage/
```

Full Stack Development [Unit-II]

```
# __init__.py

# module.py

# In module.py:

def say_hello():

    return "Hello from the module!"

# Using the package in another script

from mypackage.module import say_hello

print(say_hello()) # Output: Hello from the module!
```

Creating Your Own Modules

- **Definition:** Python files containing functions, classes, or variables that can be imported.
- **Example:**

```
# my_module.py

def add(a, b):

    return a + b

# main.py

import my_module

print(my_module.add(2, 3)) # Output: 5
```

Installing Third-Party Packages with pip

- **Using pip:**

Full Stack Development [Unit-II]

```
pip install package_name
```

- **Example:**

```
pip install requests
```

- **Importing Installed Packages:**

```
import requests
```

```
response = requests.get("https://api.example.com")
```

```
print(response.status_code)
```

10. Advanced Python Concepts

List Comprehensions

- **Definition:** Concise way to create lists.
- **Syntax:**


```
[expression for item in iterable if condition]
```
- **Benefits:** More readable and efficient than traditional loops.

Example: List Comprehensions

```
squares = [x**2 for x in range(5)]
```

```
print(squares) # Output: [0, 1, 4, 9, 16]
```

```
# With condition
```

```
even_squares = [x**2 for x in range(10) if x % 2 == 0]
```

Full Stack Development [Unit-II]

```
print(even_squares) # Output: [0, 4, 16, 36, 64]
```

Generators

- **Definition:** Functions that return an iterator, yielding one item at a time using the yield keyword.
- **Benefits:**
 - Memory-efficient for large datasets.
 - Lazy evaluation (values are generated on the fly).

Example: Generators

```
def count_up_to(max_value):
```

```
    count = 1
```

```
    while count <= max_value:
```

```
        yield count
```

```
        count += 1
```

```
# Using the generator
```

```
for value in count_up_to(5):
```

```
    print(value)
```

```
# Output:
```

```
# 1
```

```
# 2
```


Full Stack Development [Unit-II]

3

4

5

Generator Expression

```
squares_gen = (x**2 for x in range(5))
```

```
for square in squares_gen:
```

```
    print(square)
```

Decorators

- **Definition:** Higher-order functions that modify the behavior of other functions or methods.
- **Syntax:**

```
def decorator_function(original_function):
```

```
    def wrapper_function(*args, **kwargs):
```

```
        # Modify behavior
```

```
        result = original_function(*args, **kwargs)
```

```
        # Modify behavior
```

```
        return result
```

```
    return wrapper_function
```

```
@decorator_function
```

```
def display():
```

Full Stack Development [Unit-II]

```
print("Function is running")
```

```
display()
```

- **Use Cases:**
 - Logging
 - Access control and authentication
 - Caching
 - Timing functions

Example: Decorators

```
def decorator_function(original_function):  
  
    def wrapper_function(*args, **kwargs):  
  
        print("Before function call")  
  
        result = original_function(*args, **kwargs)  
  
        print("After function call")  
  
        return result  
  
    return wrapper_function  
  
@decorator_function  
  
def display():  
  
    print("Function is running")  
  
display()
```

Full Stack Development [Unit-II]

Output:

Before function call

Function is running

After function call

Context Managers

- **Definition:** Manage resources (like files) using with statements.
- **Creating Custom Context Managers:**
 - Using classes with `__enter__` and `__exit__` methods.
 - Using generator functions with `contextlib` module.

Example: Custom Context Manager Using Class

```
class MyContext:

    def __enter__(self):

        print("Entering the context")

        return self

    def __exit__(self, exc_type, exc_val, exc_tb):

        print("Exiting the context")

with MyContext():

    print("Inside the context")
```

Output:

Full Stack Development [Unit-II]

Entering the context

Inside the context

Exiting the context

Metaclasses

- **Definition:** Classes of classes that define how classes behave.
- **Use Cases:**
 - Enforcing coding standards.
 - Automatic resource management.
 - Advanced framework development.

Note: Metaclasses are an advanced topic and typically used in complex scenarios.

11. Python Libraries

NumPy (Numerical Python)

- **Purpose:** Efficient numerical computations and array manipulations.
- **Key Features:**
 - N-dimensional arrays (ndarray).
 - Mathematical functions for array operations.
 - Linear algebra, Fourier transforms, and random number generation.
- **Basic Usage:**

Full Stack Development [Unit-II]

```
import numpy as np

array = np.array([1, 2, 3, 4])

print(array * 2) # Output: [2 4 6 8]
```

Pandas (Data Analysis)

- **Purpose:** Data manipulation and analysis, especially for tabular data.
- **Key Features:**
 - DataFrames and Series data structures.
 - Data cleaning, transformation, and aggregation.
 - Integration with other data sources (CSV, Excel, SQL).
- **Basic Usage:**

```
import pandas as pd

df = pd.read_csv("data.csv")

print(df.head())

df['new_column'] = df['existing_column'] * 2
```

Matplotlib (Plotting)

- **Purpose:** Data visualization through static, animated, and interactive plots.
- **Key Features:**
 - Plot types: Line, bar, scatter, histogram, etc.

Full Stack Development [Unit-II]

- Customization options for plots.
- Integration with Pandas and NumPy.
- **Basic Usage:**

```
import matplotlib.pyplot as plt
```

```
x = [1, 2, 3, 4]
```

```
y = [10, 20, 25, 30]
```

```
plt.plot(x, y)
```

```
plt.xlabel('X-axis')
```

```
plt.ylabel('Y-axis')
```

```
plt.title('Simple Line Plot')
```

```
plt.show()
```

Requests

- **Purpose:** Sending HTTP requests and interacting with web APIs.
- **Key Features:**
 - Simplified HTTP methods (GET, POST, PUT, DELETE).
 - Handling responses and status codes.
 - Session management and cookies.
- **Basic Usage:**

```
import requests

response = requests.get("https://api.example.com/data")

if response.status_code == 200:

    data = response.json()

    print(data)

else:

    print("Failed to retrieve data")
```

Additional Useful Libraries

- **BeautifulSoup:** Web scraping and parsing HTML/XML.
- **Flask/Django:** Web development frameworks.
- **TensorFlow/PyTorch:** Machine learning and deep learning frameworks.
- **SQLAlchemy:** Database ORM (Object-Relational Mapping).

12. Testing and Debugging

Unit Testing with unittest

- **Purpose:** Verify that individual units of code work as intended.
- **Key Concepts:**
 - **Test Case:** A single unit of testing.

Full Stack Development [Unit-II]

- **Test Suite:** Collection of test cases.
- **Assertions:** Methods to check for expected outcomes.
- **Basic Usage:**

```
import unittest

def add(a, b):

    return a + b

class TestAddFunction(unittest.TestCase):

    def test_add_positive_numbers(self):

        self.assertEqual(add(2, 3), 5)

    def test_add_negative_numbers(self):

        self.assertEqual(add(-1, -1), -2)

    def test_add_zero(self):

        self.assertEqual(add(0, 5), 5)

if __name__ == '__main__':

    unittest.main()
```

Example: Running Tests

```
$ python test_add.py
```

```
-----

Ran 3 tests in 0.001s
```


OK

Debugging with pdb

- **Purpose:** Python's built-in interactive debugger.
- **Key Features:**
 - Set breakpoints.
 - Step through code.
 - Inspect variables and evaluate expressions.

- **Basic Usage:**

```
import pdb
```

```
def faulty_add(a, b):
```

```
    pdb.set_trace() # Sets a breakpoint
```

```
    return a + b
```

```
result = faulty_add(2, '3') # This will raise a TypeError
```

```
print(result)
```

- **Common pdb Commands:**
 - n (next): Execute the next line.
 - c (continue): Continue execution until the next breakpoint.
 - l (list): Display source code around the current line.
 - p (print): Print the value of a variable.

Full Stack Development [Unit-II]

- q (quit): Exit the debugger.

Example: Using pdb

```
import pdb

def divide(a, b):

    pdb.set_trace()

    return a / b

print(divide(10, 0))
```

- **Execution Flow:**
 - The debugger will pause at `pdb.set_trace()`.
 - You can inspect the values of `a` and `b`.
 - Step through the code to observe the error.

Best Practices in Testing and Debugging

- **Write Clear and Concise Tests:** Each test should focus on a single functionality.
- **Use Meaningful Test Names:** Describe what the test is verifying.
- **Automate Testing:** Integrate tests into development workflows (e.g., Continuous Integration).
- **Handle Exceptions Gracefully:** Use try-except blocks to manage unexpected errors.
- **Use Logging:** Implement logging to track the application's behavior and debug issues.

Module 1: Introduction to Databases and MySQL

1.1 Understanding Databases

What is a Database? A database is an organized collection of data, structured to facilitate efficient storage, retrieval, and management. Databases are essential for various applications, from simple record-keeping systems to complex enterprise solutions. They ensure data integrity, consistency, and accessibility.

Types of Databases (Relational vs. Non-Relational)

- **Relational Databases (RDBMS):** Utilize tables to store data, with relationships defined between tables using keys. They use Structured Query Language (SQL) for data manipulation. Examples include MySQL, PostgreSQL, and SQL Server.
- **Non-Relational Databases (NoSQL):** Designed for flexibility and scalability, they store data in formats like documents, key-value pairs, wide-columns, or graphs. Examples include MongoDB, Redis, and Cassandra. They are ideal for handling large volumes of unstructured data.

1.2 Introduction to MySQL

History and Evolution of MySQL MySQL was originally developed by MySQL AB in 1995 and has since evolved into one of the most popular open-source relational database management systems. Acquired by Sun Microsystems in 2008 and subsequently by Oracle Corporation in 2010, MySQL continues to be widely used due to its reliability, performance, and ease of use.

MySQL vs. Other RDBMS (e.g., PostgreSQL, SQL Server)

Full Stack Development [Unit-II]

- **MySQL:** Known for its speed and ease of use, it is widely adopted in web applications. It offers various storage engines, with InnoDB being the default for transactional support.
- **PostgreSQL:** Renowned for its advanced features, extensibility, and compliance with SQL standards. It supports complex queries, transactions, and custom data types.
- **SQL Server:** A proprietary RDBMS developed by Microsoft, offering deep integration with other Microsoft products and robust business intelligence tools.

1.3 Setting Up MySQL Environment

Installation on Windows, macOS, and Linux Installing MySQL involves downloading the appropriate installer for your operating system from the [official MySQL website](#). The installation process typically includes setting up the MySQL server, configuring security settings, and installing MySQL Workbench for a graphical interface.

MySQL Workbench Overview MySQL Workbench is a unified visual tool for database architects, developers, and DBAs. It provides functionalities for database design, SQL development, administration, and performance tuning, making it easier to manage MySQL databases.

Command-Line Interface Basics The MySQL Command-Line Interface (CLI) allows direct interaction with the MySQL server using SQL commands. It is a powerful tool for executing queries, managing databases, and performing administrative tasks.

Example: Install MySQL on your operating system and connect using MySQL Workbench.

1. Download and Install MySQL:

- Visit the [MySQL Downloads](#) page.
- Choose the appropriate installer for your OS and follow the installation instructions.

2. Launch MySQL Workbench:

- Open MySQL Workbench and create a new connection using the credentials set during installation.
 - Test the connection to ensure successful setup.
-

Module 2: MySQL Basics

2.1 MySQL Architecture

Server Components MySQL architecture comprises several key components:

- **Client Layer:** Interfaces like MySQL Workbench or CLI through which users interact with the server.
- **Connection Manager:** Handles client connections and authentication.
- **SQL Parser and Optimizer:** Parses SQL queries and optimizes them for efficient execution.
- **Storage Engine:** Manages how data is stored, retrieved, and updated.

Storage Engines (InnoDB, MyISAM)

- **InnoDB:** The default storage engine, supporting transactions, foreign keys, and row-level locking. It ensures data integrity and is suitable for high-concurrency environments.
- **MyISAM:** An older storage engine that offers fast read operations but lacks transaction support and row-level locking. It is suitable for read-heavy applications.

2.2 Creating Databases and Tables

CREATE DATABASE and CREATE TABLE Statements

Full Stack Development [Unit-II]

- **CREATE DATABASE:** Used to create a new database.

```
CREATE DATABASE database_name;
```

- **CREATE TABLE:** Defines a new table within a database, specifying columns and their data types.

```
CREATE TABLE table_name (  
  
    column1 datatype,  
  
    column2 datatype,  
  
    ...  
  
);
```

Data Types in MySQL MySQL supports various data types, including:

- **Numeric Types:** INT, FLOAT, DECIMAL
- **Date and Time Types:** DATE, DATETIME, TIMESTAMP
- **String Types:** VARCHAR, TEXT, CHAR

2.3 Basic SQL Operations

SELECT, INSERT, UPDATE, DELETE

- **SELECT:** Retrieves data from one or more tables.

```
SELECT column1, column2 FROM table_name;
```

- **INSERT:** Adds new records to a table.

```
INSERT INTO table_name (column1, column2) VALUES (value1, value2);
```

- **UPDATE:** Modifies existing records in a table.

```
UPDATE table_name SET column1 = value1 WHERE condition;
```

- **DELETE:** Removes records from a table.

```
DELETE FROM table_name WHERE condition;
```

Filtering Data with WHERE Clause The WHERE clause filters records based on specified conditions.

```
SELECT * FROM table_name WHERE condition;
```

Sorting Results with ORDER BY The ORDER BY clause sorts the result set in ascending (ASC) or descending (DESC) order.

```
SELECT * FROM table_name ORDER BY column1 ASC, column2 DESC;
```

Example: Create a student's database and a student's table with fields like id, name, age, and grade.

Insert sample records and perform basic queries.

```
CREATE DATABASE students_db;
```

```
USE students_db;
```

```
CREATE TABLE students (
```

```
    id INT AUTO_INCREMENT PRIMARY KEY,
```

```
    name VARCHAR(100),
```

```
    age INT,
```

```
    grade CHAR(1)
```

```
);
```

```
INSERT INTO students (name, age, grade) VALUES
```

```
('Alice', 20, 'A'),
```

```
('Bob', 22, 'B'),
```

```
('Charlie', 19, 'C');
```

```
SELECT * FROM students WHERE age > 20 ORDER BY name DESC;
```

Module 3: Advanced SQL Queries

3.1 Joins and Relationships

Understanding Primary and Foreign Keys

- **Primary Key:** A unique identifier for each record in a table.
- **Foreign Key:** A field in one table that uniquely identifies a row in another table, establishing a relationship between the two tables.

Inner Join, Left Join, Right Join, Full Outer Join

- **Inner Join:** Returns records with matching values in both tables.

```
SELECT columns FROM table1 INNER JOIN table2 ON table1.id = table2.foreign_id;
```

- **Left Join (Left Outer Join):** Returns all records from the left table and matched records from the right table.

```
SELECT columns FROM table1 LEFT JOIN table2 ON table1.id = table2.foreign_id;
```


Full Stack Development [Unit-II]

- **Right Join (Right Outer Join):** Returns all records from the right table and matched records from the left table.

```
SELECT columns FROM table1 RIGHT JOIN table2 ON table1.id = table2.foreign_id;
```

- **Full Outer Join:** Returns all records when there is a match in either left or right table. (Note: MySQL does not support FULL OUTER JOIN directly; it can be emulated using UNION.)

3.2 Aggregation and Grouping

Aggregate Functions (COUNT, SUM, AVG, MIN, MAX)

- **COUNT:** Counts the number of rows.

```
SELECT COUNT(*) FROM table_name;
```

- **SUM:** Calculates the total sum of a numeric column.

```
SELECT SUM(column_name) FROM table_name;
```

- **AVG:** Calculates the average value of a numeric column.

```
SELECT AVG(column_name) FROM table_name;
```

- **MIN/MAX:** Finds the minimum or maximum value in a column.

```
SELECT MIN(column_name), MAX(column_name) FROM table_name;
```

GROUP BY and HAVING Clauses

- **GROUP BY:** Groups rows that have the same values in specified columns into summary rows.

```
SELECT column1, COUNT(*) FROM table_name GROUP BY column1;
```

- **HAVING:** Filters groups based on aggregate functions.

Full Stack Development [Unit-II]

```
SELECT column1, COUNT(*) FROM table_name GROUP BY column1 HAVING COUNT(*) > 5;
```

3.3 Subqueries and Nested Queries

Single-Row and Multi-Row Subqueries

- **Single-Row Subqueries:** Return only one row.

```
SELECT column FROM table WHERE column = (SELECT column FROM table WHERE condition);
```

- **Multi-Row Subqueries:** Return multiple rows and are used with operators like IN or EXISTS.

```
SELECT column FROM table WHERE column IN (SELECT column FROM table WHERE condition);
```

Correlated Subqueries Subqueries that reference columns from the outer query.

```
SELECT e1.name  
  
FROM employees e1  
  
WHERE e1.salary > (SELECT AVG(e2.salary) FROM employees e2 WHERE e2.department =  
e1.department);
```

Example: Create courses and enrollments tables and perform joins to fetch student-course relationships.

```
CREATE TABLE courses (  
  
    course_id INT AUTO_INCREMENT PRIMARY KEY,  
  
    course_name VARCHAR(100)  
  
);
```

Full Stack Development [Unit-II]

```
CREATE TABLE enrollments (  
  
    enrollment_id INT AUTO_INCREMENT PRIMARY KEY,  
  
    student_id INT,  
  
    course_id INT,  
  
    FOREIGN KEY (student_id) REFERENCES students(id),  
  
    FOREIGN KEY (course_id) REFERENCES courses(course_id)  
  
);  
  
SELECT students.name, courses.course_name  
  
FROM enrollments  
  
INNER JOIN students ON enrollments.student_id = students.id  
  
INNER JOIN courses ON enrollments.course_id = courses.course_id;
```

Module 4: Database Design and Normalization

4.1 Principles of Database Design

Entity-Relationship (ER) Modeling ER modeling is a systematic approach to designing databases by identifying entities, their attributes, and relationships between entities. It helps in visualizing the database structure before implementation.

Defining Entities and Relationships

- **Entities:** Objects or concepts that can have data stored about them (e.g., Students, Courses).

Full Stack Development [Unit-II]

- **Relationships:** Associations between entities (e.g., Enrollments link Students to Courses).

4.2 Normalization

Normalization is the process of organizing data to reduce redundancy and improve data integrity. It involves dividing large tables into smaller, related tables.

First Normal Form (1NF)

- Ensure that each table cell contains only atomic (indivisible) values.
- Each record is unique.

Second Normal Form (2NF)

- Achieve 1NF.
- Remove partial dependencies; non-key attributes must depend on the entire primary key.

Third Normal Form (3NF)

- Achieve 2NF.
- Remove transitive dependencies; non-key attributes must not depend on other non-key attributes.

Boyce-Codd Normal Form (BCNF) A stricter version of 3NF where every determinant is a candidate key.

4.3 Denormalization and When to Use It

Denormalization involves combining normalized tables to improve read performance, especially in scenarios where complex joins can be a performance bottleneck. It is used judiciously when the benefits of faster queries outweigh the costs of data redundancy.

Example: Design an ER diagram for a library system and normalize the tables up to 3NF.

Full Stack Development [Unit-II]

1. **Entities:** Books, Members, Transactions.
 2. **Relationships:** Members borrow Books through Transactions.
 3. **Normalization Steps:**
 - **1NF:** Ensure all tables have atomic values.
 - **2NF:** Separate tables so that non-key attributes fully depend on the primary key.
 - **3NF:** Remove transitive dependencies by ensuring non-key attributes depend only on the primary key.
-

Module 5: MySQL Data Types and Constraints

5.1 Data Types in MySQL

Numeric, Date and Time, String Types

- **Numeric Types:** INT, SMALLINT, TINYINT, FLOAT, DOUBLE, DECIMAL.
- **Date and Time Types:** DATE, DATETIME, TIMESTAMP, TIME, YEAR.
- **String Types:** CHAR, VARCHAR, TINYTEXT, TEXT, MEDIUMTEXT, LONGTEXT, BINARY, VARBINARY.

Choosing Appropriate Data Types Selecting the right data type is crucial for optimizing storage and performance. Consider the nature of the data, range of values, and required precision.

5.2 Constraints

Primary Key, Foreign Key

- **Primary Key:** Uniquely identifies each record in a table.

Full Stack Development [Unit-II]

- **Foreign Key:** Establishes a link between two tables, enforcing referential integrity.

NOT NULL, UNIQUE, DEFAULT

- **NOT NULL:** Ensures that a column cannot have NULL values.
- **UNIQUE:** Ensures all values in a column are unique.
- **DEFAULT:** Sets a default value for a column if none is provided.

CHECK Constraints Ensure that values in a column meet specific criteria.

```
CREATE TABLE example (  
  
    age INT,  
  
    CHECK (age >= 18)  
  
);
```

5.3 Indexes

Creating Indexes Indexes improve the speed of data retrieval operations.

```
CREATE INDEX index_name ON table_name (column1, column2);
```

Types of Indexes (Unique, Composite)

- **Unique Index:** Ensures that all values in the indexed column are unique.
- **Composite Index:** An index on multiple columns, improving query performance on those columns.

Index Optimization Optimize indexes by analyzing query patterns and ensuring that indexes are used effectively without causing excessive overhead on write operations.

Example: Modify the students table to add constraints and indexes.

```
ALTER TABLE students  
  
ADD CONSTRAINT unique_name UNIQUE (name),  
  
ADD INDEX idx_age (age);
```

Module 6: Stored Procedures, Functions, and Triggers

6.1 Stored Procedures

Creating and Calling Stored Procedures Stored procedures are precompiled SQL statements that can be executed multiple times with different parameters.

```
CREATE PROCEDURE procedure_name (IN param1 datatype, OUT param2 datatype)  
  
BEGIN  
  
    -- SQL statements  
  
END;  
  
CALL procedure_name(value1, @output);
```

Parameters and Control Structures Stored procedures can accept input (IN), output (OUT), and input-output (INOUT) parameters. They support control structures like loops and conditionals for complex operations.

6.2 User-Defined Functions

Creating Functions User-defined functions perform calculations or operations and return a single value.

Full Stack Development [Unit-II]

```
CREATE FUNCTION function_name (param1 datatype) RETURNS datatype
```

```
BEGIN
```

```
-- SQL statements
```

```
RETURN value;
```

```
END;
```

Using Functions in Queries Functions can be used within SQL statements to perform operations on data.

```
SELECT function_name(column) FROM table_name;
```

6.3 Triggers

Creating Triggers for INSERT, UPDATE, DELETE Triggers are special procedures that automatically execute in response to certain events on a table.

```
CREATE TRIGGER trigger_name
```

```
AFTER INSERT ON table_name
```

```
FOR EACH ROW
```

```
BEGIN
```

```
-- SQL statements
```

```
END;
```

Use Cases for Triggers

- **Auditing:** Automatically log changes to data.
- **Validation:** Enforce complex business rules.

Full Stack Development [Unit-II]

- **Cascading Actions:** Automatically update related tables.

Example: Create a stored procedure to add a new student and a trigger to log updates.

```
DELIMITER //
```

```
CREATE PROCEDURE AddStudent(IN studentName VARCHAR(100), IN studentAge INT, IN  
studentGrade CHAR(1))
```

```
BEGIN
```

```
    INSERT INTO students (name, age, grade) VALUES (studentName, studentAge, studentGrade);
```

```
END //
```

```
DELIMITER ;
```

```
CALL AddStudent('David', 21, 'B');
```

```
-- Trigger Example
```

```
CREATE TABLE update_log (
```

```
    log_id INT AUTO_INCREMENT PRIMARY KEY,
```

```
    student_id INT,
```

```
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
```

```
);
```

```
CREATE TRIGGER after_student_update
```

```
AFTER UPDATE ON students
```

```
FOR EACH ROW
```

Full Stack Development [Unit-II]

BEGIN

INSERT INTO update_log (student_id) VALUES (NEW.id);

END;

Module 7: Transactions and Concurrency Control

7.1 Understanding Transactions

ACID Properties (Atomicity, Consistency, Isolation, Durability)

- **Atomicity:** Ensures that all operations within a transaction are completed; if not, the transaction is aborted.
- **Consistency:** Ensures that a transaction brings the database from one valid state to another.
- **Isolation:** Ensures that concurrent transactions do not interfere with each other.
- **Durability:** Ensures that once a transaction is committed, it remains so, even in the event of a system failure.

7.2 Transaction Control Statements

START TRANSACTION, COMMIT, ROLLBACK

- **START TRANSACTION:** Begins a new transaction.

START TRANSACTION;

- **COMMIT:** Saves all changes made during the transaction.

COMMIT;

Full Stack Development [Unit-II]

- **ROLLBACK:** Reverts all changes made during the transaction.

ROLLBACK;

7.3 Isolation Levels

Read Uncommitted, Read Committed, Repeatable Read, Serializable

- **Read Uncommitted:** Allows dirty reads; transactions can see uncommitted changes.
- **Read Committed:** Prevents dirty reads; transactions only see committed changes.
- **Repeatable Read:** Ensures that if a row is read twice in the same transaction, the data remains the same.
- **Serializable:** The highest isolation level, ensuring complete isolation from other transactions.

7.4 Handling Deadlocks

Deadlocks occur when two or more transactions prevent each other from progressing. MySQL detects deadlocks and rolls back one of the transactions to resolve the issue. Best practices to avoid deadlocks include:

- **Consistent Order:** Access tables and rows in the same order across transactions.
- **Short Transactions:** Keep transactions as short as possible.
- **Proper Indexing:** Ensure efficient access paths to minimize lock contention.

Example: Demonstrate a transaction that transfers funds between two accounts.

START TRANSACTION;

Full Stack Development [Unit-II]

```
UPDATE accounts SET balance = balance - 100 WHERE account_id = 1;
```

```
UPDATE accounts SET balance = balance + 100 WHERE account_id = 2;
```

```
COMMIT;
```

Module 8: Security and User Management

8.1 MySQL Security Features

Authentication Methods MySQL supports various authentication plugins, including:

- **Native Password:** Default method using a hashing algorithm.
- **Caching SHA-2 Password:** More secure hashing mechanism.
- **External Authentication:** Integrates with external systems like LDAP.

Encryption in MySQL

- **Data Encryption:** Protects data at rest using features like Transparent Data Encryption (TDE).
- **Connection Encryption:** Secures data in transit using SSL/TLS protocols.

8.2 User Accounts and Privileges

Creating Users Create new MySQL user accounts with specific authentication credentials.

```
CREATE USER 'username'@'host' IDENTIFIED BY 'password';
```

Granting and Revoking Privileges Assign or remove specific permissions to users, controlling their access to databases and operations.

Full Stack Development [Unit-II]

```
GRANT SELECT, INSERT ON database_name.* TO 'username'@'host';
```

```
REVOKE DELETE ON database_name.* FROM 'username'@'host';
```

8.3 Best Practices for Securing MySQL

- **Use Strong Passwords:** Enforce complex passwords for all user accounts.
- **Least Privilege Principle:** Grant only the necessary permissions required for users to perform their tasks.
- **Regular Updates:** Keep MySQL and its dependencies updated to patch security vulnerabilities.
- **Secure Connections:** Use SSL/TLS for encrypting data transmitted between clients and the server.
- **Monitor and Audit:** Regularly monitor access logs and audit user activities to detect suspicious behavior.

Example: Create a new user with limited privileges

```
CREATE USER 'new_user'@'localhost' IDENTIFIED BY 'password123';
```

```
GRANT SELECT, INSERT ON students_db.* TO 'new_user'@'localhost';
```

```
FLUSH PRIVILEGES;
```

Module 9: Backup, Recovery, and Maintenance

9.1 Backup Strategies

Logical Backups (mysqldump) Logical backups involve exporting database structures and data into a format like SQL statements, which can be easily restored.

Full Stack Development [Unit-II]

```
mysqldump -u root -p database_name > backup.sql
```

Physical Backups (Copying Data Files) Physical backups involve copying the actual database files. This method is faster for large databases but requires careful handling to ensure data consistency.

9.2 Recovery Techniques

Restoring from Backups Use backup files to restore databases in case of data loss or corruption.

```
mysql -u root -p database_name < backup.sql
```

Point-in-Time Recovery Allows restoring the database to a specific moment before an incident occurred, using binary logs alongside backups.

9.3 Database Maintenance

Optimizing Tables Regularly optimize tables to improve performance and reclaim unused space.

```
OPTIMIZE TABLE table_name;
```

Monitoring Performance Use tools like MySQL Workbench, performance_schema, and third-party monitoring solutions to track database performance metrics.

Using EXPLAIN for Query Optimization Analyze query execution plans to identify and resolve performance bottlenecks.

```
EXPLAIN SELECT * FROM table_name WHERE condition;
```

Example: Perform a backup using mysqldump and restore it.

```
# Backup
```

```
mysqldump -u root -p students_db > students_db_backup.sql
```

Restore

```
mysql -u root -p students_db < students_db_backup.sql
```

Module 10: Advanced Topics

10.1 Views

Creating and Managing Views Views are virtual tables created by querying one or more tables. They simplify complex queries and enhance security by restricting data access.

```
CREATE VIEW view_name AS
```

```
SELECT column1, column2 FROM table_name WHERE condition;
```

Updatable vs. Non-Updatable Views

- **Updatable Views:** Allow data modification (INSERT, UPDATE, DELETE) if they meet certain criteria.
- **Non-Updatable Views:** Do not support data modification, typically due to complex queries involving joins or aggregations.

10.2 Index Optimization

Analyzing Query Performance Use tools like EXPLAIN to understand how queries are executed and identify potential optimizations.

Using EXPLAIN to Optimize Queries Analyze the output of EXPLAIN to determine if indexes are being used effectively and to identify areas for improvement.

Full Stack Development [Unit-II]

```
EXPLAIN SELECT * FROM table_name WHERE condition;
```

10.3 Partitioning

Types of Partitioning (Range, List, Hash)

- **Range Partitioning:** Divides data based on ranges of column values.
- **List Partitioning:** Divides data based on a list of discrete values.
- **Hash Partitioning:** Distributes data based on a hash function applied to a column.

Managing Partitioned Tables Create and maintain partitioned tables to improve performance and manageability.

```
CREATE TABLE table_name (  
  
    column1 datatype,  
  
    ...  
  
)  
  
PARTITION BY RANGE (column1) (  
  
    PARTITION p0 VALUES LESS THAN (100),  
  
    PARTITION p1 VALUES LESS THAN (200)  
  
);
```

10.4 Full-Text Search

Implementing Full-Text Indexes Enable efficient text searching within large text columns.

Full Stack Development [Unit-II]

```
CREATE FULLTEXT INDEX ft_index ON table_name(column_name);
```

Performing Full-Text Queries Use MATCH and AGAINST to perform full-text searches.

```
SELECT * FROM table_name
```

```
WHERE MATCH(column_name) AGAINST('search_term');
```

10.5 Replication and High Availability

Master-Slave Replication Set up a primary (master) server that replicates data to one or more secondary (slave) servers, enhancing data availability and scalability.

Setting Up Replication Configure replication by setting up binary logs on the master and connecting slaves to these logs.

```
-- On Master
```

```
CHANGE MASTER TO MASTER_HOST='master_host', MASTER_USER='replica_user',
```

```
MASTER_PASSWORD='password', MASTER_LOG_FILE='mysql-bin.000001',
```

```
MASTER_LOG_POS=107;
```

```
START SLAVE;
```

```
-- On Slave
```

```
CHANGE MASTER TO MASTER_HOST='master_host', MASTER_USER='replica_user',
```

```
MASTER_PASSWORD='password', MASTER_LOG_FILE='mysql-bin.000001',
```

```
MASTER_LOG_POS=107;
```

```
START SLAVE;
```

Full Stack Development [Unit-II]

Failover Strategies Implement strategies like automated failover using tools like MySQL Router or third-party solutions to ensure high availability in case of master server failure.

Example: Create a view for high-achieving students.

```
CREATE VIEW high_achievers AS
```

```
SELECT name, grade
```

```
FROM students
```

```
WHERE grade = 'A';
```

Module 11: MySQL with Programming Languages

11.1 Connecting MySQL with PHP

Using PDO and MySQLi PHP provides two main extensions for interacting with MySQL:

- **PDO (PHP Data Objects):** Offers a consistent interface for various databases and supports prepared statements.
- **MySQLi (MySQL Improved):** Provides enhanced features for MySQL, including support for transactions and prepared statements.

Example with PDO:

```
<?php
```

```
try {
```

```
    $pdo = new PDO('mysql:host=localhost;dbname=students_db', 'root', 'password123');
```

Full Stack Development [Unit-II]

```
$pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

// Insert a new student

$stmt = $pdo->prepare("INSERT INTO students (name, age, grade) VALUES (:name, :age,
:grade)");

$stmt->execute(['name' => 'Eve', 'age' => 23, 'grade' => 'B']);

// Fetch all students

$stmt = $pdo->query("SELECT * FROM students");

$students = $stmt->fetchAll(PDO::FETCH_ASSOC);

foreach ($students as $student) {

    echo "{$student['name']} - {$student['grade']}\n";

}

} catch (PDOException $e) {

    echo "Error: " . $e->getMessage();

}

?>
```

11.2 Connecting MySQL with Python

Using mysql-connector-python or SQLAlchemy

- **mysql-connector-python:** A MySQL driver written in Python.

Full Stack Development [Unit-II]

- **SQLAlchemy:** An ORM (Object-Relational Mapper) that provides a high-level interface for interacting with databases.

Example: Connect to MySQL using Python and perform basic operations.

```
import mysql.connector

# Establish connection

conn = mysql.connector.connect(

    host="localhost",

    user="root",

    password="password123",

    database="students_db"

)

cursor = conn.cursor()

# Insert a new student

cursor.execute("INSERT INTO students (name, age, grade) VALUES (%s, %s, %s)", ("Eve", 23, 'B'))

conn.commit()

# Fetch all students

cursor.execute("SELECT * FROM students")

for row in cursor.fetchall():

    print(row)
```

Full Stack Development [Unit-II]

Close connection

cursor.close()

conn.close()

11.3 Connecting MySQL with Java

Using JDBC Java Database Connectivity (JDBC) provides a standard API for connecting Java applications to databases.

Example: Connecting to MySQL using JDBC.

```
import java.sql.*;

public class MySQLConnection {

    public static void main(String[] args) {

        String url = "jdbc:mysql://localhost:3306/students_db";

        String user = "root";

        String password = "password123";

        try {

            // Load MySQL JDBC Driver

            Class.forName("com.mysql.cj.jdbc.Driver");

            // Establish connection

            Connection conn = DriverManager.getConnection(url, user, password);
```

Full Stack Development [Unit-II]

```
// Insert a new student

String insertQuery = "INSERT INTO students (name, age, grade) VALUES (?, ?, ?)";

PreparedStatement pstmt = conn.prepareStatement(insertQuery);

pstmt.setString(1, "Eve");

pstmt.setInt(2, 23);

pstmt.setString(3, "B");

pstmt.executeUpdate();

// Fetch all students

String selectQuery = "SELECT * FROM students";

Statement stmt = conn.createStatement();

ResultSet rs = stmt.executeQuery(selectQuery);

while (rs.next()) {

    System.out.println(rs.getInt("id") + " - " + rs.getString("name") + " - " + rs.getInt("age")
+ " - " + rs.getString("grade"));

}

// Close connections

rs.close();

stmt.close();

pstmt.close();
```

```
        conn.close();

    } catch (Exception e) {

        e.printStackTrace();

    }

}

}
```

Part 1: MongoDB

1. Introduction to MongoDB

- **What is MongoDB?**
 - A NoSQL, document-oriented database designed for scalability and flexibility.
- **Key Features**
 - Schema-less design, JSON-like documents (BSON), horizontal scalability, high performance.
- **Use Cases**
 - Real-time analytics, content management, IoT applications, etc.

2. Setting Up MongoDB

- **Installation**
 - Installing MongoDB on different operating systems (Windows, macOS, Linux).
- **MongoDB Atlas**
 - Setting up a cloud-hosted MongoDB database.
- **Basic Commands**
 - Starting and stopping the MongoDB server.
 - Using mongo shell.

3. MongoDB Data Model

- **Databases and Collections**

Full Stack Development [Unit-II]

- Understanding databases as namespaces and collections as tables.
- **Documents**
 - BSON format, data types, and structure.
- **Schema Design**
 - Embedding vs. referencing, normalization vs. denormalization.

Example:

```
{  
  
  "_id": ObjectId("507f1f77bcf86cd799439011"),  
  
  "name": "John Doe",  
  
  "email": "john.doe@example.com",  
  
  "age": 30,  
  
  "address": {  
  
    "street": "123 Main St",  
  
    "city": "Anytown",  
  
    "state": "CA"  
  
  },  
  
  "hobbies": ["reading", "gaming", "hiking"]  
  
}
```

4. CRUD Operations

- **Create**
 - insertOne(), insertMany().
- **Read**
 - find(), findOne(), query operators.
- **Update**
 - updateOne(), updateMany(), update operators (\$set, \$unset, \$inc).
- **Delete**
 - deleteOne(), deleteMany().

Example: Inserting a Document

```
db.users.insertOne({  
  
  name: "Jane Smith",  
  
  email: "jane.smith@example.com",  
  
  age: 25  
  
});
```

5. Querying Data

- **Basic Queries**
 - Filtering by fields, comparison operators (\$eq, \$gt, \$lt, etc.).
- **Advanced Queries**

Full Stack Development [Unit-II]

- Logical operators (\$and, \$or, \$not), regex searches, array queries.
- **Projection**
 - Selecting specific fields to return.
- **Sorting and Pagination**
 - sort(), limit(), skip().

Example: Finding Users Older Than 25

```
db.users.find({ age: { $gt: 25 } });
```

6. Indexing

- **Purpose of Indexes**
 - Improve query performance.
- **Types of Indexes**
 - Single-field, compound, multi-key, text, geospatial.
- **Creating and Managing Indexes**
 - createIndex(), listIndexes(), dropIndex().

Example: Creating an Index on the email Field

```
db.users.createIndex({ email: 1 }, { unique: true });
```

7. Aggregation Framework

- **Aggregation Pipeline**
 - Stages: \$match, \$group, \$sort, \$project, \$lookup, etc.

Full Stack Development [Unit-II]

- **Common Aggregation Operations**

- Grouping, filtering, transforming data.

- **Aggregation Examples**

Example: Counting Users by Age

```
db.users.aggregate([  
  
  { $group: { _id: "$age", count: { $sum: 1 } } }  
  
]);
```

8. Data Modeling Best Practices

- **Schema Design Considerations**

- Data access patterns, embedding vs. referencing.

- **Handling Relationships**

- One-to-one, one-to-many, many-to-many.

- **Normalization vs. Denormalization**

9. Replication and Sharding

- **Replication**

- Replica sets, data redundancy, failover.

- **Sharding**

- Horizontal scaling, shard keys, balancing.

10. Security in MongoDB

Full Stack Development [Unit-II]

- **Authentication**
 - Users, roles, and permissions.
- **Authorization**
 - Access control, role-based access.
- **Encryption**
 - Data encryption at rest and in transit.
- **Best Practices**
 - Securing connections, auditing.

11. Backup and Restore

- **Backup Strategies**
 - Logical backups using mongodump, mongorestore.
 - Cloud backups with MongoDB Atlas.
- **Restore Procedures**

12. Performance Tuning

- **Monitoring Tools**
 - MongoDB Compass, mongostat, mongotop, Atlas monitoring.
- **Optimization Techniques**
 - Index optimization, query optimization, hardware considerations.

Part 2: Mongoose

1. Introduction to Mongoose

- **What is Mongoose?**
 - An Object Data Modeling (ODM) library for MongoDB and Node.js.
- **Advantages of Using Mongoose**
 - Schema enforcement, data validation, middleware support, easier CRUD operations.

2. Setting Up Mongoose

- **Installation**
 - Using npm: `npm install mongoose`.
- **Connecting to MongoDB**
 - Establishing a connection using `mongoose.connect()`.

Example: Connecting to MongoDB

```
const mongoose = require('mongoose');

mongoose.connect('mongodb://localhost:27017/mydatabase', {

  useNewUrlParser: true,

  useUnifiedTopology: true

})

.then(() => console.log('Connected to MongoDB'))
```

Full Stack Development [Unit-II]

```
.catch(err => console.error('Connection error', err));
```

3. Defining Schemas and Models

- **Schemas**
 - Defining the structure of documents, data types, default values, validation.
- **Models**
 - Creating models from schemas to interact with collections.

Example: Defining a User Schema and Model

```
const userSchema = new mongoose.Schema({  
  
  name: { type: String, required: true },  
  
  email: { type: String, required: true, unique: true },  
  
  age: { type: Number, min: 0 },  
  
  address: {  
  
    street: String,  
  
    city: String,  
  
    state: String  
  
  },  
  
  hobbies: [String]  
  
});  
  
const User = mongoose.model('User', userSchema);
```

4. CRUD Operations with Mongoose

- **Create**
 - Using new Model(), save(), Model.create().
- **Read**
 - find(), findOne(), findById(), query helpers.
- **Update**
 - updateOne(), updateMany(), findByIdAndUpdate().
- **Delete**
 - deleteOne(), deleteMany(), findByIdAndDelete().

Example: Creating a New User

```
const newUser = new User({  
  
  name: 'Alice Johnson',  
  
  email: 'alice.johnson@example.com',  
  
  age: 28  
  
});  
  
newUser.save()  
  
  .then(user => console.log('User saved:', user))  
  
  .catch(err => console.error(err));
```


5. Validation

- **Built-in Validators**
 - Required fields, data type validation, min/max values, enum.
- **Custom Validators**
 - Defining custom validation logic.

Example: Adding a Custom Validator for Email Format

```
const userSchema = new mongoose.Schema({  
  
  email: {  
  
    type: String,  
  
    required: true,  
  
    unique: true,  
  
    validate: {  
  
      validator: function(v) {  
  
        return /^[^\\s@]+@[^\\s@]+\\.([^\\s@]+)$/.test(v);  
  
      },  
  
      message: props => `${props.value} is not a valid email!`  
  
    }  
  
  }  
  
})
```

Full Stack Development [Unit-II]

```
});
```

6. Middleware (Hooks)

- **Pre and Post Hooks**

- `pre('save')`, `post('save')`, `pre('find')`, etc.

- **Use Cases**

- Data modification before saving, logging, validation.

Example: Pre-save Hook to Hash Passwords

```
const bcrypt = require('bcrypt');

userSchema.pre('save', async function(next) {

  if (this.isModified('password')) {

    this.password = await bcrypt.hash(this.password, 10);

  }

  next();

});
```

7. Virtuals and Getters/Setters

- **Virtuals**

- Computed properties not stored in the database.

- **Getters and Setters**

- Transforming data when getting or setting values.

Full Stack Development [Unit-II]

Example: Virtual Property for Full Name

```
userSchema.virtual('fullName').get(function() {  
  
  return `${this.firstName} ${this.lastName}`;  
  
});
```

8. Population (Handling References)

- **Referencing Documents**
 - Using ObjectId to reference documents in other collections.
- **Populating References**
 - Using populate() to fetch related documents.

Example: Defining and Populating References

```
const postSchema = new mongoose.Schema({  
  
  title: String,  
  
  content: String,  
  
  author: { type: mongoose.Schema.Types.ObjectId, ref: 'User' }  
  
});  
  
const Post = mongoose.model('Post', postSchema);  
  
// Fetching a post with populated author  
  
Post.findOne({ title: 'Sample Post' })  
  
  .populate('author')
```

Full Stack Development [Unit-II]

```
.then(post => console.log(post))
```

```
.catch(err => console.error(err));
```

9. Query Helpers and Plugins

- **Query Helpers**

- Adding reusable query logic.

- **Plugins**

- Extending Mongoose functionality with third-party or custom plugins.

Example: Using a Plugin for Timestamps

```
const timestampPlugin = function(schema, options) {  
  
  schema.add({ createdAt: Date, updatedAt: Date });  
  
  schema.pre('save', function(next) {  
  
    const now = Date.now();  
  
    this.updatedAt = now;  
  
    if (!this.createdAt) {  
  
      this.createdAt = now;  
  
    }  
  
    next();  
  
  });  
  
};
```

```
userSchema.plugin(timestampPlugin);
```

10. Transactions

- **Using Transactions with Mongoose**
 - Ensuring atomic operations across multiple documents or collections.
- **Session Management**

Example: Performing a Transaction

```
const session = await mongoose.startSession();

session.startTransaction();

try {

  const opts = { session };

  await User.updateOne({ _id: userId }, { $inc: { balance: -amount } }, opts);

  await User.updateOne({ _id: recipientId }, { $inc: { balance: amount } }, opts);

  await session.commitTransaction();

  session.endSession();

} catch (error) {

  await session.abortTransaction();

  session.endSession();

  throw error;
```

```
}
```

11. Error Handling

- **Handling Validation Errors**
 - Catching and processing Mongoose validation errors.
- **Handling Database Errors**
 - Duplicate key errors, connection errors.

Example: Handling Validation Errors

```
newUser.save()

.then(user => console.log('User saved:', user))

.catch(err => {

  if (err.name === 'ValidationError') {

    console.error('Validation Error:', err.message);

  } else {

    console.error(err);

  }

});
```

Serverless Computing

Serverless computing is a cloud computing model where developers can build and deploy applications without having to manage the underlying infrastructure. The cloud provider automatically provisions, scales, and manages the servers. It allows developers to focus on writing code, while the provider handles server management tasks like patching, scaling, and maintenance.

In serverless computing, the term "serverless" doesn't mean that servers are not used, but rather that the user does not need to be concerned with the servers at all.

Key Features

1. **No Server Management:** Developers don't need to provision or maintain servers. The cloud provider automatically manages the infrastructure.
2. **Automatic Scaling:** The serverless platform automatically scales the application in response to the workload. For example, if your app has a spike in traffic, the serverless platform will allocate more resources without intervention.
3. **Pay-as-You-Go:** You only pay for the exact amount of compute time you consume. This is typically based on the number of requests or the time your code runs (measured in milliseconds).
4. **Event-Driven Execution:** Serverless functions are often triggered by events, such as HTTP requests, database changes, or file uploads.

Examples of Serverless Computing Platforms

1. AWS Lambda

Full Stack Development [Unit-II]

- **Overview:** AWS Lambda is Amazon's serverless computing service. It allows you to run code without provisioning or managing servers. You upload your code, and AWS Lambda executes it in response to triggers (like changes in data, HTTP requests via API Gateway, or scheduled events).
- **Key Features:**
 - Supports multiple programming languages: Python, Node.js, Java, Go, and more.
 - Integrated with AWS services like S3, DynamoDB, and CloudWatch.
 - Automatically scales depending on the number of requests.
- **Example Use Case:**
 - **Image Processing Pipeline:** An e-commerce site uses AWS Lambda to automatically resize and optimize user-uploaded images. The Lambda function is triggered when an image is uploaded to an S3 bucket, performs the resizing, and stores the optimized image back in another S3 bucket.

```
import boto3

def lambda_handler(event, context):

    s3_client = boto3.client('s3')

    # Triggered when an image is uploaded to the S3 bucket

    source_bucket = event['Records'][0]['s3']['bucket']['name']

    key = event['Records'][0]['s3']['object']['key']

    # Process the image (e.g., resize, compress) here
```


Save the processed image to another S3 bucket

2. Azure Functions

- **Overview:** Azure Functions is Microsoft's serverless offering. Like AWS Lambda, it enables developers to run event-triggered code without worrying about server infrastructure.
- **Key Features:**
 - Supports multiple programming languages (C#, JavaScript, Python, etc.).
 - Native integration with Microsoft Azure services such as Azure Blob Storage and Azure Cosmos DB.
 - Offers a durable function feature that manages long-running workflows.
- **Example Use Case:**
 - **Real-time Data Processing:** A company uses Azure Functions to process data in real-time from IoT devices. Whenever a device sends data to Azure Event Hubs, the function triggers, processes the data (like anomaly detection), and stores it in a database.

```
[FunctionName("IoTDataProcessor")]
```

```
public static async Task Run(
```

```
[EventHubTrigger("myHub", Connection = "EventHubConnectionAppSetting")] EventData[] events,
```

```
ILogger log)
```

```
{
```

```
    foreach (EventData eventData in events)
```

```
    {
```

Full Stack Development [Unit-II]

```
string data = Encoding.UTF8.GetString(eventData.Body.Array);

// Process the incoming IoT data

log.LogInformation($"Processed event: {data}");

}

}
```

Benefits of Serverless Computing

1. **Cost-Efficiency:** You only pay for the actual usage rather than pre-allocating server capacity. This makes it ideal for unpredictable workloads.
2. **Simplified Development:** Developers can focus purely on application code, as the cloud provider handles server provisioning, patching, scaling, and other operational concerns.
3. **Scalability:** Automatically scales up and down based on demand without the need for manual intervention.
4. **Faster Time to Market:** Since developers don't need to manage infrastructure, they can rapidly deploy and iterate on their applications.

Drawbacks of Serverless Computing

1. **Cold Starts:** When a function has not been used for a while, there may be a delay (cold start) as the server initializes.
2. **Limited Execution Time:** Many serverless platforms have limits on execution time, which might not be suitable for long-running applications.

3. **Vendor Lock-in:** Relying heavily on a specific cloud provider's serverless platform may make it difficult to migrate to another platform in the future.
4. **Complex Debugging:** Debugging serverless applications can be more complex since they are spread across many small functions and events.